操作系统

Operating Systems

刘川意 教授
liuchuanyi@hit.edu.cn

哈尔滨工业大学（深圳）

2025年10月

Module 5: Virtualizing Memory 虚拟内存

1. 虚拟地址空间（基础）
2. 内存管理API（memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Virtual Address

- **Every address** in a running program is virtual 用户态程序的内存地址是虚拟的
  - OS translates the virtual address to physical address 由OS内核完成虚拟地址到物理地址的转换

```
1.  #include <stdio.h>
2.  #include <stdlib.h>

3.  int main(int argc, char *argv[]){

4.      printf("location of code  : %p\n", (void *) main);
5.      printf("location of heap  : %p\n", (void *) malloc(1));
6.      int x = 3;
7.      printf("location of stack : %p\n", (void *) &x);

8.      return x;
9.  }
```

A simple program that prints out addresses

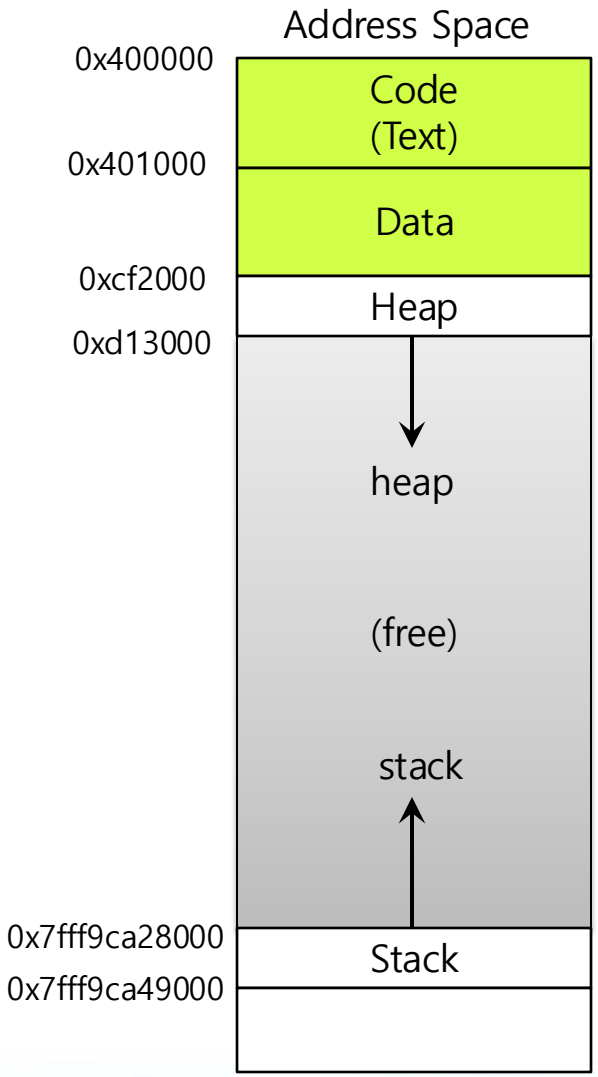
```
location of code  :0x556f705fb155
location of heap  :0x556f70ead670
location of stack :0x7ffcdc4cde64
```

- code： 存放程序执行代码
- stack：由编译器自动分配释放，存放函数的参数值，局部变量等值
- heap：存放进程运行中被动态分配的内存

Virtual Address(Cont.)

- The output in 64-bit Linux machine

```
location of code  : 0x40057d
location of heap  : 0xcf2010
location of stack : 0x7fff9ca45fcc
```



Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（memory API ）
   a) 基础函数：malloc(), sizeof, free() （基础）
   b) 内存操作的常见错误 （基础）
   c) 其他内存API及错误 （进阶）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Memory API: malloc()

```
#include <stdlib.h>

void* malloc(size_t size)
```

- Allocate a memory region on the heap.
  - Argument
    - `size_t size` : size of the memory block(in bytes)
    - `size_t` is an unsigned integer type.
  - Return
    - Success : a void type pointer to the memory block allocated by `malloc`
    - Fail : a null pointer

sizeof()

- Routines and macros are utilized for `size` in `malloc` instead typing in a number directly.
- Two types of results of `sizeof` with variables
  - The actual size of 'x' is known at run-time.

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

```
4
```

  - The actual size of 'x' is known at compile-time.

```
int x[10];
printf("%d\n", sizeof(x));
```

```
40
```

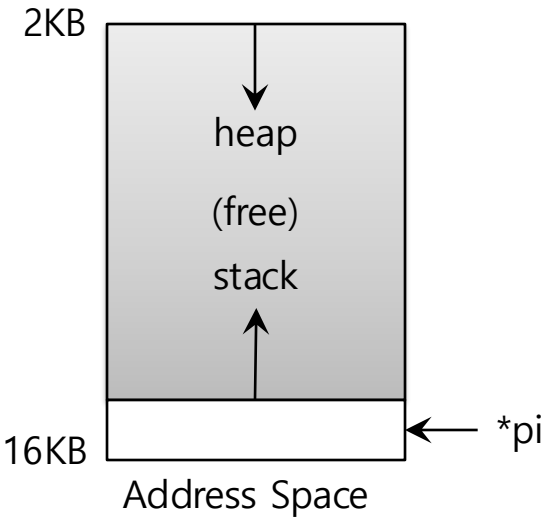
Memory API: free()

```
#include <stdlib.h>

void free(void* ptr)
```

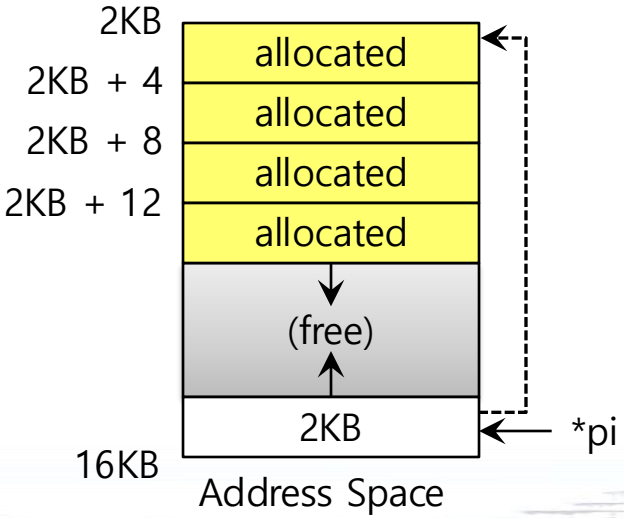
- Free a memory region allocated by a call to `malloc`.
  - Argument
    - `void *ptr` : a pointer to a memory block allocated with `malloc`
  - Return
    - none

Memory Allocating



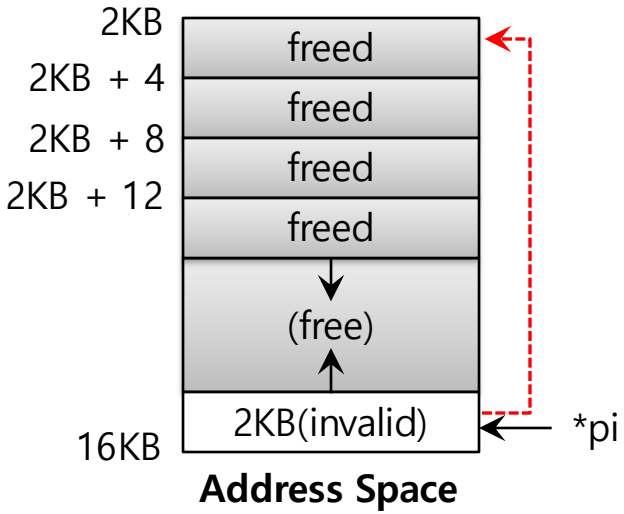
pointer

```
int *pi; // local variable
```

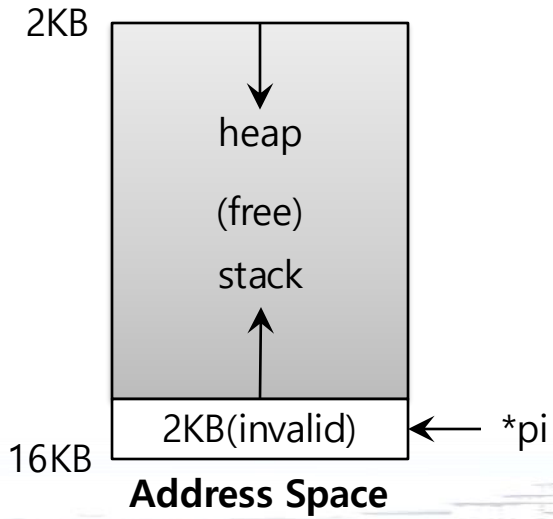



```
pi = (int *)malloc(sizeof(int)* 4);
```

Memory Freeing




```
free(pi);
```



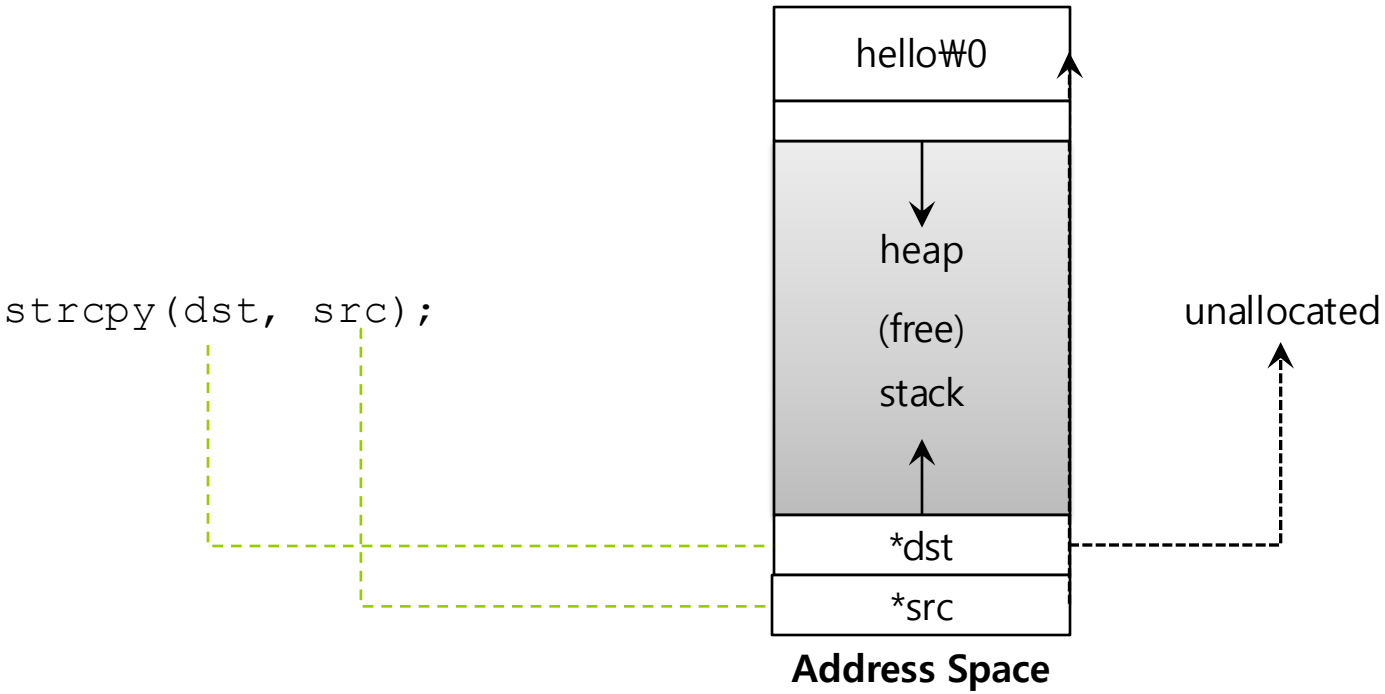
Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（memory API ）
   a) 基础函数：malloc(), sizeof, free()（基础）
   b) 内存操作的常见错误（基础）
   c) 其他内存API及错误（进阶）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Forgetting To Allocate Memory

- Incorrect code

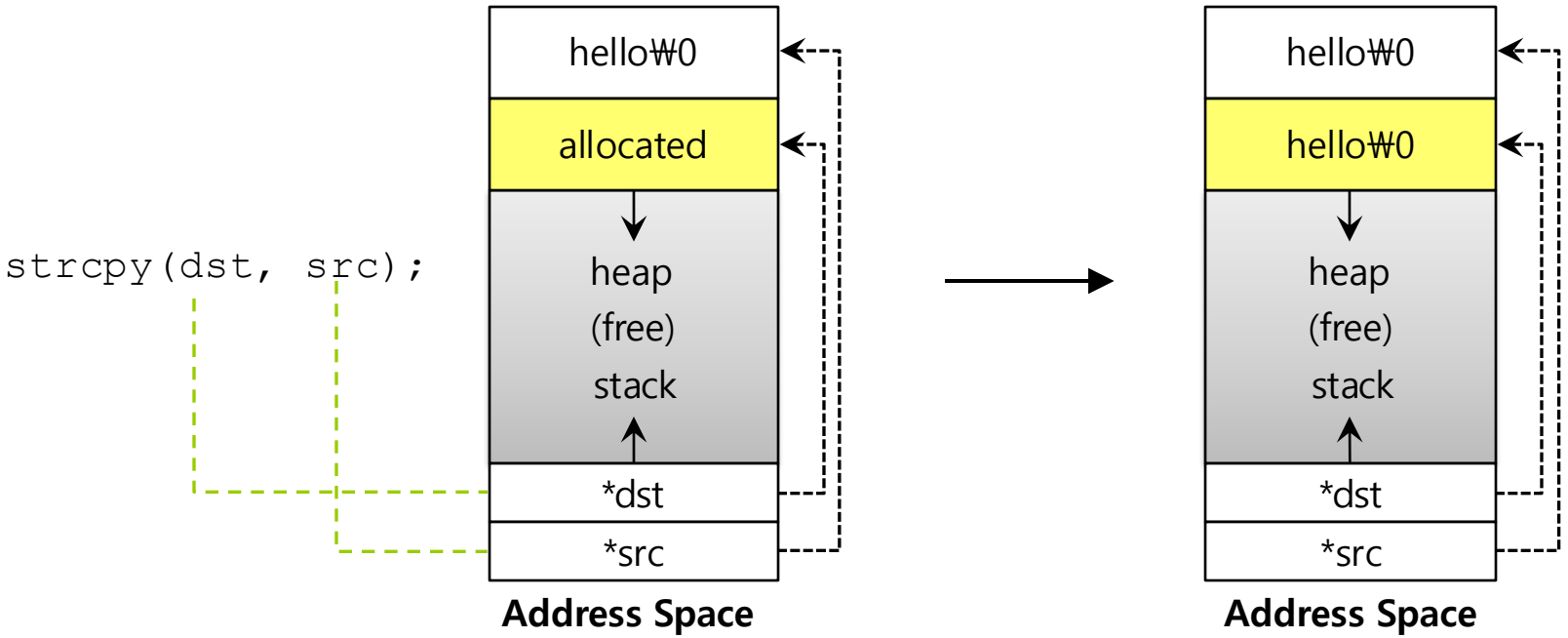
```
char *src = "hello"; //character string constant
char *dst;           //unallocated
strcpy(dst, src);    //segfault and die
```



Forgetting To Allocate Memory(Cont.)

- Correct code

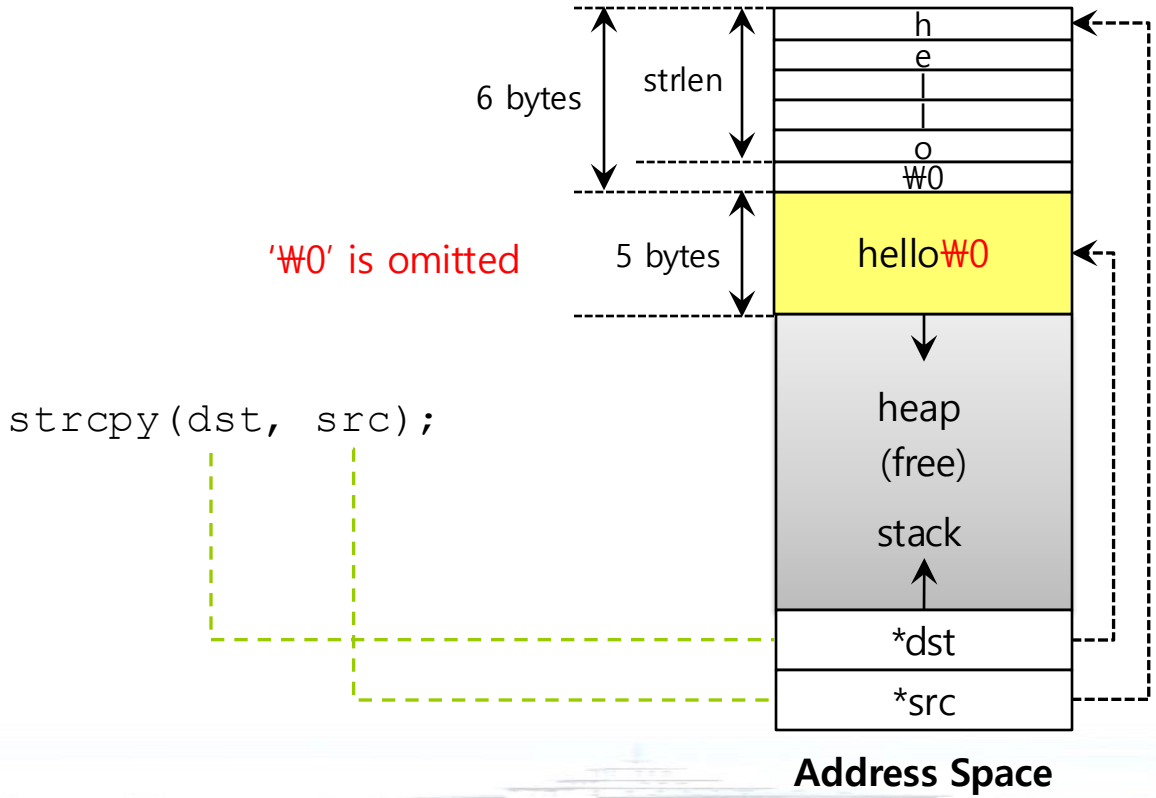
```
char *src = "hello";   //character string constant
char *dst (char *)malloc(strlen(src) + 1 ); // allocated
strcpy(dst, src);      //work properly
```



Not Allocating Enough Memory

- Incorrect code, but work properly

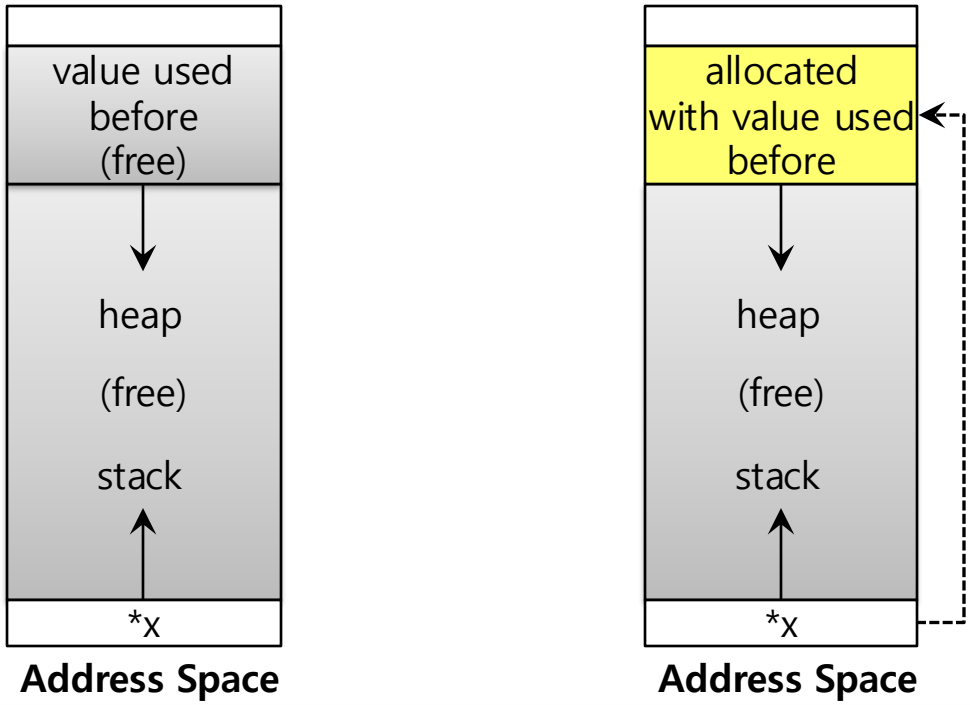
```
char *src = "hello"; //character string constant
char *dst (char *)malloc(strlen(src)); // too small
strcpy(dst, src);    //work properly
```



Forgetting to Initialize

- Encounter an uninitialized read

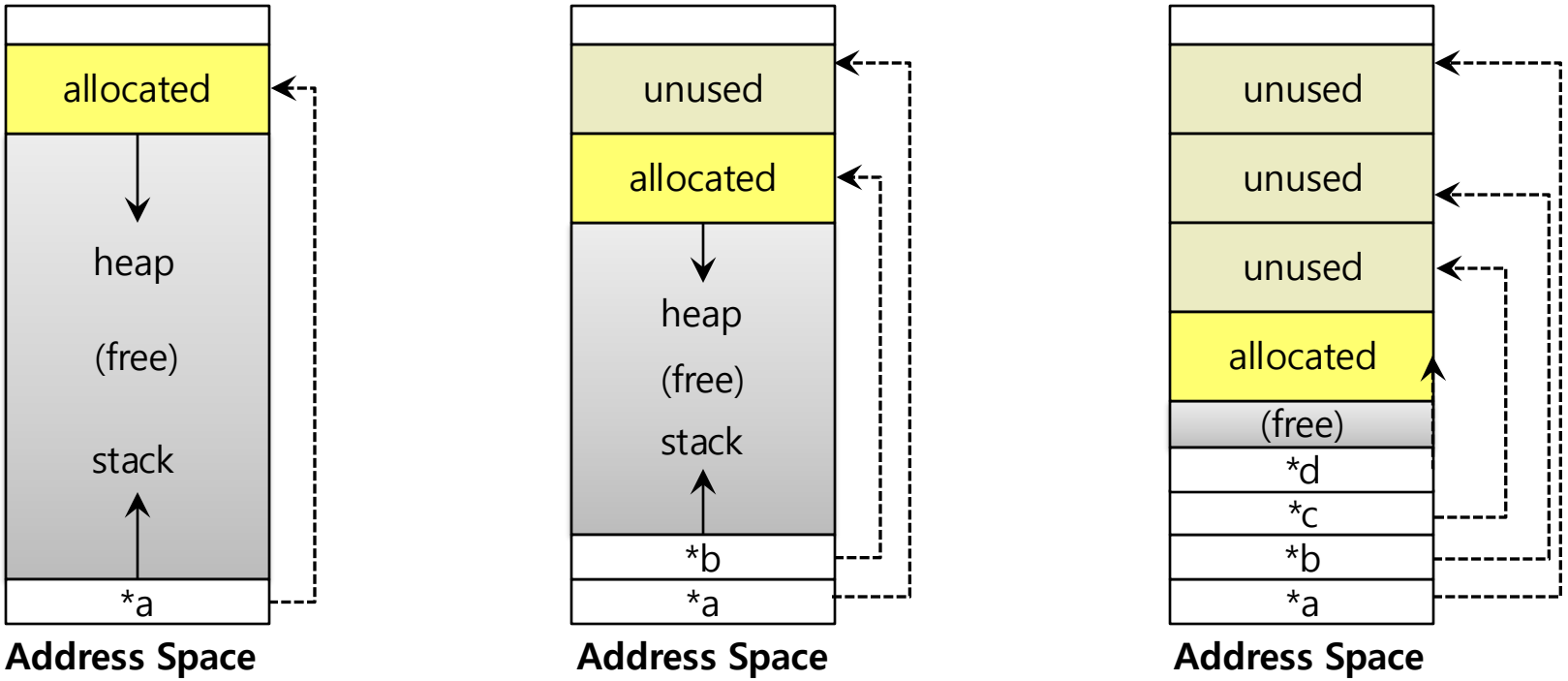
```
int *x = (int *)malloc(sizeof(int)); // allocated
printf("*x = %d\n", *x); // uninitialized memory access
```



Memory Leak

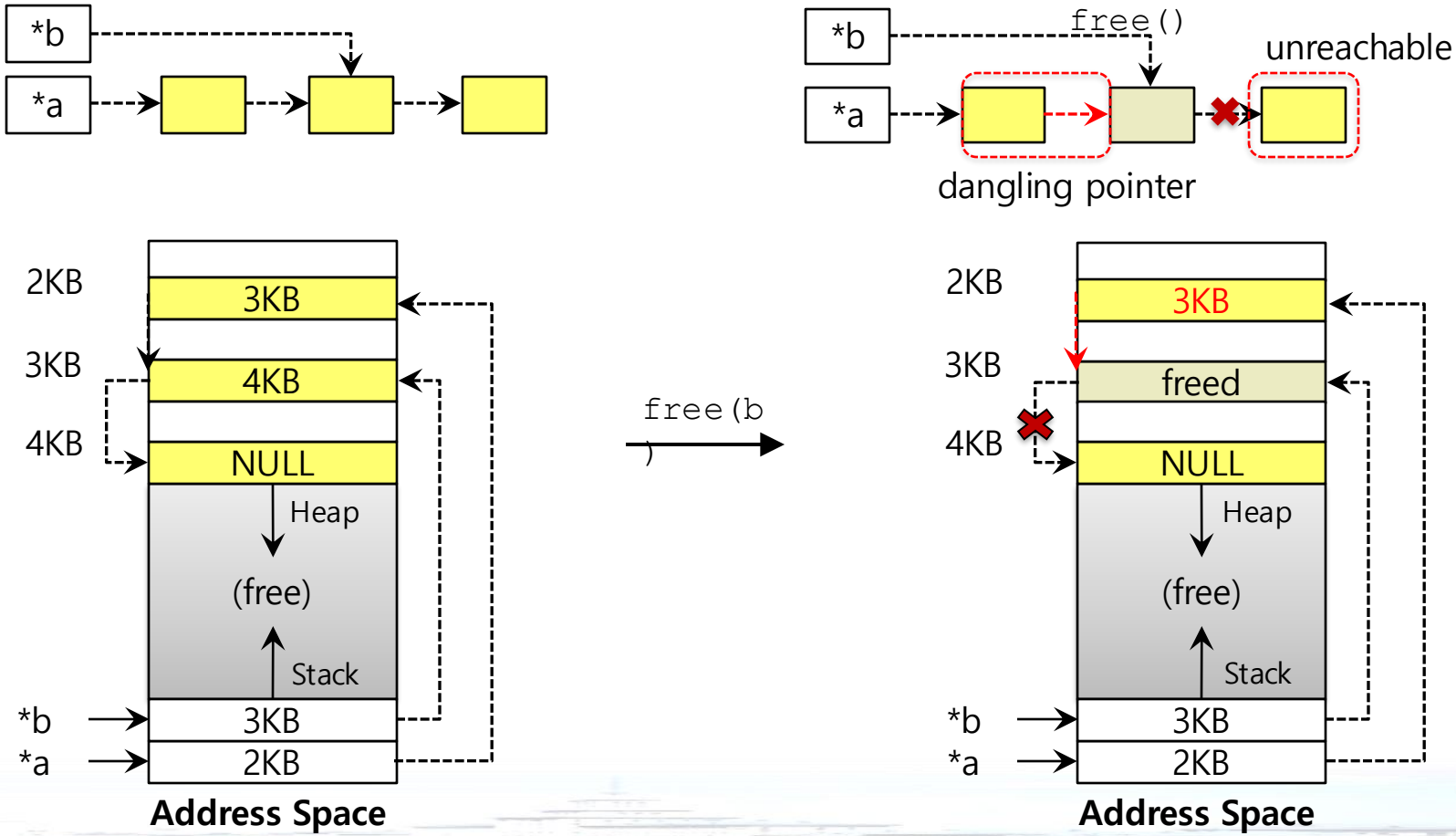
- A program runs out of memory and eventually dies.

unused : unused, but not freed



Dangling Pointer

- Freeing memory before it is finished using
  - A program accesses to memory with an invalid pointer



Module 5: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（memory API ）
   a) 基础函数：malloc(), sizeof, free()（基础）
   b) 内存操作的常见错误（基础）
   c) 其他内存API及错误（进阶）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Other Memory APIs: calloc()

```
#include <stdlib.h>

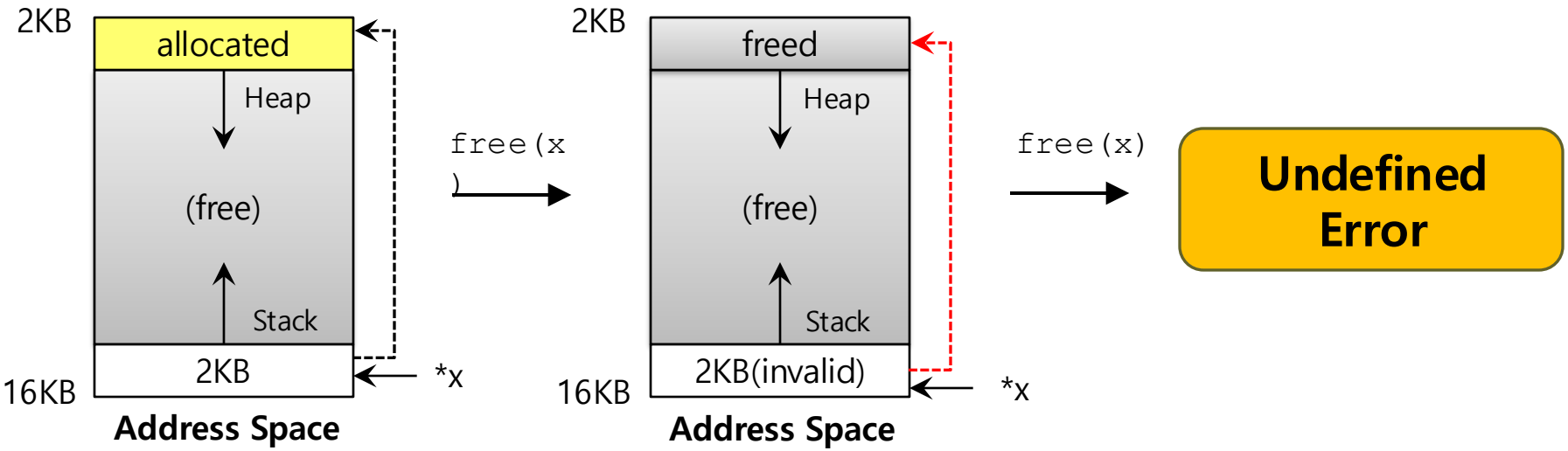
void *calloc(size_t num, size_t size)
```

- Allocate memory on the heap and zeroes it before returning.
  - Argument
    - `size_t num` : number of blocks to allocate
    - `size_t size` : size of each block(in bytes)
  - Return
    - Success : a void type pointer to the memory block allocated by `calloc`
    - Fail : a null pointer

Double Free

- Free memory that was freed already.

```
int *x = (int *)malloc(sizeof(int)); // allocated
free(x); // free memory
free(x); // free repeatedly
```



Other Memory APIs: realloc()

```
#include <stdlib.h>

void *realloc(void *ptr, size_t size)
```

- Change the size of memory block.
  - A pointer returned by `realloc` may be either the same as `ptr` or a new.
  - Argument
    - `void *ptr`: Pointer to memory block allocated with `malloc`, `calloc` or `realloc`
    - `size_t size`: New size for the memory block(in bytes)
  - Return
    - Success: Void type pointer to the memory block
    - Fail : Null pointer

System Calls

```
#include <unistd.h>

int brk(void *addr)
void *sbrk(intptr_t increment);
```

- `malloc` library call use `brk` system call.
  - `brk` is called to expand the program's *break*.
    - *break*: The location of **the end of the heap** in address space
  - `sbrk` is an additional call similar with `brk`.
  - Programmers **should never directly call** either `brk` or `sbrk`.

System Calls(Cont.)


```
#include <sys/mman.h>

void *mmap(void *ptr, size_t length, int port, int flags,
int fd, off_t offset)
```

- `mmap` system call can create **an anonymous** memory region.

Module 5: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（memory API ）
3. Mechanism: 地址翻译（Address Translation）
   a) 地址翻译（基础）
   b) 重定位地址空间（基础）
   c) 内存虚拟化中OS要做什么（基础）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Memory Virtualizing with Efficiency and Control

- Memory virtualizing takes a similar strategy known as **limited direct execution(LDE)** （有限直接执行） for efficiency and control.
- In memory virtualizing, efficiency and control are attained by hardware support.
  - e.g., registers, TLB(Translation Look-aside Buffer)s, page-table

Address Translation

- Hardware transforms a **virtual address** to a **physical address**.
  - The desired information is actually stored in a physical address.

- The OS must get involved at key points to set up the hardware.
  - The OS must manage memory to judiciously intervene.

Example: Address Translation

- C - Language code

```
void func()
        int x;
        ...
        x = x + 3; // this is the line of code we are interested in
```

- **Load** a value from memory
- **Increment** it by three
- **Store** the value back into memory

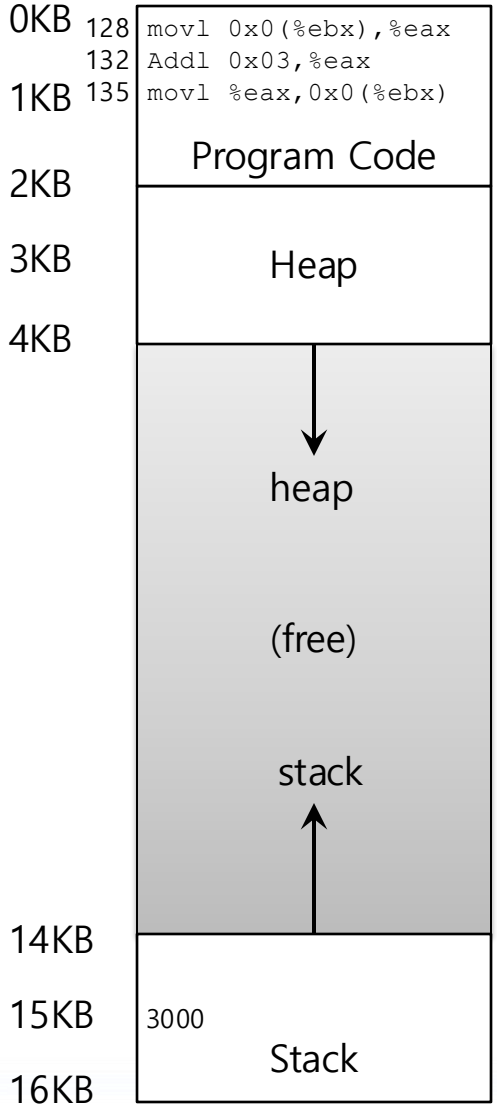
Example: Address Translation(Cont.)

- Assembly

```
128 : movl 0x0(%ebx), %eax      ; load 0+ebx into eax
132 : addl $0x03, %eax          ; add 3 to eax register
135 : movl %eax, 0x0(%ebx)      ; store eax back to mem
```

  - **Load** the value at that address into `eax` register.
  - **Add** 3 to `eax` register.
  - **Store** the value in `eax` back into memory.

Example: Address Translation(Cont.)



- Fetch instruction at address 128
- Execute this instruction (load from address 15KB)
- Fetch instruction at address 132
- Execute this instruction (no memory reference)
- Fetch the instruction at address 135
- Execute this instruction (store to address 15 KB)

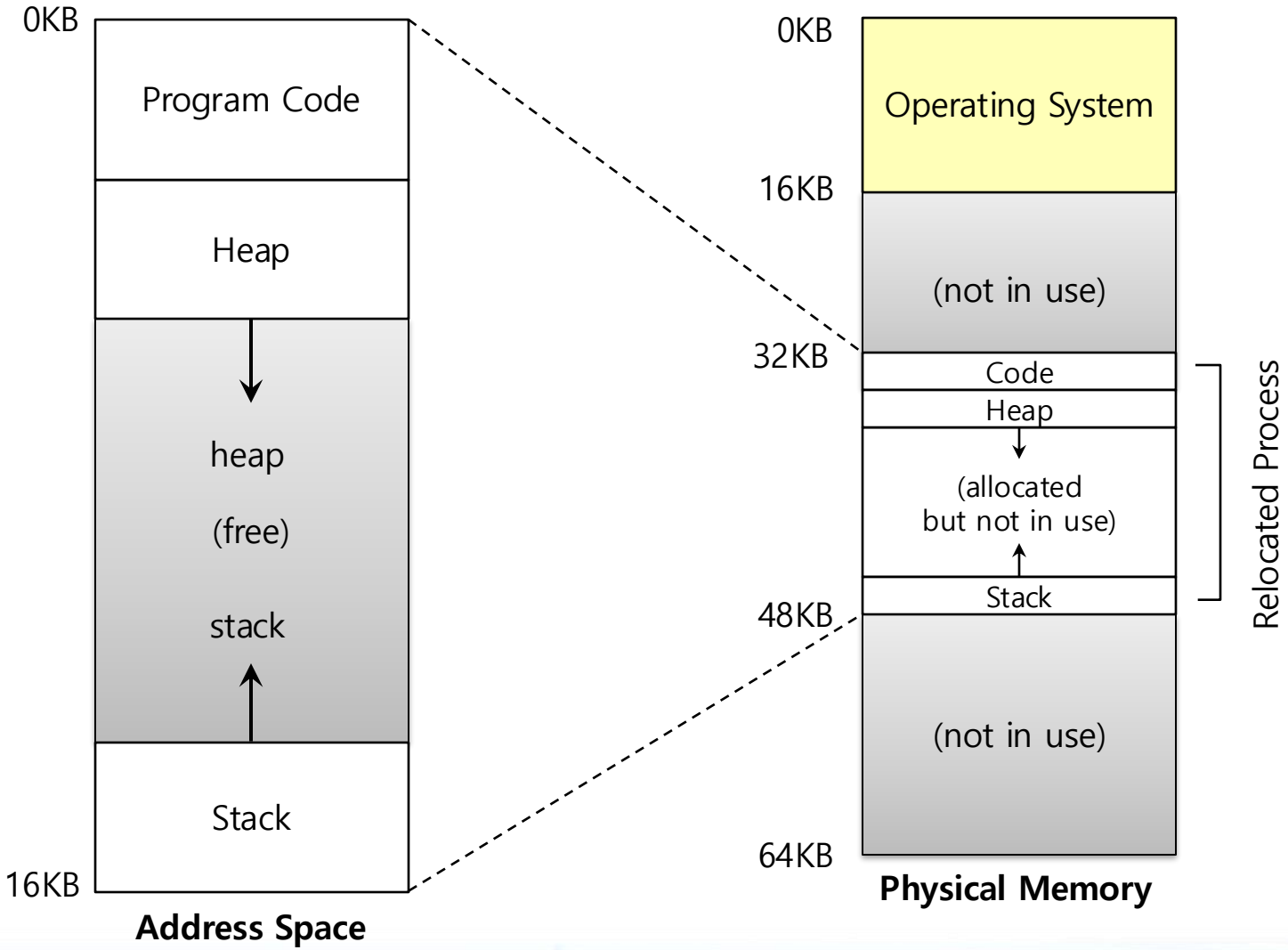
Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（memory API ）
3. Mechanism: 地址翻译（Address Translation）
   a) 地址翻译（基础）
   b) 重定位地址空间（基础）
   c) 内存虚拟化中OS要做什么（基础）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

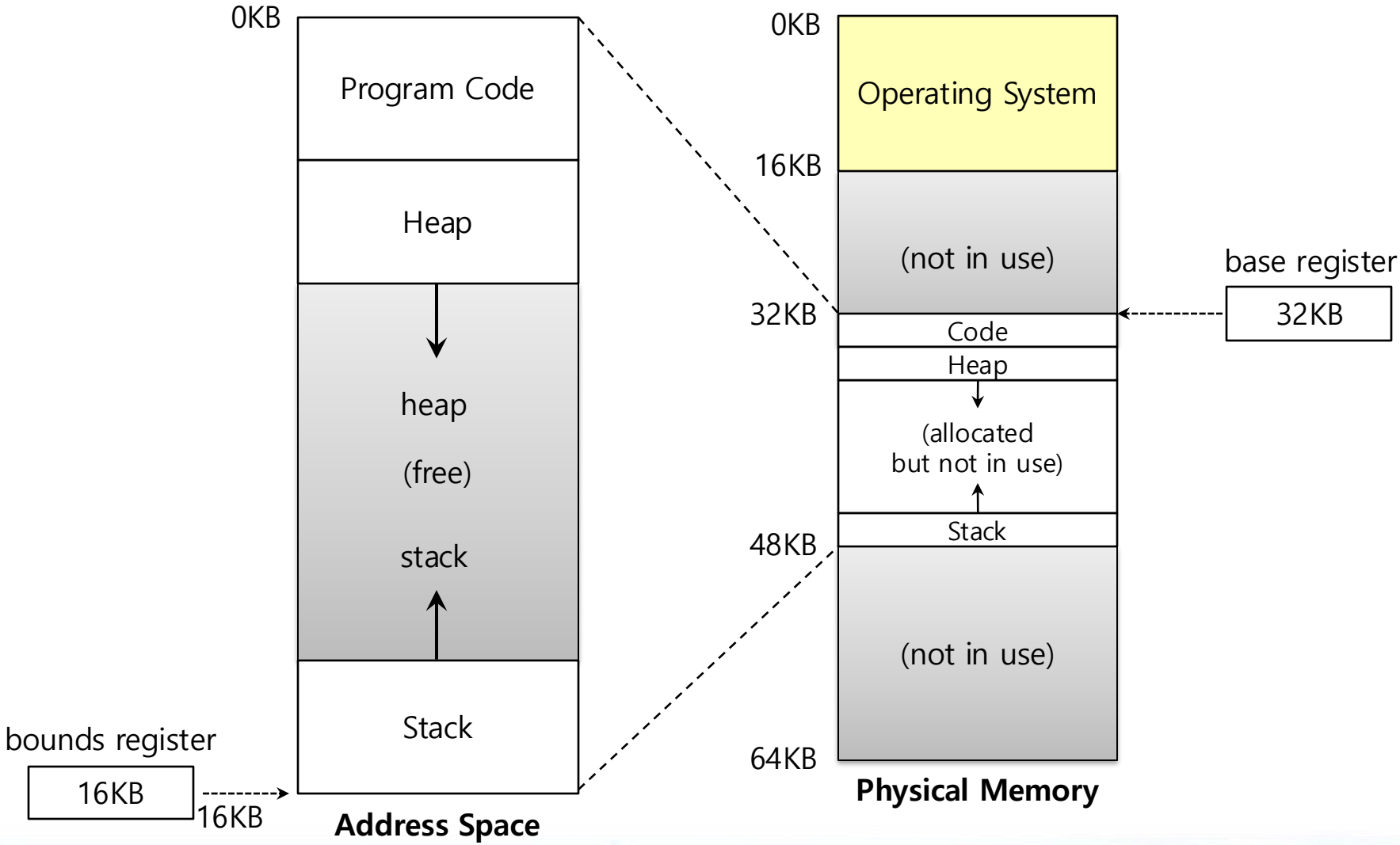
Relocation Address Space

- The OS wants to place the process **somewhere else** in physical memory, not at address 0.
  - The address space start at address 0.

A Single Relocated Process



Base and Bounds Register



Dynamic(Hardware base) Relocation

- When a program starts running, the OS decides **where** in physical memory a process should be **loaded**.
  - Set the **base** register a value.

$$phycal\ address = virtual\ address + base$$

  - Every virtual address must **not be greater than bound** and **negative.**

$$0 \leq\ virtual\ addressvirtual\ address < bounds$$

Relocation and Address Translation

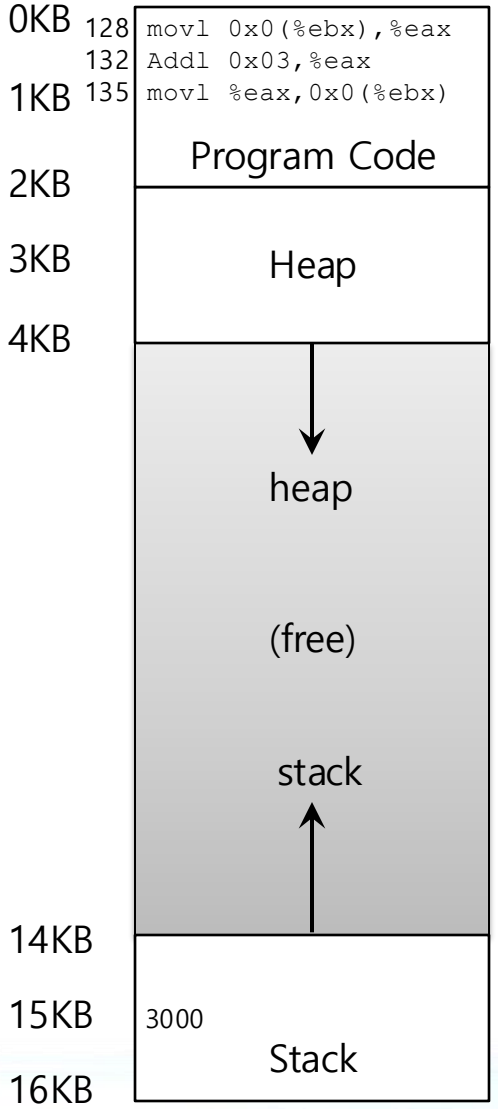
```
128 : movl 0x0(%ebx), %eax
```

- **Fetch** instruction at address 128

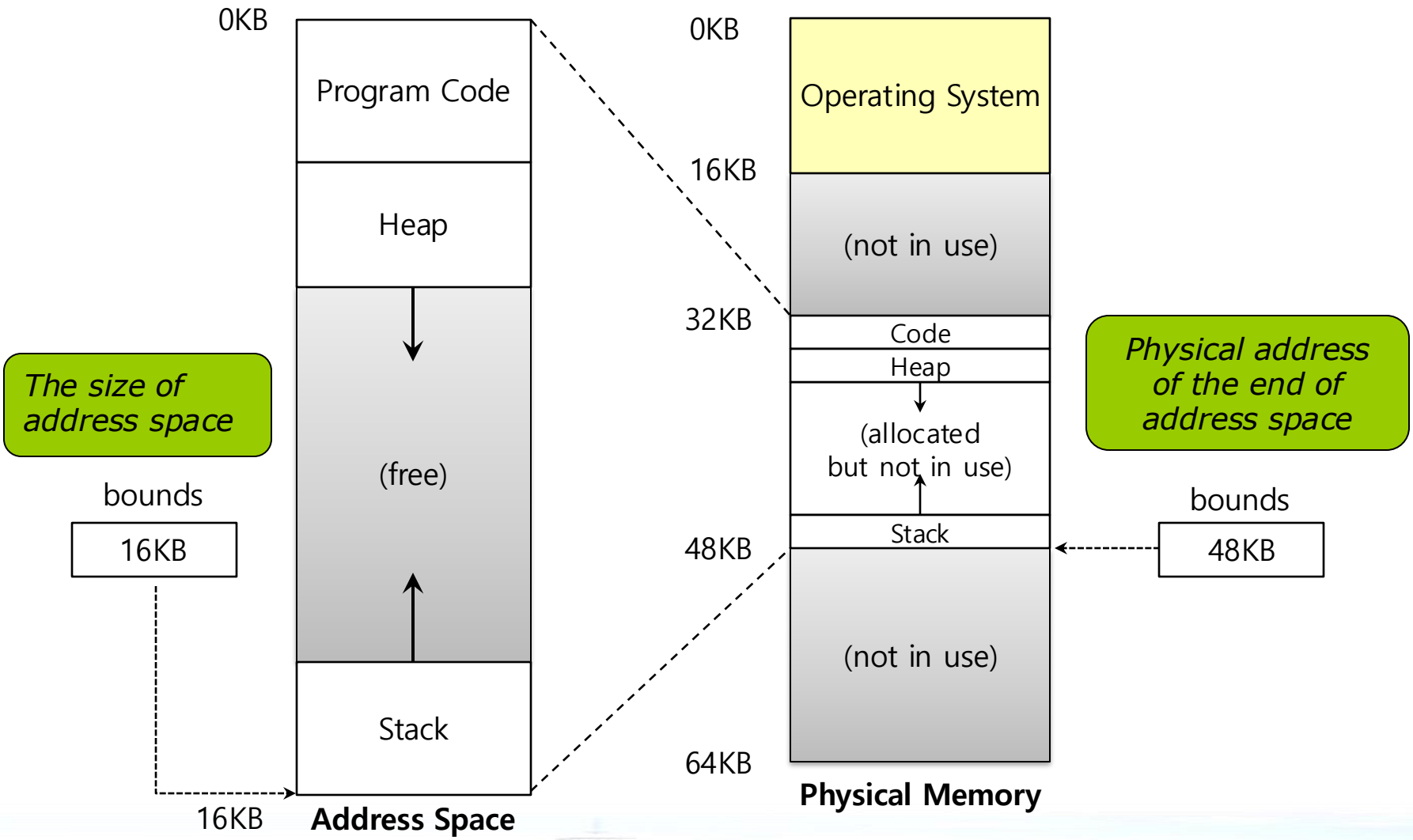
32896 = 128 + 32KB(base)

- **Execute** this instruction
  - Load from address 15KB

47KB = 15KB + 32KB(base)



Two ways of Bounds Register



Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（memory API ）
3. Mechanism: 地址翻译（Address Translation）
   a) 地址翻译（基础）
   b) 重定位地址空间（基础）
   c) 内存虚拟化中OS要做什么（基础）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

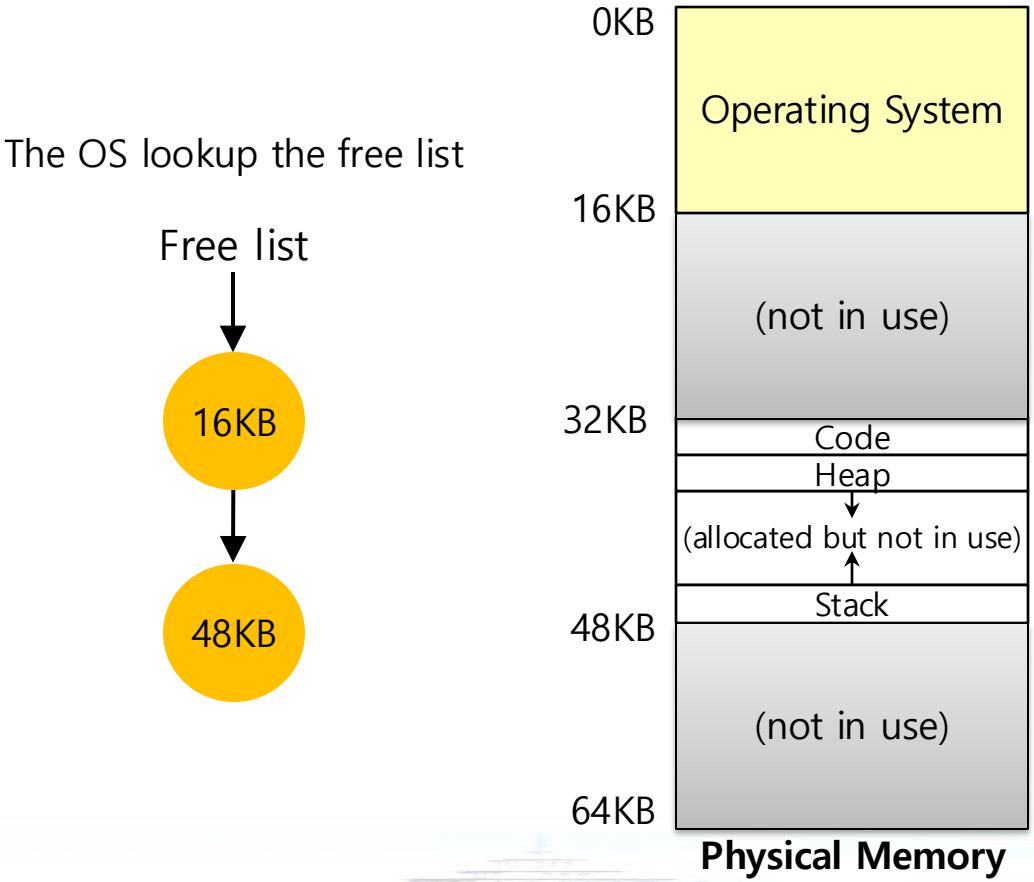
OS Issues for Memory Virtualizing

- The OS must **take action** to implement **base-and-bounds** approach.
- Three critical junctures OS需要承担3个关键的接合点
  - When a process **starts running:**
    - Finding space for address space in physical memory 在物理内存中为该进程分配空间
  - When a process is **terminated:**
    - Reclaiming the memory for use 回收该进程的物理空间
  - When context **switch occurs:**
    - Saving and storing the base-and-bounds pair 保存和恢复相应进程的base-and-bounds值对

OS Issues: When a Process Starts Running

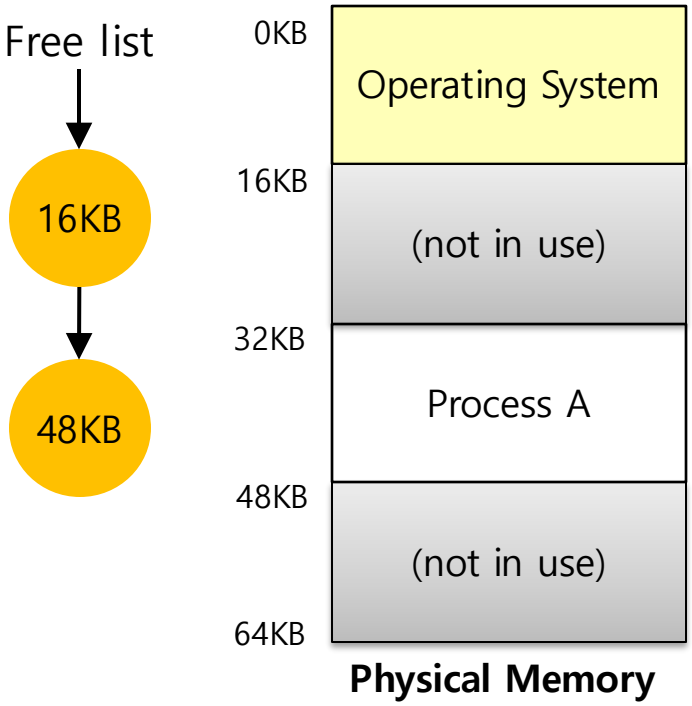
- The OS must **find a room** (空间) for a new address space.
  - free list : A list of the range of the physical memory which are not in use.

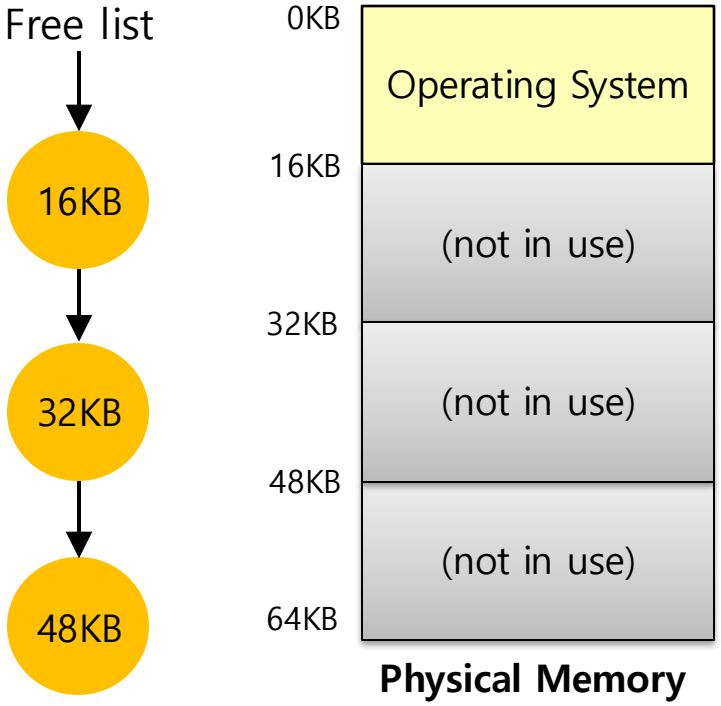
The OS lookup the free list



OS Issues: When a Process Is Terminated

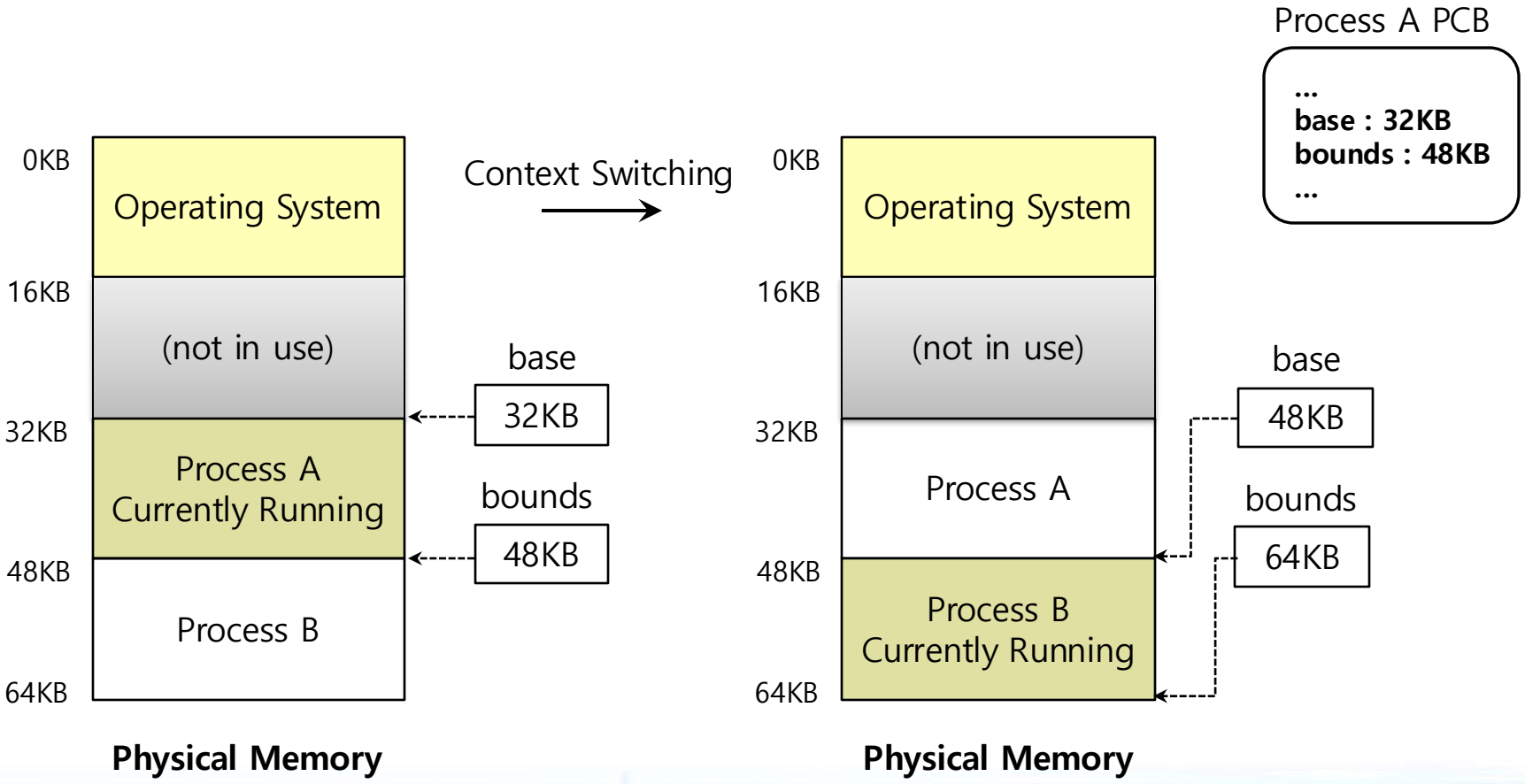
- The OS must **put the memory back** on the free list.





OS Issues: When Context Switch Occurs

- The OS must **save and restore** the base-and-bounds pair.
  - In **process structure** or **process control block(**PCB)



例题

【2011统考真题】在虚拟内存管理中，地址变换机构将逻辑地址变换为虚拟地址，形成该逻辑地址的阶段是（ ）。

A. 编辑

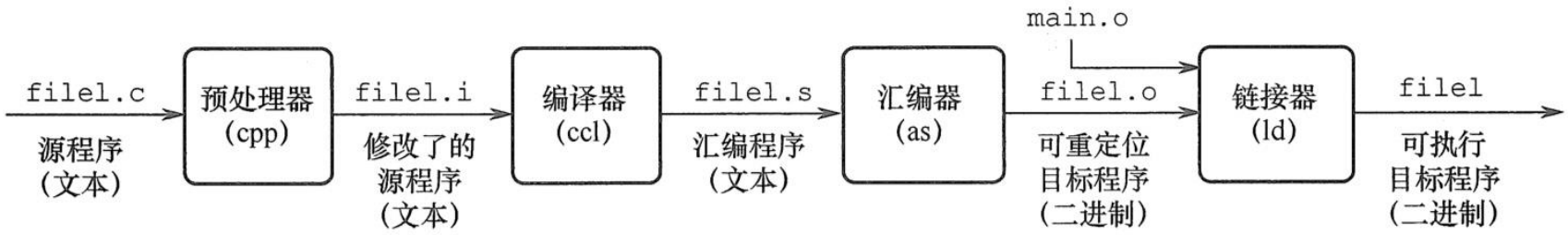
B. 编译

C. 链接

D. 装载

答案：C

解析：编译后的程序需要经过链接才能装载，而链接后形成的目标程序中的地址就是逻辑地址。以C语言为例：C程序经过预处理->编译->汇编->链接产生了可执行文件。链接阶段主要完成重定位，形成整个程序的完整逻辑地址空间。



例题

动态重定位是在作业的（ ）中进行的。

A. 编译过程　B. 装入过程　C. 连接过程　D. 执行过程。

A. 编辑

B. 编译

C. 链接

D. 装载

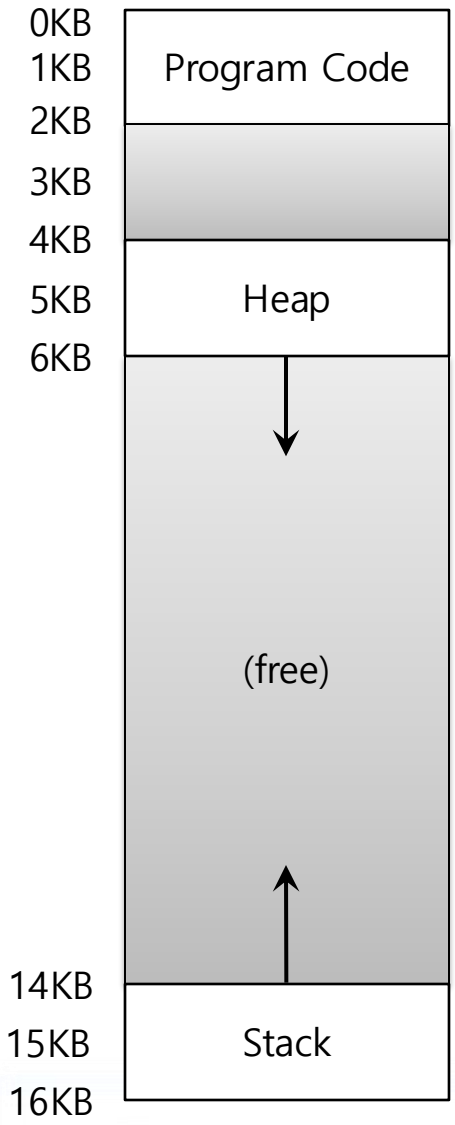
答案：D

在装入过程中，操作系统将作业加载到内存中时，根据实际的内存位置调整地址，使得程序能够在任何内存位置正确执行，而不需要在编译或连接时确定固定的内存地址。这是动态重定位的核心功能。

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）（基础）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Inefficiency of the Base and Bound Approach



- **Big chunk of "free"** space
- "free" space **takes up** physical memory.
- Hard to run when an address space **does not fit** into physical memory

How to support Large Address Space ?

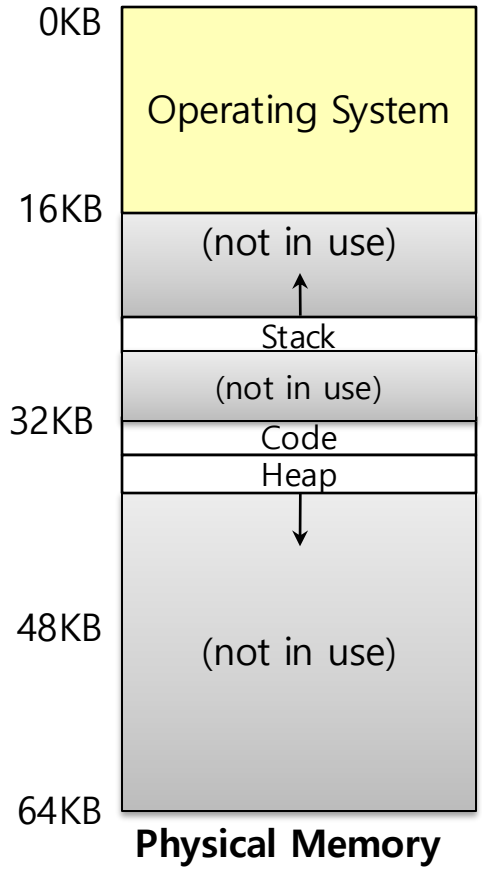
Segmentation

- Segment is just **a contiguous portion** of the address space of a particular length. (段只是特定长度的连续地址空间)
  - Logically-different segment: code, stack, heap

- Each segment can be **placed** in **different part of physical memory**.
  - **Base** and **bounds** exist **per each segment**.

  - “1个进程，1对base & bounds” → “1个进程，3对base & bounds” ?

Placing Segment In Physical Memory

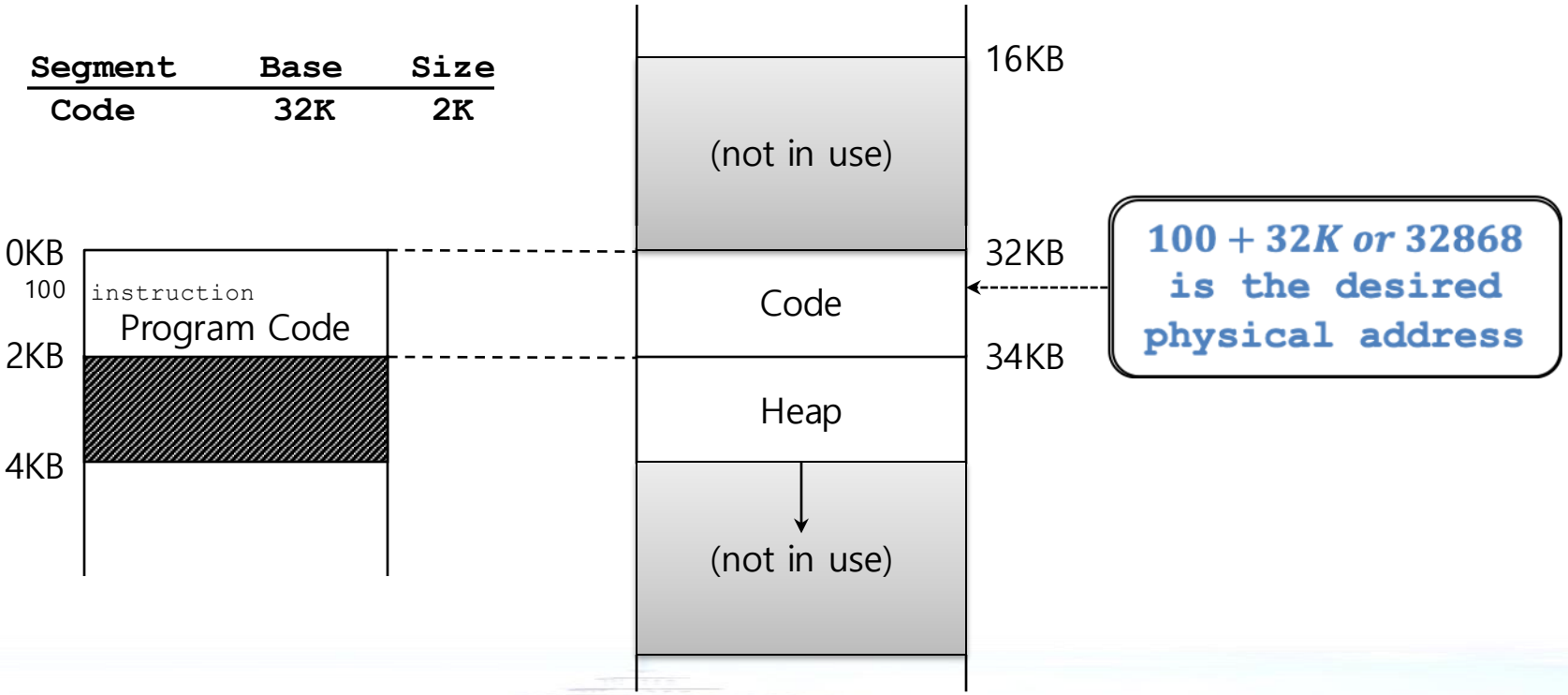


Segment	Base	Size
Code	32K	2K
Heap	34K	2K
Stack	28K	2K

Address Translation on Segmentation

$$physical\ address = offset + base$$

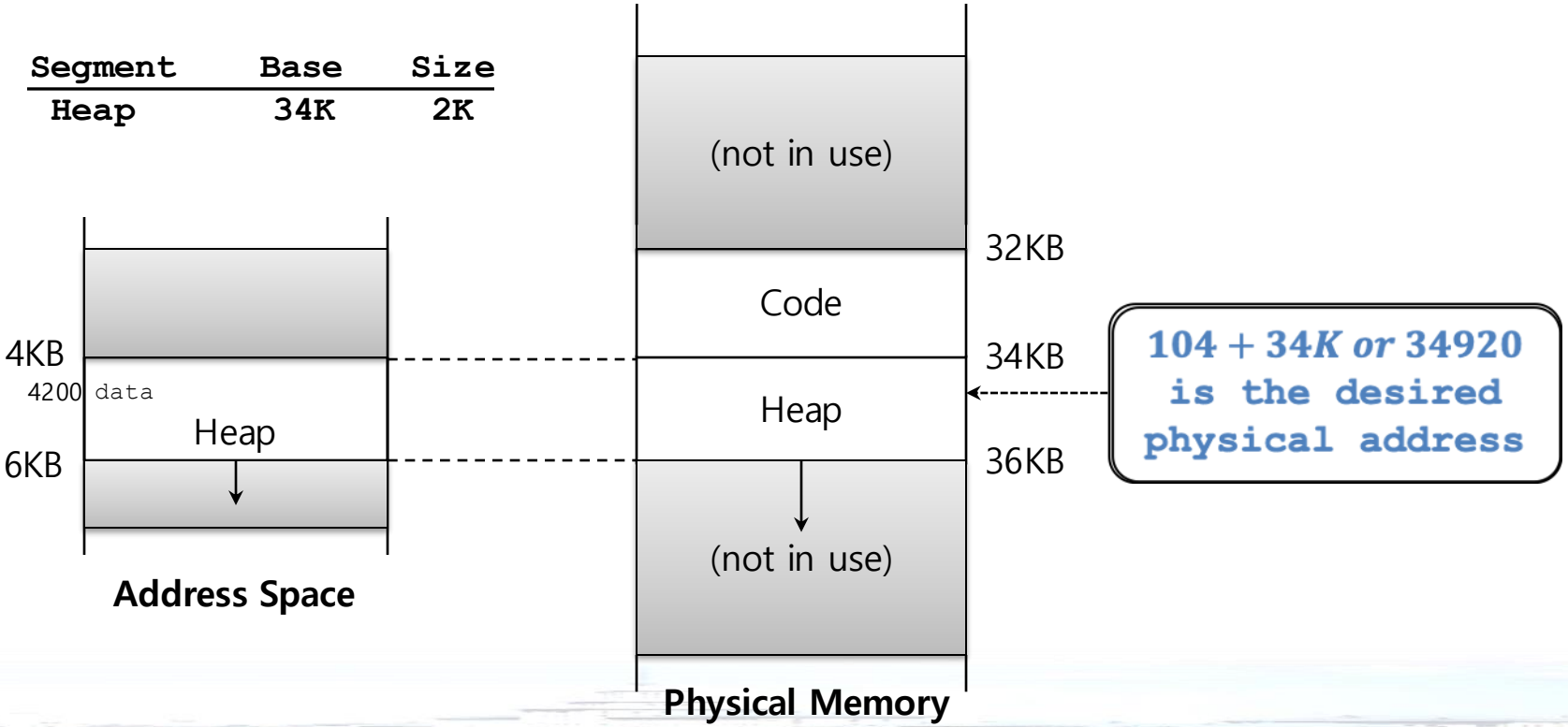
- The `offset` of virtual address 100 is `100`.
  - The code segment **starts at virtual address 0** in address space.



Address Translation on Segmentation(Cont.)

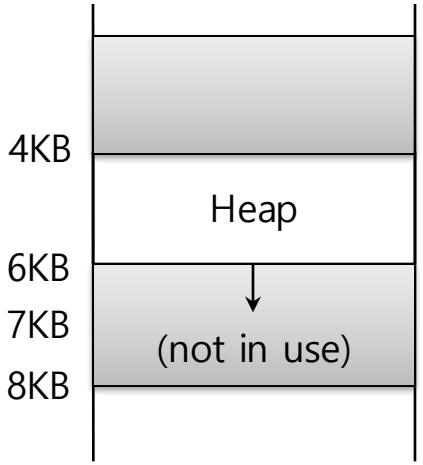
Virtual address + *base* is not the correct physical address

- The `offset` of virtual address 4200 is `104`.
  - The heap segment **starts at virtual address 4096** in address space.



Segmentation Fault or Violation

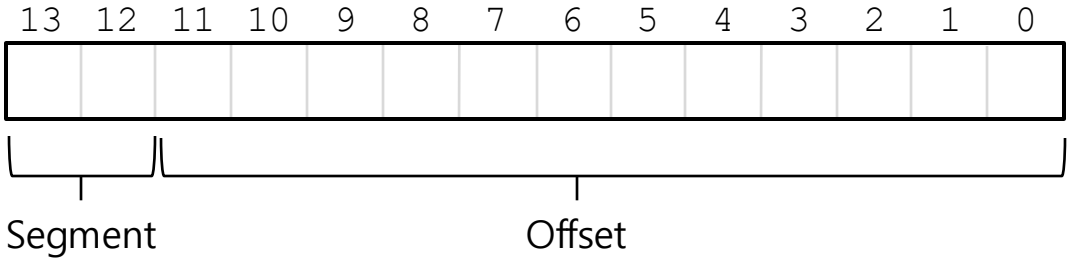
- If an **illegal address** such as 7KB which is beyond the end of heap is referenced, the OS occurs **segmentation fault**. (段错误)
  - The hardware detects that address is **out of bounds**. (越界)



Address Space

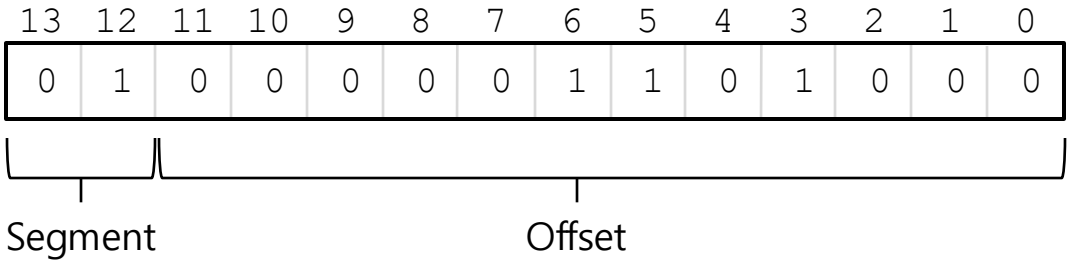
Referring to Segment

- **Explicit approach**
  - Chop up the address space into segments based on the **top few bits** of virtual address. （根据虚拟地址的前几位将地址空间分割成段）



- Example: virtual address 4200 (01000001101000)

Segment	bits
Code	00
Heap	01
Stack	10
-	11



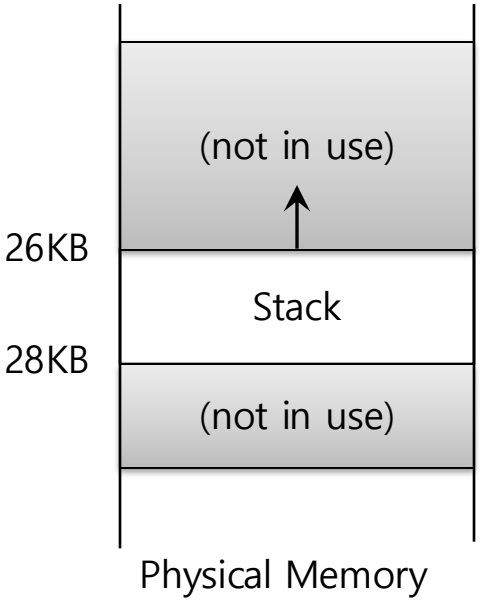
Referring to Segment(Cont.)

```
1   // get top 2 bits of 14-bit VA
2   Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
3   // now get offset
4   Offset = VirtualAddress & OFFSET_MASK
5   if (Offset >= Bounds[Segment])
6        RaiseException(PROTECTION_FAULT)
7   else
8        PhysAddr = Base[Segment] + Offset
9        Register = AccessMemory(PhysAddr)
```

- SEG_MASK = 0x3000(11000000000000)
- SEG_SHIFT = 12
- OFFSET_MASK = 0xFFF (00111111111111)

Referring to Stack Segment

- Stack grows **backward**.
- **Extra hardware support** is needed.
  - The hardware checks which way the segment grows.
  - 1: positive direction, 0: negative direction



Segment Register(with Negative-Growth Support)

Segment	Base	Size	Grows Positive?
Code	32K	2K	1
Heap	34K	2K	1
Stack	28K	2K	0

访问*Virtual address* 15KB，其对应的物理地址是多少?

Support for Sharing

- 分段带来额外的好处，即： 代码共享（**Code sharing**）
- Segment can be **shared between address** space.
  - **Code sharing** is still in use in systems today.
  - by extra hardware support.
- Extra hardware support is needed in form of **Protection bits.**
  - **A few more bits** per segment to indicate **permissions** of **read,** write and **execute**.

Segment Register Values(with Protection)

Segment	Base	Size	Grows Positive?	Protection
Code	32K	2K	1	Read-Execute
Heap	34K	2K	1	Read-Write
Stack	28K	2K	0	Read-Write

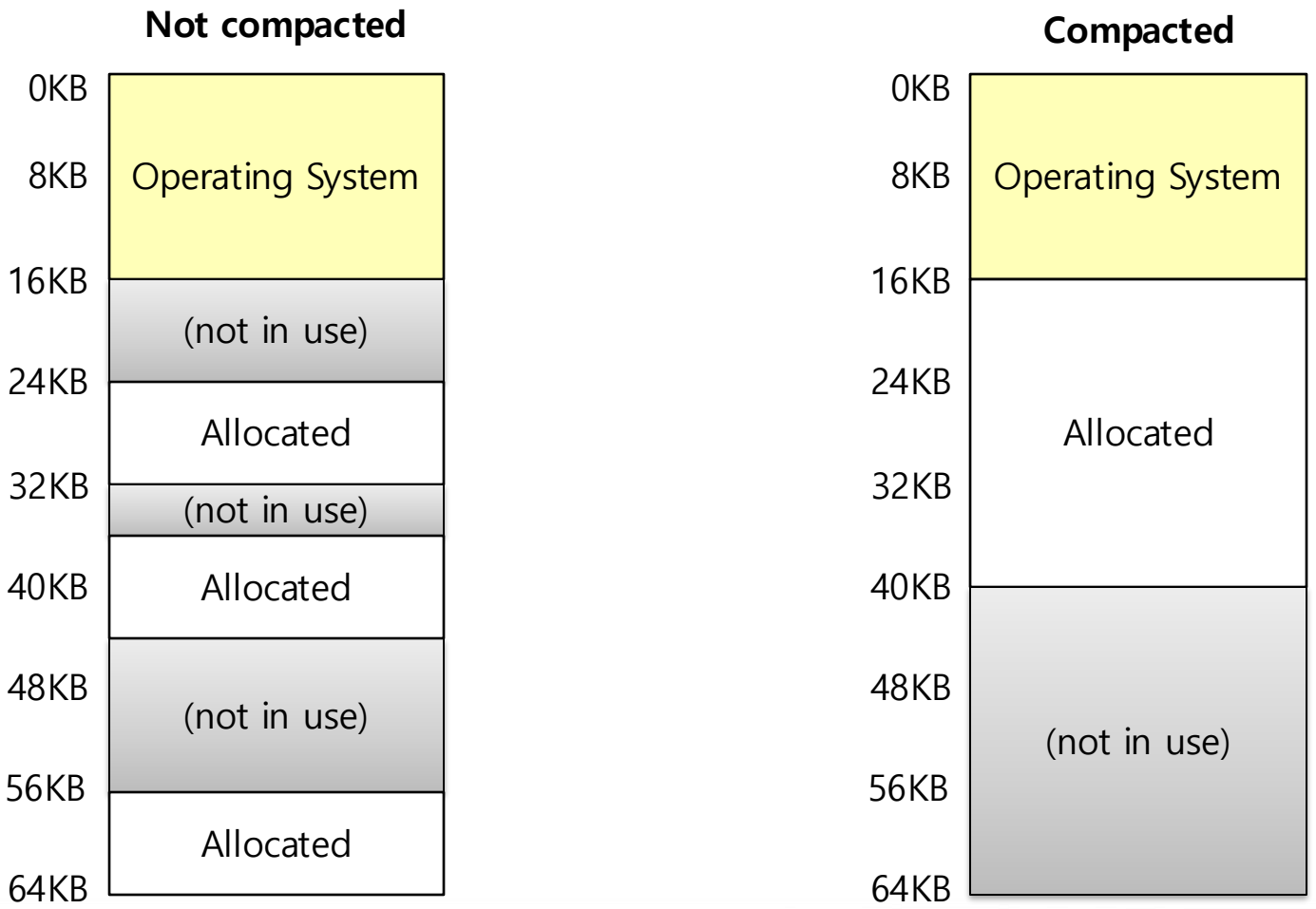
Fine-Grained and Coarse-Grained

- **Coarse-Grained**（粗粒度）means segmentation in a small number.
  - e.g., code, heap, stack.
- **Fine-Grained**（细粒度）segmentation allows **more flexibility** for address space in some early system.
  - To support many segments, Hardware support with a **segment table** is required.
  - 更细粒度的segment，OS可以更好了解哪些段在使用，哪些没有，从而更高效利用内存

OS support:

- **Context Switch**
  - Segment registers 段寄存器需要保存和恢复
  - OS确保这些段寄存器在进程运行前被正确赋值
- **Free Space Management**
  - **External Fragmentation**: little holes of **free space** in physical memory that make difficulty to allocate new segments.
    - There is **24KB free**, but **not in one contiguous** segment.
    - The OS **cannot** satisfy the **20KB request**.

Memory Compaction



OS support:

- **Context Switch**
  - Segment registers 段寄存器需要保存和恢复
  - OS确保这些段寄存器在进程运行前被正确赋值
- **Free Space Management**
  - **External Fragmentation**: little holes of **free space** in physical memory that make difficulty to allocate new segments.
    - There is **24KB free**, but **not in one contiguous** segment.
    - The OS **cannot** satisfy the **20KB request**.
  - **Compaction**: **rearranging** the exiting segments in physical memory. While Compaction is **costly**.
    - **Stop** running process.
    - **Copy** data to somewhere.
    - **Change** segment register value.
  - Free-list Management
    - Worst-fit， best-fit，first-fit，buddy algorithm，.....
    - 如果解决一个问题有很多种方法，就没有一种是特别好的

例题

采用段式存储管理的系统中，若地址用24位表示，其中8位表示段号，则允许每段的最大长度是（ ）。

A. 2^{24}B

B. 2^{16}B

C. 2^{8}B

D. 2^{32}B

答案：B

解析：采用段式管理的系统中，其逻辑地址分为段号和页内偏移量。本题的地址一共24位，使用了8位表示段号，那么把剩下的16位全部用来表示段内偏移量就能使每段长度最大从而每段允许的最大长度为2^16。

例题

在分段存储管理系统中，用共享段表描述所有被共享的段。若进程P1和P2共享段S，下列叙述中，错误的是（ ）

A. 在物理内存中仅保存一份段S的内容

B. 段S在P1和P2中应该具有相同的段号

C. P1和P2共享段S在共享段表中的段表项

D. P1和P2都不再使用段S时才回收段S所占的内存空间

答案：C

解析：

A. 段的共享是通过两个作业的段表中相应表项指向被共享的段的同一个物理副本来实现的，因此在内存中仅保存一份段S的内容，即A选项正确；

B. 段表项存放的是段的物理地址（包括段始址和段长度），对于共享段S来说物理地址唯一，即P1和P2共享段S在共享段表中的段表项，即B选项正确；

C. 段S对于进程P1、P2来说，使用位置不同，所以在不同进程中的逻辑段号可能不同，即C选项错误；

D. 为了保证进程可以顺利使用段S，段S必须确保在没有任何进程使用它后才能被删除，可在段表项中设置一个共享进程计数count，即D选项正确。

Module 4: Virtualizing Memory 虚拟内存

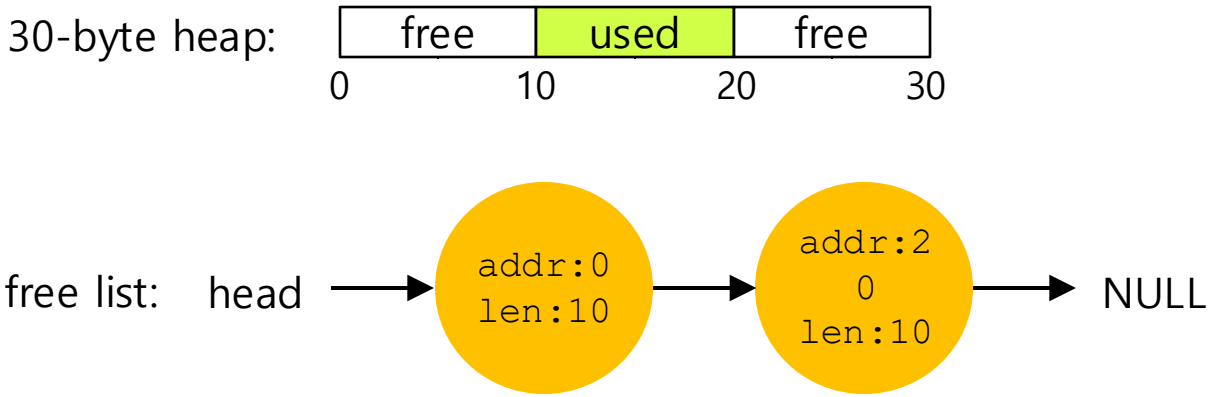
1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
   a) 基础概念（基础）
   b) Free List（基础）
   c) 管理策略（基础）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Assumptions

- 若需要管理的空闲空间被划分成固定大小的单元，则管理容易
- 若空闲空间由大小不同的单元构成，则管理变得困难！
  - 满足变长的分配请求，该如何组织空闲空间?
  - 如何使碎片最小化?
  - 不同方法的开销（包括时间开销和空间开销）如何?
- 分配接口假设以malloc()和free()为例
- **External Fragmentation** VS Internal Fragmentation
- 内存一旦被分配，无法被重定位，但是：
  - Segmented OS kernel
  - 强类型程序设计语言

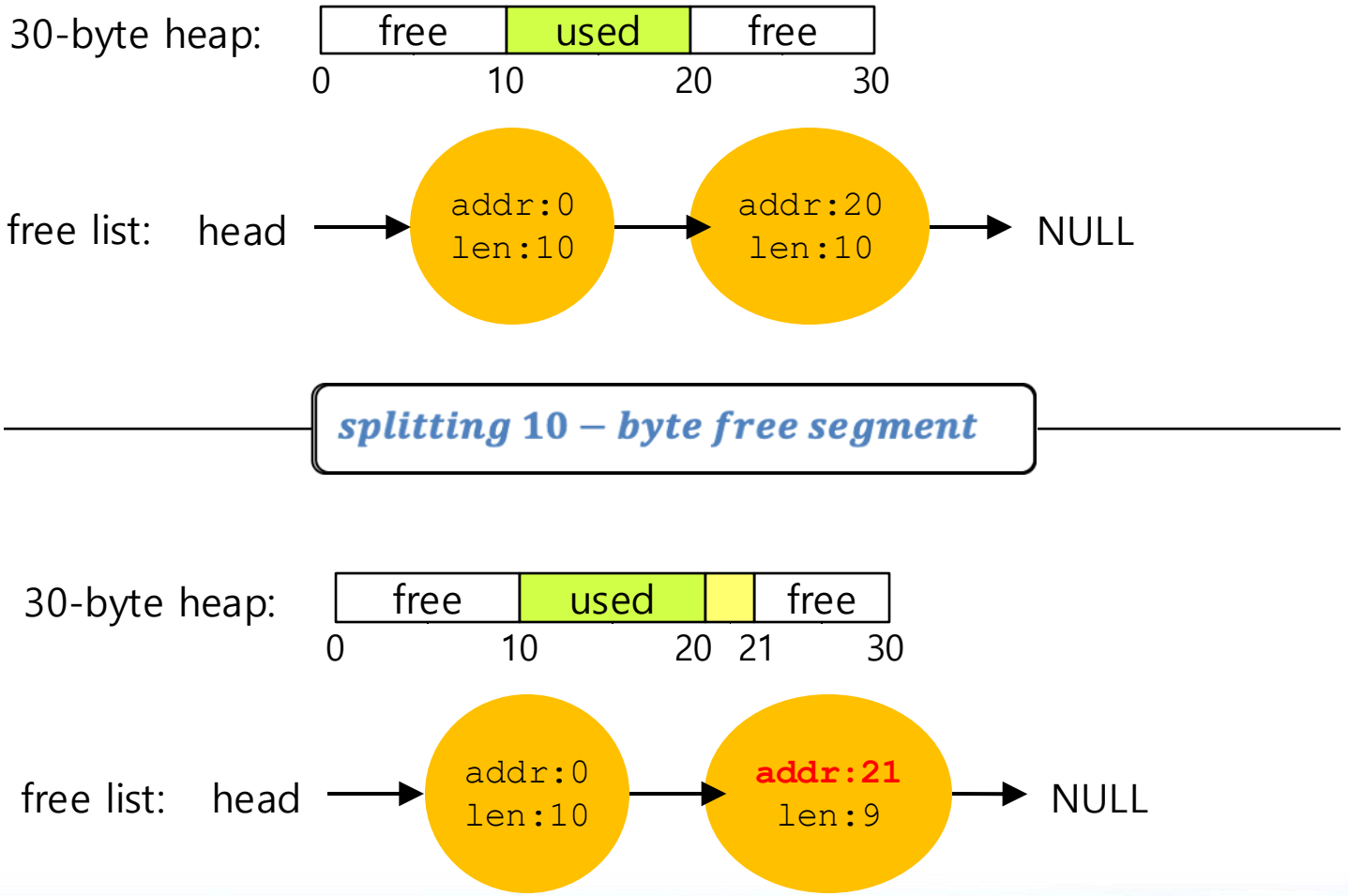
Splitting

- Finding a free chunk of memory that can satisfy the request and splitting it into two.
  - When request for memory allocation is **smaller** than the size of free chunks.



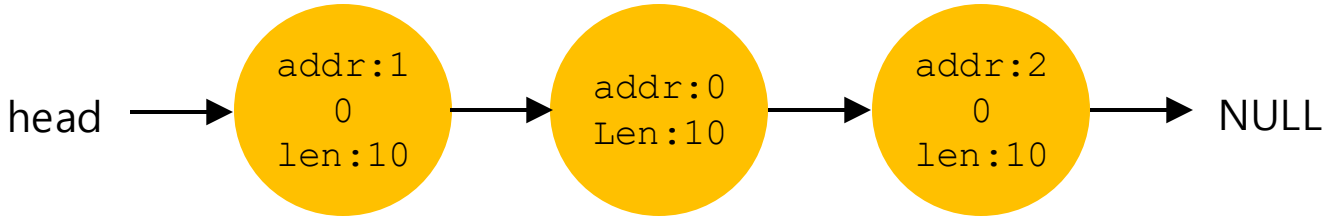
Splitting(Cont.)

- Two 10-bytes free segment with **1-byte request**



Coalescing

- If a user requests memory that is **bigger than free chunk size**, the list will **not find** such a free chunk.
- Coalescing: **Merge** returning a free chunk with existing chunks into a large single free chunk if **addresses** of them are **nearby**. （合并：如果它们的地址在附近，则将返回的空闲块与现有块合并为一个大的单个空闲块）



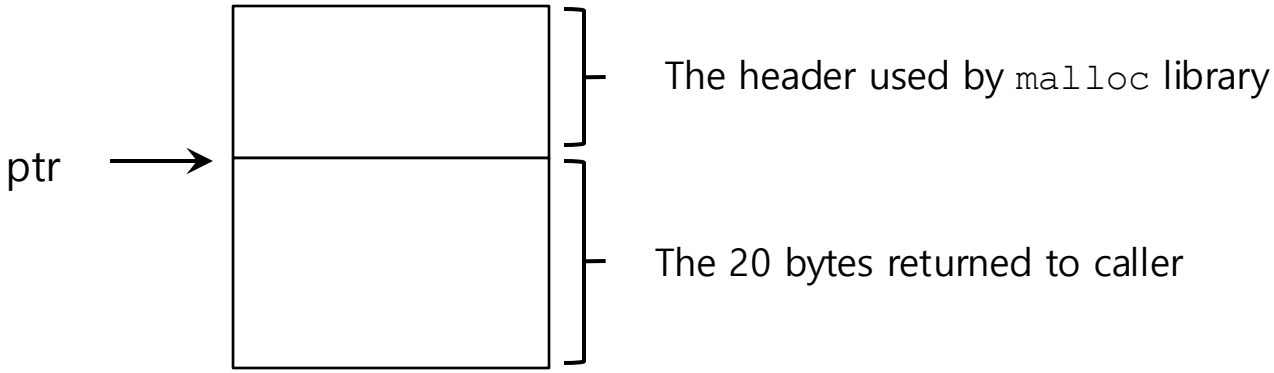
Coalescing free chunks



Tracking The Size of Allocated Regions

- The interface to `free(void *ptr)` does **not take** a **size parameter**.
  - How does the library **know the size** of memory region that will be back **into free list**?
- Most allocators store **extra information** in a **header** block.

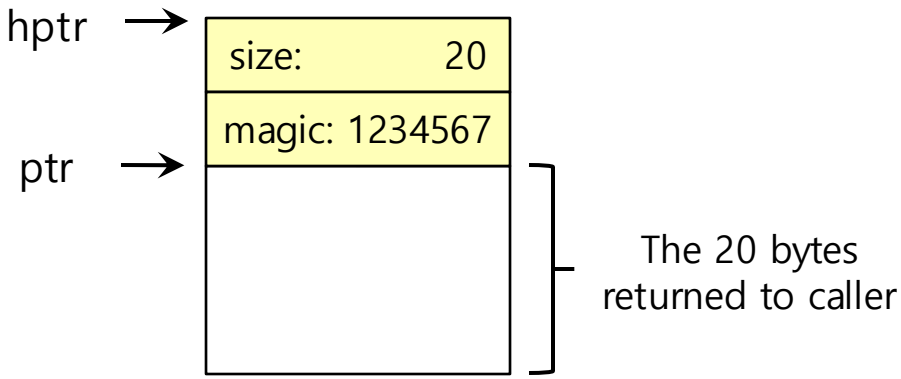
```
ptr = malloc(20);
```



An Allocated Region Plus Header

The Header of Allocated Memory Chunk

- The header minimally **contains the size** of the allocated memory region.
- The header may also contain
  - Additional pointers to speed up deallocation
  - A magic number for integrity checking



Specific Contents Of The Header

```
typedef struct __header_t {
    int size;
    int magic;
} header_t;
```

A Simple Header

The Header of Allocated Memory Chunk(Cont.)

- The **size** for free region is the **size of the header plus the size of the space** allocated to the user.
  - If a user **request `N` bytes**, the library searches for a free chunk of **size `N` plus the size of the header**

- Simple pointer arithmetic to find the header pointer.

```
void free(void *ptr) {
        header_t *hptr = (void *)ptr - sizeof(header_t);
}
```

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
   a) 基础概念（基础）
   b) Free List （基础）
   c) 管理策略（基础）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Embedding A Free List

- The memory-allocation library **initializes** the heap and **puts** the first element of **the free list** in the **free space**. (内存分配库初始化堆并将空闲列表的第一个元素放入空闲空间)
  - The library **can't use** `malloc()` to build a list **within itself**.

Embedding A Free List(Cont.)

- Description of a node of the list

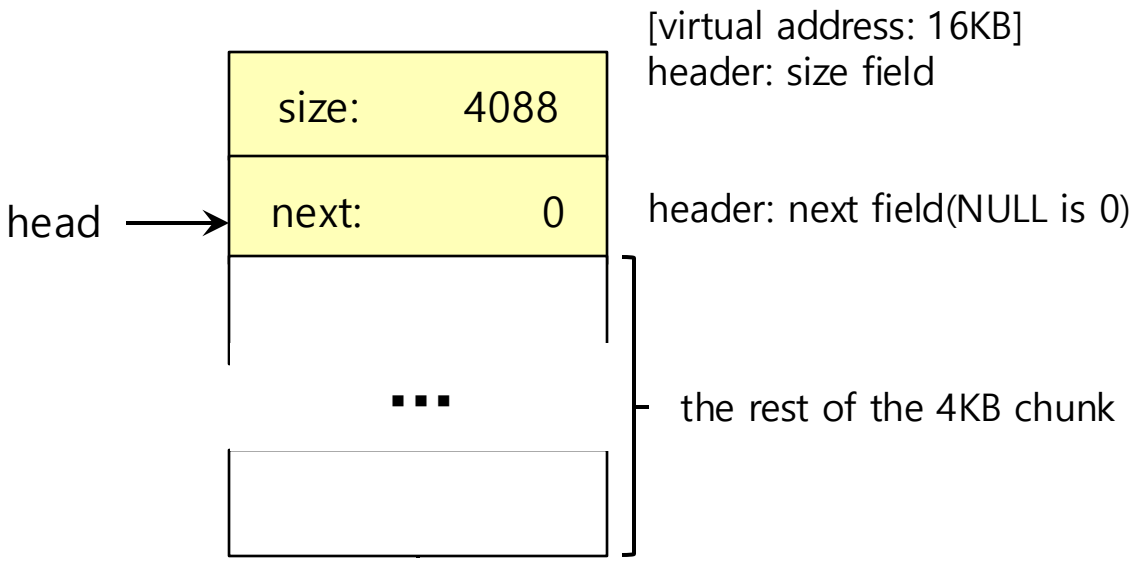
```
typedef struct __node_t {
        int size;
        struct __node_t *next;
} nodet_t;
```

- Building heap and putting a free list
  - Assume that the heap is built vi `mmap()` system call.

```
// mmap() returns a pointer to a chunk of free space
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                                MAP_ANON|MAP_PRIVATE, -1, 0);
head->size = 4096 - sizeof(node_t);
head->next = NULL;
```

A Heap With One Free Chunk

```
// mmap() returns a pointer to a chunk of free space
node_t *head = mmap(NULL, 4096, PROT_READ|PROT_WRITE,
                                MAP_ANON|MAP_PRIVATE, -1, 0);
head->size = 4096 - sizeof(node_t);
head->next = NULL;
```



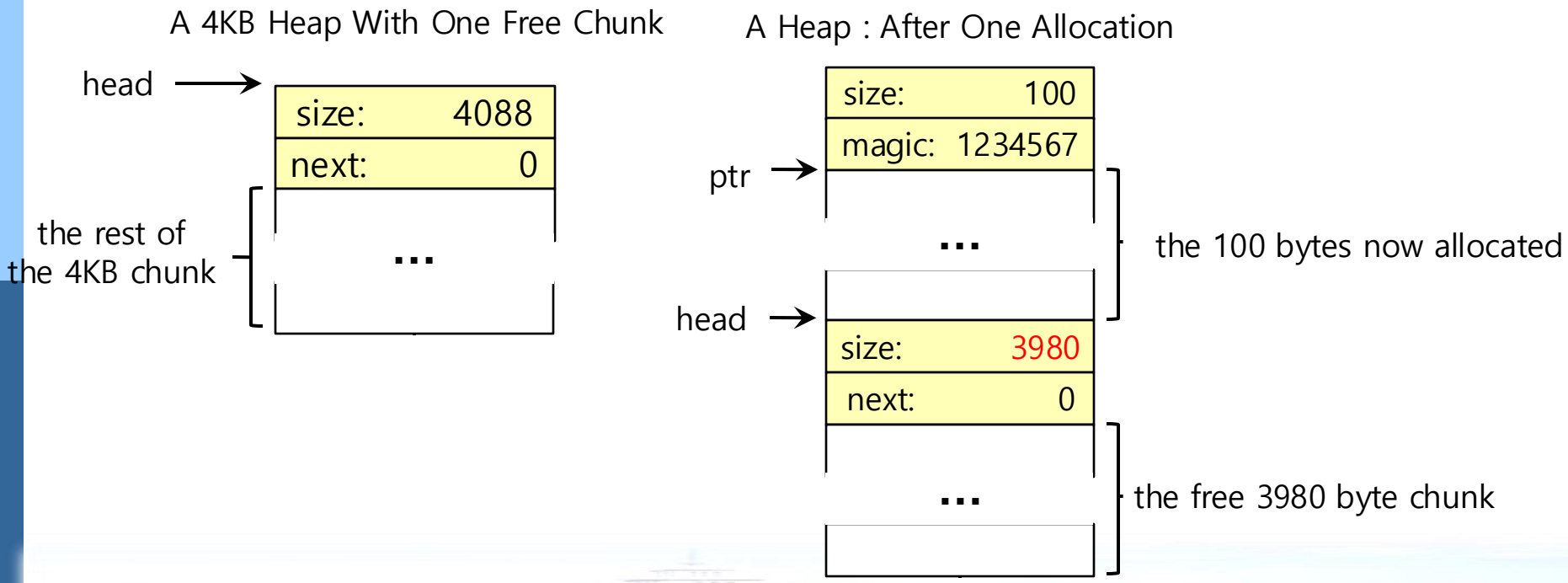
Embedding A Free List: Allocation

- If a chunk of memory is requested, the library **will first find** a chunk that is **large enough** to accommodate the request. (如果请求了一块内存，库将首先找到一个足够大以容纳请求的内存块)

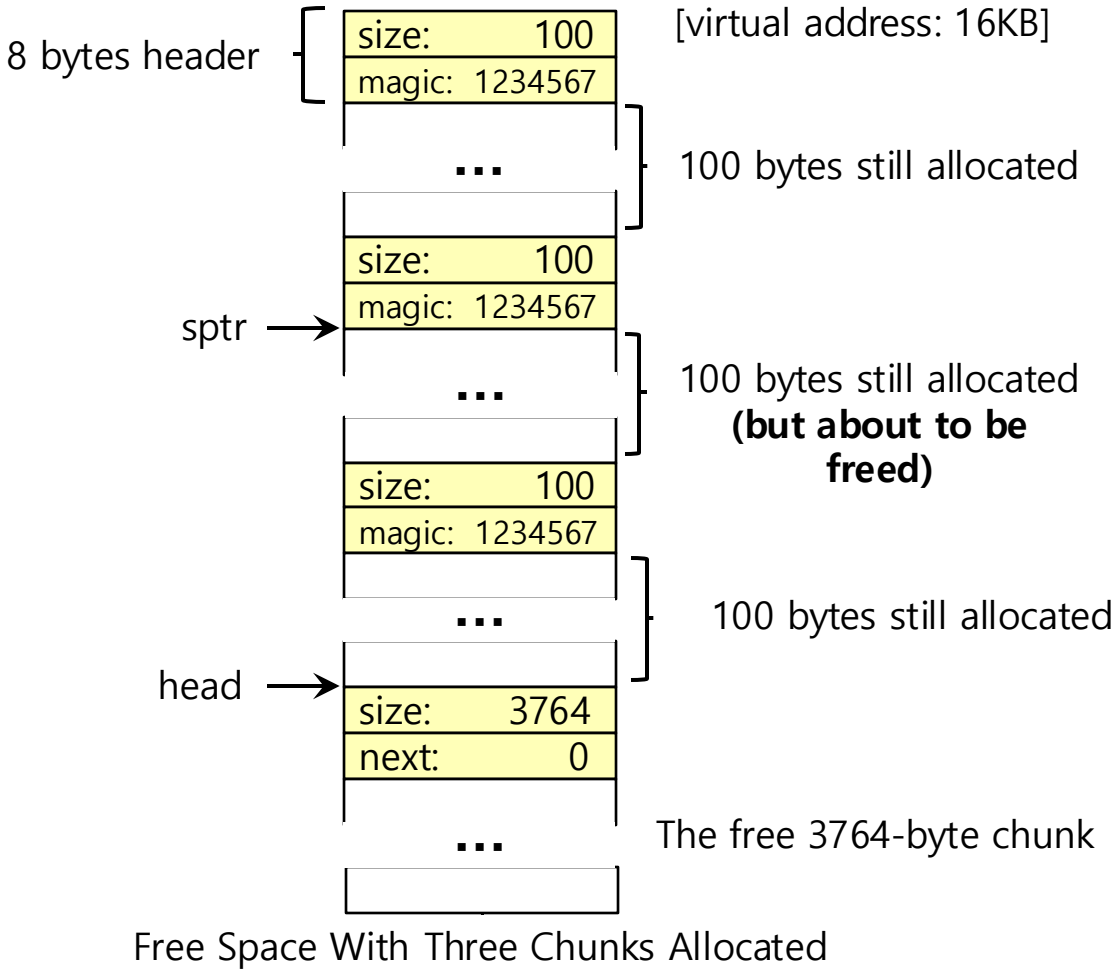
- The library will
  - **Split** the large free chunk into two.
    - **One** for the **request** and the **remaining** free chunk
  - **Shrink** the size of free chunk in the list.

Embedding A Free List: Allocation(Cont.)

- Example: a request for 100 bytes by `ptr = malloc(100)`
  - Allocating 108 bytes out of the existing one free chunk.
  - shrinking the one free chunk to 3980(4088 minus 108).



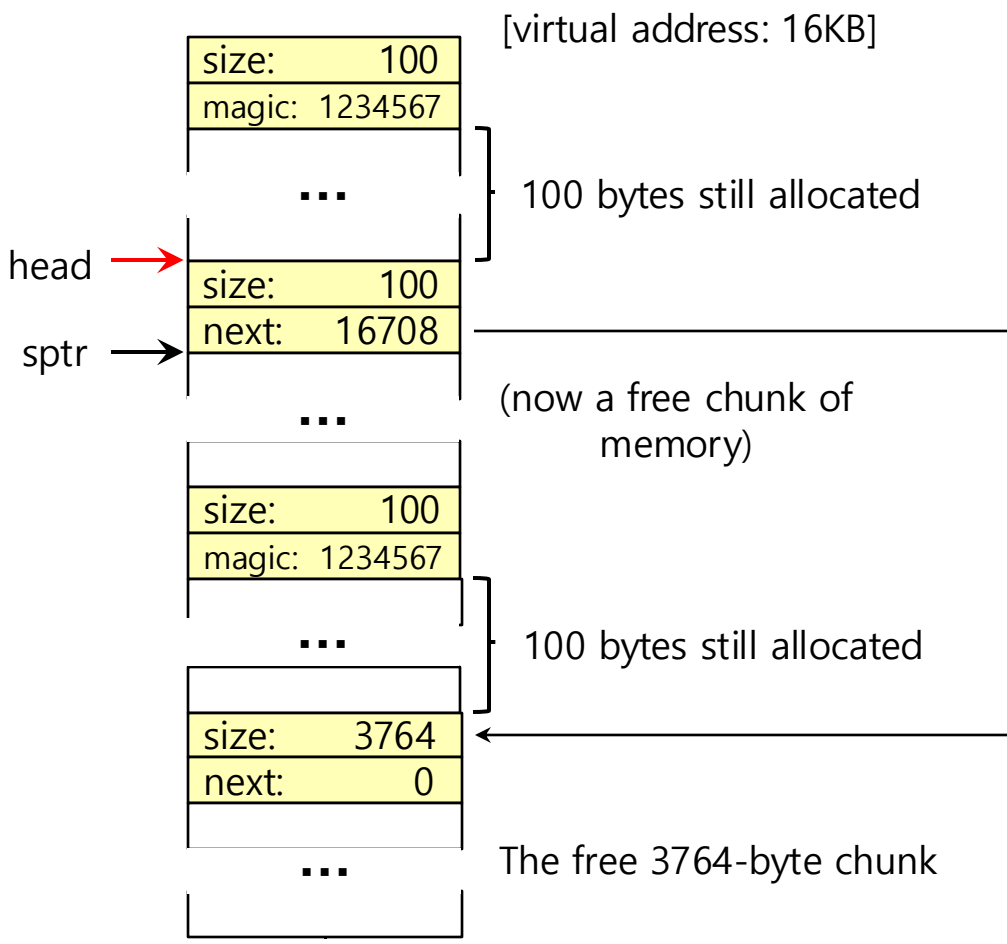
Free Space With Chunks Allocated



Free Space With Three Chunks Allocated

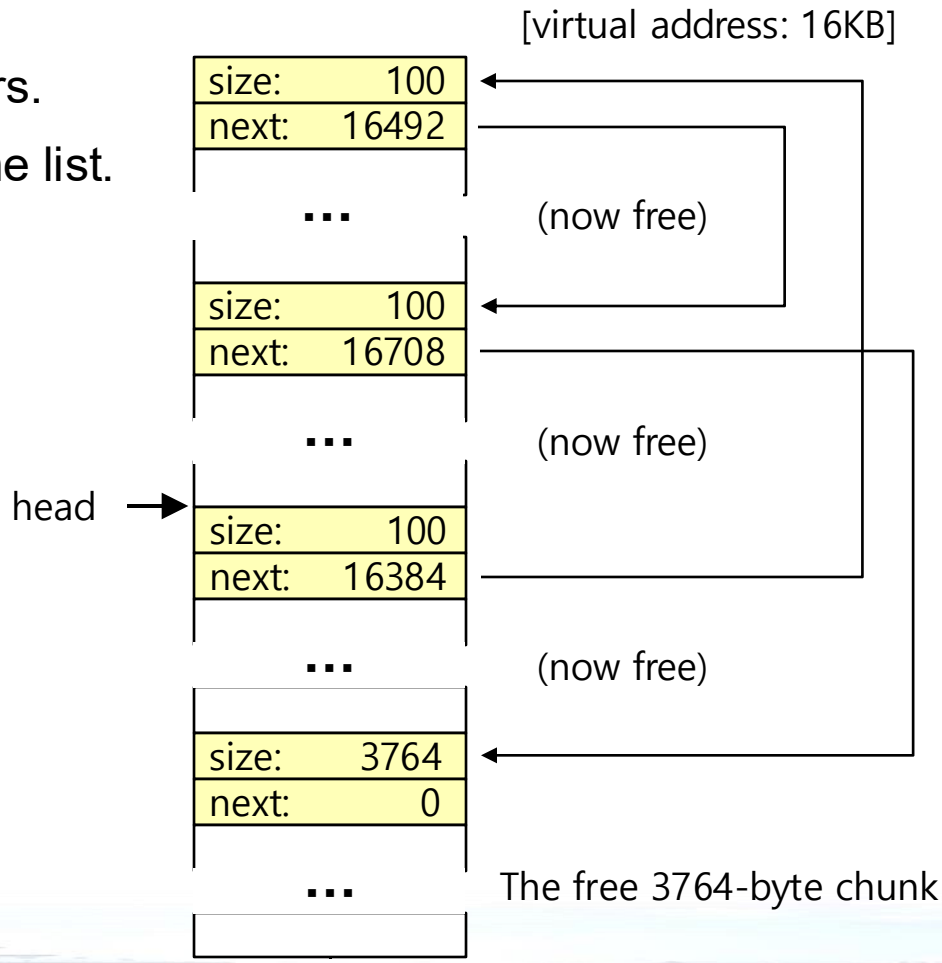
Free Space With `free()`

- Example: `free(sptr)`
  - The 100 bytes chunks is **back into** the free list.
  - The free list will **start** with **a small chunk**.
    - The list header will point the small chunk



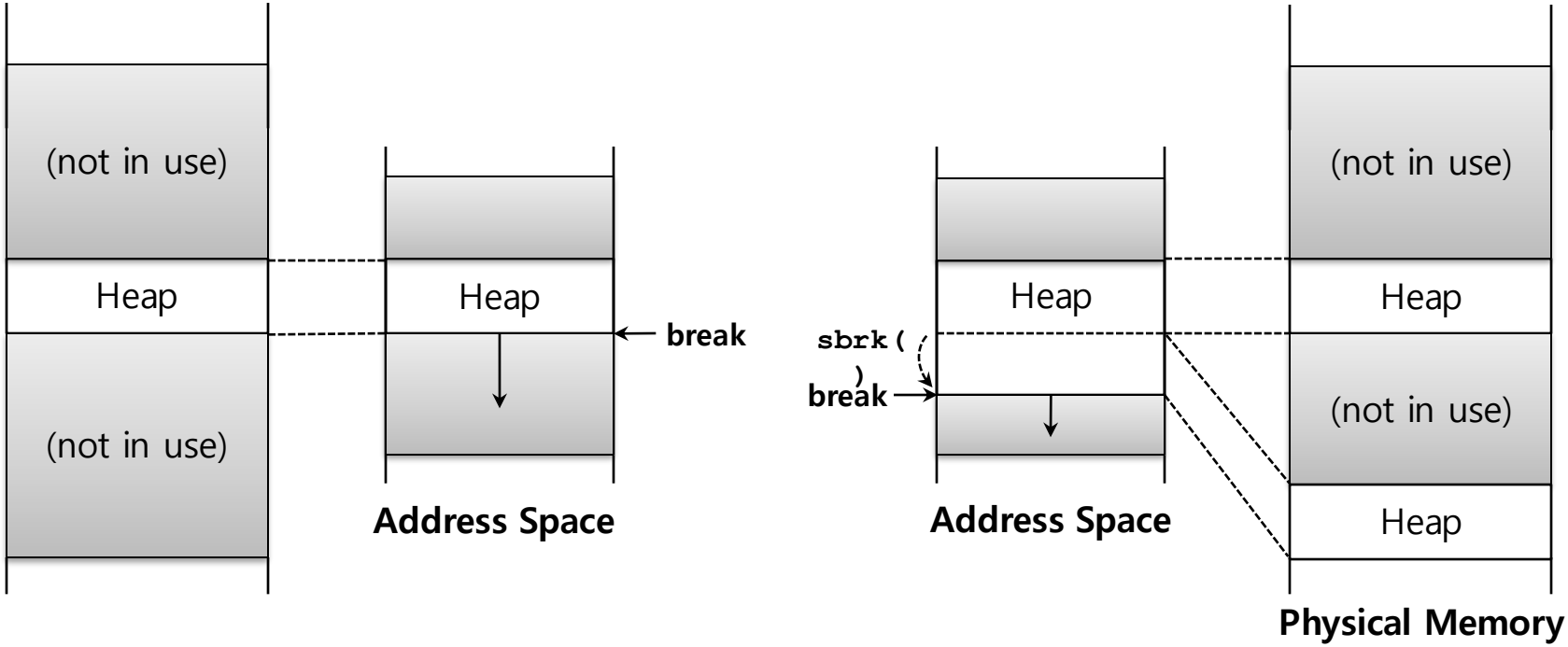
Free Space With Freed Chunks

- Let's assume that the last two in-use chunks are freed.
- **External Fragmentation** occurs.
  - **Coalescing** is needed in the list.



Growing The Heap

- Most allocators **start** with **a small-sized heap** and then **request more** memory from the OS when they run out.
  - e.g., `sbrk()`, `brk()` in most UNIX systems.



Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
   a) 基础概念（基础）
   b) Free List （基础）
   c) 管理策略（基础）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Managing Free Space: Basic Strategies

- Best Fit:
  - Finding free chunks that are **big or bigger than the request**
  - Returning the **one of smallest** in the chunks **in the group** of candidates

- Worst Fit:
  - Finding the **largest free chunks** and allocation the amount of the request
  - **Keeping the remaining chunk** on the free list.

Managing Free Space: Basic Strategies(Cont.)

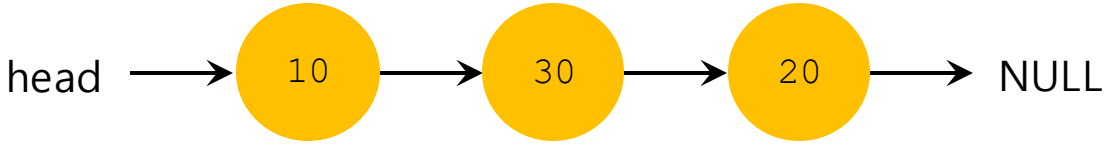
- First Fit:
  - Finding the **first chunk** that is **big enough** for the request
  - Returning the requested amount and remaining the rest of the chunk.

- Next Fit:
  - Instead of always beginning the first-fit search at the beginning of the list
  - Keep an **extra pointer** to the location within the list where one was **looking last**. 即搜索的起始点是上次返回的节点位置

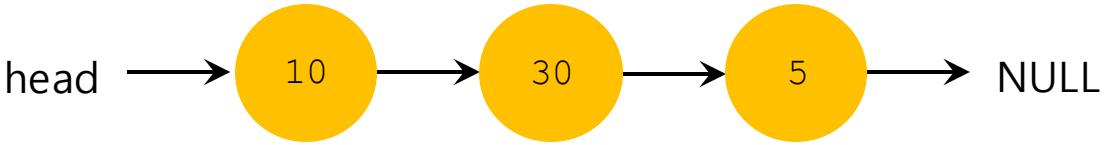
  - The idea is to **spread the searches for free space throughout the list more uniformly**, thus avoiding splintering of the beginning of the list (在整个列表中更均匀地分布对可用空间的搜索)

Examples of Basic Strategies

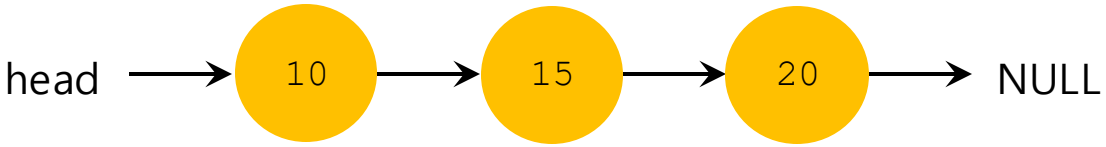
- Allocation Request Size 15



- Result of Best-fit



- Result of Worst-fit



例题

某基于动态分区存储管理的计算机，其主存容量为55MB (初始为空闲)，采用最佳适配(Best Fit)算法，分配和释放的顺序为：分配15MB，分配30MB，释放15MB，分配8MB，分配6MB ，此时主存中最大空闲分区的大小是（）。

A. 7MB

B. 9MB

C. 10MB

D. 15MB

答案：B

解析：

主存容量为55MB。

1. 第一步分配15MB以后还有55MB - 15MB = 40MB;
2. 第二步分配30MB以后还有40MB - 30MB = 10MB;
3. 第三步释放15MB以后有两个空闲区：15MB和10MB;
4. 第四步分配8MB，则空闲区为15MB和2MB;
5. 第五步分配6MB，则空闲区为9MB和2MB。

Other Approaches: Segregated List

- Segregated List:
  - Keeping free chunks in different size in a separate list for the size of popular request. 拿出一部分内存专门满足最频繁内存分配请求，碎片就不是事了
  - New Complication:
    - **How much** memory should dedicate to **the pool of memory** that serves **specialized requests** of a given size?
  - **Slab allocator** handles this issue.

Other Approaches: Segregated List(Cont.)

- Slab Allocator
  - Allocate a number of object caches.
    - The objects are likely to e requested frequently.
    - e.g., locks, file-system inodes, etc.
  - **Request some memory** from a more general memory allocator when **a given cache is running low** on free space.（size一般是page size和object size的公倍数）
  - 数据结构初始化和销毁的代价很大，通过将空闲对象保持在初始化状态，slab allocator避免了频繁的初始化和销毁，显著降低开销
  - Benefits:
    - Reclaiming unused memory is trivial.
    - Allocating and freeing memory are fast, constant-time operations.
    - Severe external fragmentation (unused buffers on the freelist) is unlikely.
    - Internal fragmentation (per-buffer wasted space) is minimal.

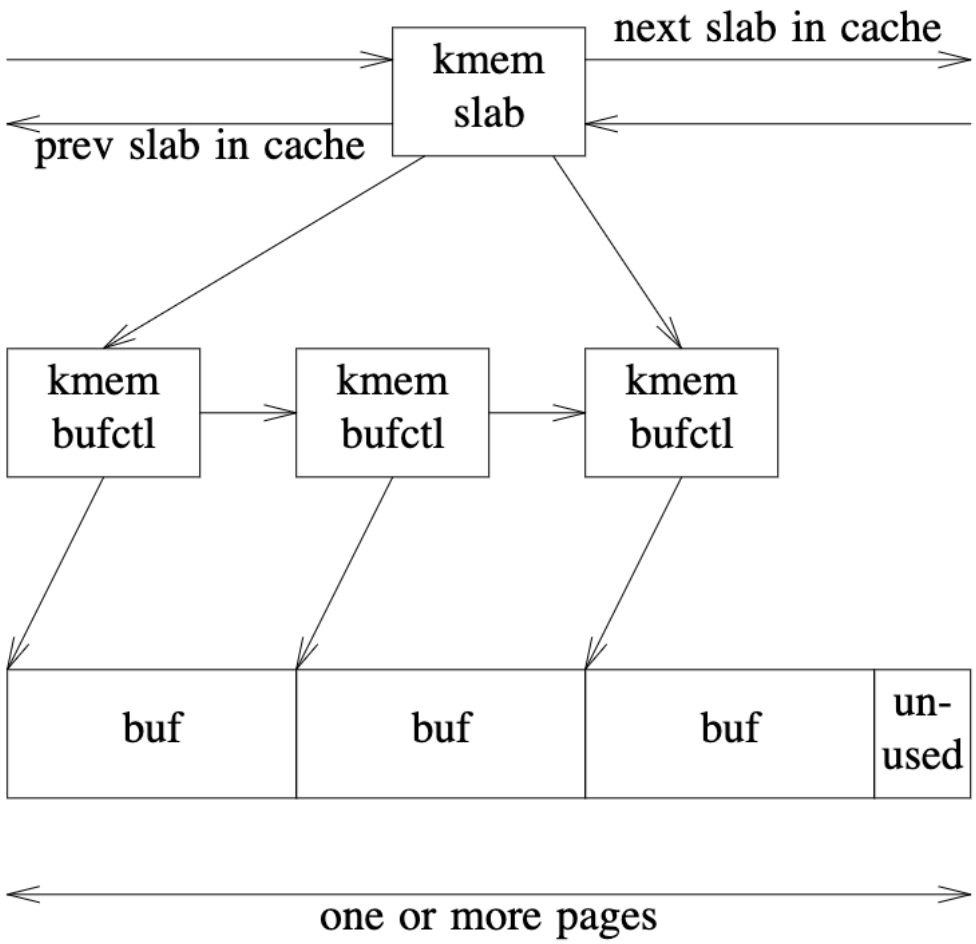
Other Approaches: Segregated List(Cont.)

- Slab
  - The slab is the primary unit of currency in the slab allocator.
  - A separate memory pool for frequently created & deleted objects of same size.
    - e.g., task_struct, inode in Linux, etc
  - Consists of one or more pages of virtually contiguous memory carved up into equal-size chunks (由一页或多页几乎连续的内存组成，分割成相等大小的块)
  - With a reference count indicating how many of those chunks have been allocated

Other Approaches: Segregated List(Cont.)

- `kmem_slab`: Each slab is managed, which maintains the slab's linkage in the cache, its reference count, and its list of free buffers. (每个slab都被管理，它维护slab在缓存中的链接、它的引用计数和它的空闲缓冲区列表)

- `kmem_bufctl`: Each buffer in the slab is managed, which holds the freelist linkage, buffer address, and a back pointer to the controlling slab. (管理slab中的每个缓冲区，它保存freelist 链接、缓冲区地址和指向控制slab的反向指针)



(bufctl-to-slab back-pointers not shown)

The Slab Allocator original paper

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

The Slab Allocator: An Object-Caching Kernel Memory Allocator

Jeff Bonwick
Sun Microsystems

Abstract

This paper presents a comprehensive design overview of the SunOS 5.4 kernel memory allocator. This allocator is based on a set of object-caching primitives that reduce the cost of allocating complex objects by retaining their state between uses. These same primitives prove equally effective for managing stateless memory (e.g. data pages and temporary buffers) because they are space-efficient and fast. The allocator's object caches respond dynamically to global memory pressure, and employ an object-coloring scheme that improves the system's overall cache utilization and bus balance. The allocator also has several statistical and debugging features that can detect a wide range of problems throughout the system.

generally superior in both space *and* time. Finally, Section 6 describes the allocator's debugging features, which can detect a wide variety of problems throughout the system.

2. Object Caching

Object caching is a technique for dealing with objects that are frequently allocated and freed. The idea is to preserve the invariant portion of an object's initial state — its *constructed* state — between uses, so it does not have to be destroyed and recreated every time the object is used. For example, an object containing a mutex only needs to have `mutex_init()` applied once — the first time the object is allocated. The object can then be freed and reallocated many times without incurring

Jeff Bonwick：工程师的价值符合幂律分布？



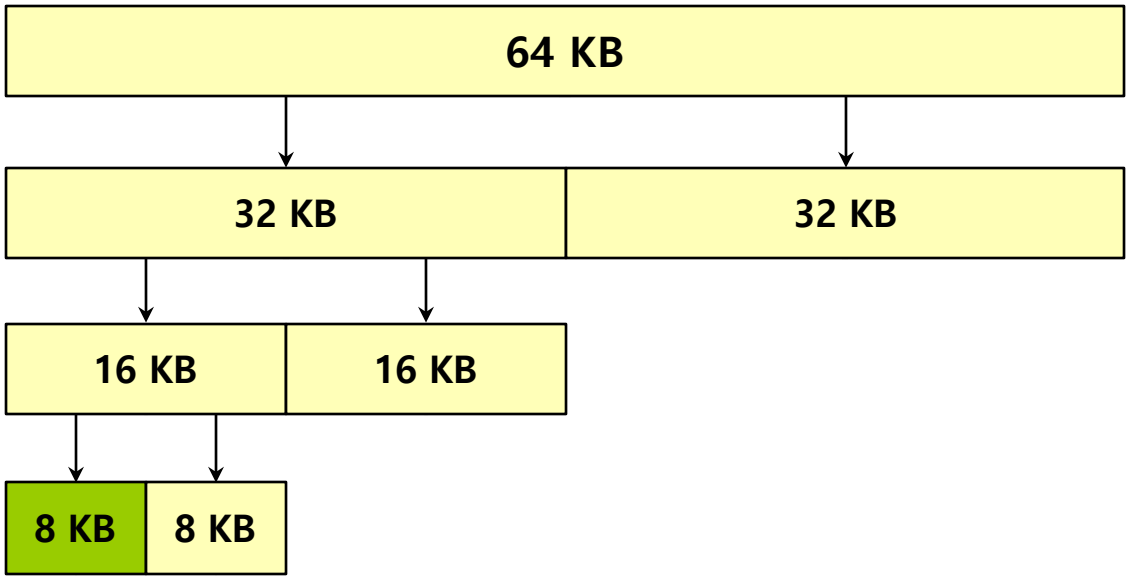
- invented and led development of the ZFS file system
- inventor of slab allocation
- In 2010 Bonwick co-founded a small company called DSSD, which was acquired by EMC Corporation in 2014

Someone who is **exceptional** in their role is not just a little better than someone who is pretty good. They are **100 times better**
——Mark Zuckerberg, Facebook)

- **One or two people can start a company that changes the face of the world forever**
- **think Google, Apple, or Facebook**

Other Approaches: Buddy Allocation

- Binary Buddy Allocation
  - For coalescing tasks 适用于任务合并
  - The allocator **divides free space** by two **until a block** that is big enough to accommodate the request is **found**. (分配器将可用空间除以2，直到找到足够大以容纳请求的块)

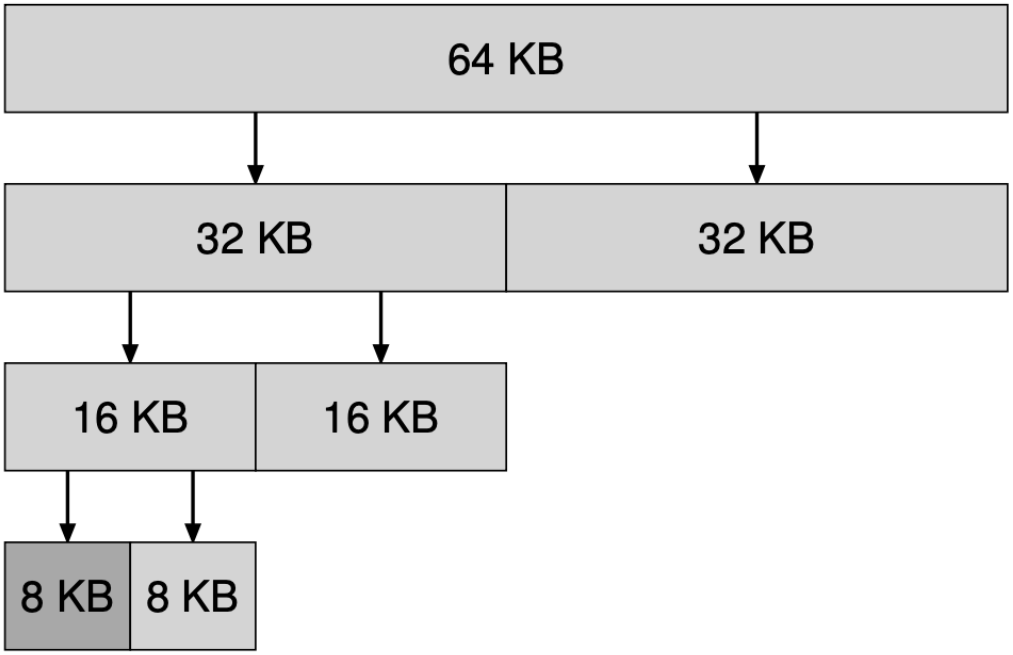


64KB free space for 7KB request

Buddy（伙伴）算法

- 在伙伴系统中，空闲存储器被认为是大小为2^N的一个大空间。当请求内存时，对可用空间的搜索会将可用空间递归地一分为二，直到找到一个足够大的块来容纳请求（再一分为二会导致空间太小）。此时，请求的块被返回给用户。

- 下图是一个在64KB可用空间中搜索7KB块时被分割的示例



- **分配时**：最左边的8KB块被分配（由较暗的灰色阴影所指示的）并返回给用户；
  - 注意，这个方案可能会受到内部碎片的影响，因为只允许分配2^k个大小块。
- **回收时**：
  1. 当将8KB块返回到空闲列表时，分配器检查“伙伴”8KB块是否空闲；如果是，它将这两个块合并为一个16KB的块。
  2. 分配器检查“伙伴”16KB块是否空闲；如果是，它将这两个块合并为一个32KB的块。
  3. 。。。。

这个递归合并过程在树上继续，要么恢复整个可用空间，要么发现伙伴正在使用就停止。

glibc

- glibc是GNU C Library 项目为 GNU 系统和 GNU/Linux 系统以及许多其他使用 Linux 作为内核的系统提供的核心库。



- 这些 API 包括open、read、write、malloc、printf、getaddrinfo、dlopen、pthread_create、crypt、login、exit等。

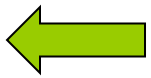
GNU C 库每 6 个月发布一次。

2023-07-31：glibc 2.38 发布。

2023-02-01：glibc 2.37 发布。

2022-08-01：glibc 2.36 发布。

2022-02-03：glibc 2.35 发布。



Ubuntu 22.04使用的是glibc 2.35

2021-08-01：glibc 2.34 发布。

2021-02-01：glibc 2.33 发布。

glibc中的malloc

- malloc是内存分配函数。glibc中的malloc使用的是ptmalloc2。
- ptmalloc2(支持Posix threads)是从dlmalloc(开发者Doug Lea) fork出来的。fork之后，增加了线程支持，并于2006年发布。正式发布后，ptmalloc2被集成到glibc源代码中。

```
void *malloc(size_t size);
void free(void *ptr);
```

- **系统调用**:malloc 在内部调用brk或mmap系统调用。
- **线程支持**:在 Linux 的早期，dlmalloc 被用作默认的内存分配器。但后来由于ptmalloc2的线程支持，它成为linux默认的内存分配器。线程支持有助于提高内存分配器性能，从而提高应用程序性能。
  - 在 dlmalloc 中，当两个线程同时调用 malloc 时，**只有一个线程可以进入临界区**，因为 freelist 数据结构在所有可用线程之间共享。因此，在多线程应用程序中内存分配需要时间，从而导致性能下降。
  - 在 ptmalloc2 中，当两个线程同时调用 malloc 时，会立即分配内存，因为每个线程维护一个单独的堆，因此维护这些堆的 freelist 数据结构也是单独的。这种行为称为per thread arena。

示例

```
1   /* Per thread arena example. */
2   #include <stdio.h>
3   #include <stdlib.h>
4   #include <pthread.h>
5   #include <unistd.h>
6   #include <sys/types.h>
7
8   void* threadFunc(void* arg) {
9           printf("Before malloc in thread 1\n");
10          getchar();
11          char* addr = (char*) malloc(1000);
12          printf("After malloc and before free in thread 1\n");
13          getchar();
14          free(addr);
15          printf("After free in thread 1\n");
16          getchar();
17  }
18
```

线程中调用malloc函数，分配1000个字节

示例

```
19  int main() {
20          pthread_t t1;
21          void* s;
22          int ret;
23          char* addr;
24
25          printf("Welcome to per thread arena example::%d\n",getpid());
26          printf("Before malloc in main thread\n");
27          getchar();
28          addr = (char*) malloc(1000);
29          printf("After malloc and before free in main thread\n");
30          getchar();
31          free(addr);
32          printf("After free in main thread\n");
33          getchar();
34          ret = pthread_create(&t1, NULL, threadFunc, NULL);
35          if(ret)
36          {
37                  printf("Thread creation error\n");
38                  return -1;
39          }
40          ret = pthread_join(t1, &s);
41          if(ret)
42          {
43                  printf("Thread join error\n");
44                  return -1;
45          }
46          return 0;
47  }
```

主程序中调用malloc函数，分配1000个字节

编译：gcc -o mthread mthread.c -lpthread

示例

- 在主程序中进行 malloc 之前:在下面的输出中，操作系统预先分配了堆空间。

在一个终端运行

```
os@os:~$ ./mthread
Welcome to per thread arena example::3324
Before malloc in main thread
```

在另一个终端运行

```
os@os:~$ cat /proc/3324/maps
aaaad0850000-aaaad0851000 r-xp 00000000 fd:00 1179786                    /home/os/mthread
aaaad0861000-aaaad0862000 r--p 00001000 fd:00 1179786                    /home/os/mthread
aaaad0862000-aaaad0863000 rw-p 00002000 fd:00 1179786                    /home/os/mthread
aaab08267000-aaab08288000 rw-p 00000000 00:00 0                          [heap]
ffff83610000-ffff83799000 r-xp 00000000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff83799000-ffff837a8000 ---p 00189000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837a8000-ffff837ac000 r--p 00188000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ac000-ffff837ae000 rw-p 0018c000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ae000-ffff837ba000 rw-p 00000000 00:00 0
ffff837cf000-ffff837fa000 r-xp 00000000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff83804000-ffff83806000 rw-p 00000000 00:00 0
ffff83806000-ffff83808000 r--p 00000000 00:00 0                          [vvar]
ffff83808000-ffff83809000 r-xp 00000000 00:00 0                          [vdso]
ffff83809000-ffff8380b000 r--p 0002a000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff8380b000-ffff8380d000 rw-p 0002c000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffe1258000-ffffe1279000 rw-p 00000000 00:00 0                          [stack]
```



示例

- 在主程序中进行 malloc 之后，free之前：在下面的输出中，由于主程序使用malloc申请分配1000个字节，在已分配[heap]的范围之内，内存分配没有变化。

在一个终端运行

```
os@os:~$ ./mthread
Welcome to per thread arena example::3324
Before malloc in main thread

After malloc and before free in main thread
```

在另一个终端运行

```
os@os:~$ cat /proc/3324/maps
aaaad0850000-aaaad0851000 r-xp 00000000 fd:00 1179786                    /home/os/mthread
aaaad0861000-aaaad0862000 r--p 00001000 fd:00 1179786                    /home/os/mthread
aaaad0862000-aaaad0863000 rw-p 00002000 fd:00 1179786                    /home/os/mthread
aaab08267000-aaab08288000 rw-p 00000000 00:00 0                          [heap]
ffff83610000-ffff83799000 r-xp 00000000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff83799000-ffff837a8000 ---p 00189000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837a8000-ffff837ac000 r--p 00188000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ac000-ffff837ae000 rw-p 0018c000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ae000-ffff837ba000 rw-p 00000000 00:00 0
ffff837cf000-ffff837fa000 r-xp 00000000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff83804000-ffff83806000 rw-p 00000000 00:00 0
ffff83806000-ffff83808000 r--p 00000000 00:00 0                          [vvar]
ffff83808000-ffff83809000 r-xp 00000000 00:00 0                          [vdso]
ffff83809000-ffff8380b000 r--p 0002a000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff8380b000-ffff8380d000 rw-p 0002c000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffe1258000-ffffe1279000 rw-p 00000000 00:00 0                          [stack]
```



示例

- 在主程序中进行free之后：内存分配没有变化。

在一个终端运行

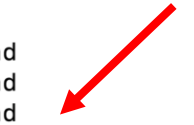
```
os@os:~$ ./mthread
Welcome to per thread arena example::3324
Before malloc in main thread

After malloc and before free in main thread

After free in main thread
```

在另一个终端运行

```
os@os:~$ cat /proc/3324/maps
aaaad0850000-aaaad0851000 r-xp 00000000 fd:00 1179786                    /home/os/mthread
aaaad0861000-aaaad0862000 r--p 00001000 fd:00 1179786                    /home/os/mthread
aaaad0862000-aaaad0863000 rw-p 00002000 fd:00 1179786                    /home/os/mthread
aaab08267000-aaab08288000 rw-p 00000000 00:00 0                          [heap]
ffff83610000-ffff83799000 r-xp 00000000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff83799000-ffff837a8000 ---p 00189000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837a8000-ffff837ac000 r--p 00188000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ac000-ffff837ae000 rw-p 0018c000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ae000-ffff837ba000 rw-p 00000000 00:00 0
ffff837cf000-ffff837fa000 r-xp 00000000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff83804000-ffff83806000 rw-p 00000000 00:00 0
ffff83806000-ffff83808000 r--p 00000000 00:00 0                          [vvar]
ffff83808000-ffff83809000 r-xp 00000000 00:00 0                          [vdso]
ffff83809000-ffff8380b000 r--p 0002a000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff8380b000-ffff8380d000 rw-p 0002c000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffe1258000-ffffe1279000 rw-p 00000000 00:00 0                          [stack]
```



示例

- 在线程thread 1中进行 malloc 之前：在下面的输出中，主程序创建了线程，操作系统分配了2个内存空间ffff82e00000-ffff82e10000和ffff82e10000-ffff83610000。

在一个终端运行

```
os@os:~$ ./mthread
Welcome to per thread arena example::3324
Before malloc in main thread

After malloc and before free in main thread

After free in main thread

Before malloc in thread 1
```

在另一个终端运行

```
os@os:~$ cat /proc/3324/maps
aaaad0850000-aaaad0851000 r-xp 00000000 fd:00 1179786                    /home/os/mthread
aaaad0861000-aaaad0862000 r--p 00001000 fd:00 1179786                    /home/os/mthread
aaaad0862000-aaaad0863000 rw-p 00002000 fd:00 1179786                    /home/os/mthread
aaab08267000-aaab08288000 rw-p 00000000 00:00 0                          [heap]
ffff82e00000-ffff82e10000 ---p 00000000 00:00 0
ffff82e10000-ffff83610000 rw-p 00000000 00:00 0
ffff83610000-ffff83799000 r-xp 00000000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff83799000-ffff837a8000 ---p 00189000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837a8000-ffff837ac000 r--p 00188000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ac000-ffff837ae000 rw-p 0018c000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ae000-ffff837ba000 rw-p 00000000 00:00 0
ffff837cf000-ffff837fa000 r-xp 00000000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff83804000-ffff83806000 rw-p 00000000 00:00 0
ffff83806000-ffff83808000 r--p 00000000 00:00 0                          [vvar]
ffff83808000-ffff83809000 r-xp 00000000 00:00 0                          [vdso]
ffff83809000-ffff8380b000 r--p 0002a000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff8380b000-ffff8380d000 rw-p 0002c000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffe1258000-ffffe1279000 rw-p 00000000 00:00 0                          [stack]
```

创建线程时分配的空间

示例

- 在线程thread 1中进行malloc之后，free之前：在下面的输出中，操作系统为线程再一次分配了2个内存空间ffff7c000000-ffff7c021000和ffff7c021000-ffff80000000。

在一个终端运行

```
os@os:~$ ./mthread
Welcome to per thread arena example::3324
Before malloc in main thread

After malloc and before free in main thread

After free in main thread

Before malloc in thread 1

After malloc and before free in thread 1
```

在另一个终端运行

```
os@os:~$ cat /proc/3324/maps
aaaad0850000-aaaad0851000 r-xp 00000000 fd:00 1179786                    /home/os/mthread
aaaad0861000-aaaad0862000 r--p 00001000 fd:00 1179786                    /home/os/mthread
aaaad0862000-aaaad0863000 rw-p 00002000 fd:00 1179786                    /home/os/mthread
aaab08267000-aaab08288000 rw-p 00000000 00:00 0                          [heap]
ffff7c000000-ffff7c021000 rw-p 00000000 00:00 0
ffff7c021000-ffff80000000 ---p 00000000 00:00 0
ffff82e00000-ffff82e10000 ---p 00000000 00:00 0
ffff82e10000-ffff83610000 rw-p 00000000 00:00 0
ffff83610000-ffff83799000 r-xp 00000000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff83799000-ffff837a8000 ---p 00189000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837a8000-ffff837ac000 r--p 00188000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ac000-ffff837ae000 rw-p 0018c000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ae000-ffff837ba000 rw-p 00000000 00:00 0
ffff837cf000-ffff837fa000 r-xp 00000000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff83804000-ffff83806000 rw-p 00000000 00:00 0
ffff83806000-ffff83808000 r--p 00000000 00:00 0                          [vvar]
ffff83808000-ffff83809000 r-xp 00000000 00:00 0                          [vdso]
ffff83809000-ffff8380b000 r--p 0002a000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff8380b000-ffff8380d000 rw-p 0002c000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffe1258000-ffffe1279000 rw-p 00000000 00:00 0                          [stack]
```

线程调用malloc分配的空间

创建线程时分配的空间

示例

- 在线程thread 1中进行free之后：内存分配没有变化。

在一个终端运行

```
os@os:~$ ./mthread
Welcome to per thread arena example::3324
Before malloc in main thread

After malloc and before free in main thread

After free in main thread

Before malloc in thread 1

After malloc and before free in thread 1

After free in thread 1
```

在另一个终端运行

```
os@os:~$ cat /proc/3324/maps
aaaad0850000-aaaad0851000 r-xp 00000000 fd:00 1179786                    /home/os/mthread
aaaad0861000-aaaad0862000 r--p 00001000 fd:00 1179786                    /home/os/mthread
aaaad0862000-aaaad0863000 rw-p 00002000 fd:00 1179786                    /home/os/mthread
aaab08267000-aaab08288000 rw-p 00000000 00:00 0                          [heap]
ffff7c000000-ffff7c021000 rw-p 00000000 00:00 0
ffff7c021000-ffff80000000 ---p 00000000 00:00 0
ffff82e00000-ffff82e10000 ---p 00000000 00:00 0
ffff82e10000-ffff83610000 rw-p 00000000 00:00 0
ffff83610000-ffff83799000 r-xp 00000000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff83799000-ffff837a8000 ---p 00189000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837a8000-ffff837ac000 r--p 00188000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ac000-ffff837ae000 rw-p 0018c000 fd:00 919000                     /usr/lib/aarch64-linux-gnu/libc.so.6
ffff837ae000-ffff837ba000 rw-p 00000000 00:00 0
ffff837cf000-ffff837fa000 r-xp 00000000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff83804000-ffff83806000 rw-p 00000000 00:00 0
ffff83806000-ffff83808000 r--p 00000000 00:00 0                          [vvar]
ffff83808000-ffff83809000 r-xp 00000000 00:00 0                          [vdso]
ffff83809000-ffff8380b000 r--p 0002a000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffff8380b000-ffff8380d000 rw-p 0002c000 fd:00 918852                     /usr/lib/aarch64-linux-gnu/ld-linux-aarch64.so.1
ffffe1258000-ffffe1279000 rw-p 00000000 00:00 0                          [stack]
```

线程调用malloc分配的空间

创建线程时分配的空间

Arena的个数

- 在上面的例子中，我们看到主程序包含主 arena，thread 1 包含它自己的线程 arena。那么，无论线程数量多少，线程和 arena 之间是否可以存在一对一的映射？

当然不是。一个疯狂的应用程序可能包含更多数量的线程（多于CPU核心数量），在这种情况下，每个线程拥有一个 arena 会变得代价太大且无用。

因此，由于这个原因，malloc在实现上对arena个做了限制。

对于32位系统：arena个数 = 2 * CPU核心数
对于64位系统：arena个数 = 8 * CPU核心数

malloc/malloc.c

```
1940    .trim_threshold = DEFAULT_TRIM_THRESHOLD,
1941 #define NARENAS_FROM_NCORES(n) ((n) * (sizeof (long) == 4 ? 2 : 8))
1942    .arena_test = NARENAS_FROM_NCORES (1)
```

Other Approaches: Buddy Allocation(Cont.)

- Buddy allocation can suffer from **internal fragmentation** **(内部碎片)**.
- Buddy system makes **coalescing** **（合并）** simple.
  - **Coalescing** two blocks in to the next level of block.
  - 判断任意block的buddy block是否in use很简单： **地址只有1 bit区别**

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
   a) 基础概念（基础）
   b) 页表（基础）
   c) 访问分页内存（基础）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Concept of Paging

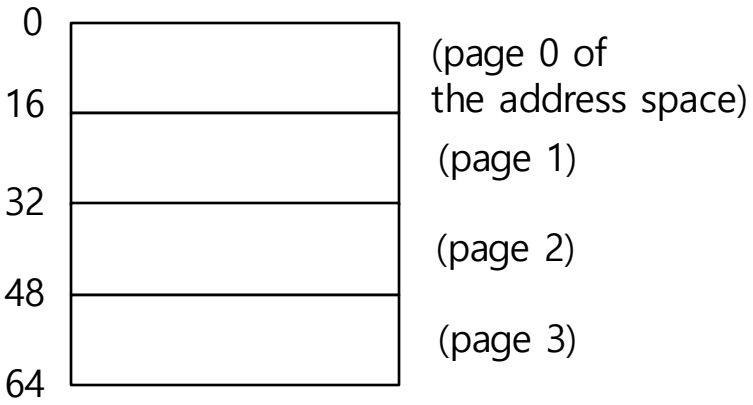
- Paging **splits up** address space into **fixed-zed** unit called a **page**.
  - 回顾对比 **Segmentation**: variable size of logical segments(code, stack, heap, etc.)
- With paging, **physical memory** is also **split** into some number of pages called a **page frame**.
- **Page table** per process is needed **to translate** the virtual address to physical address. 在内存的分页管理机制下，需要引入额外的数据结构Page Table，用以把虚拟地址翻译成物理地址

Advantages Of Paging

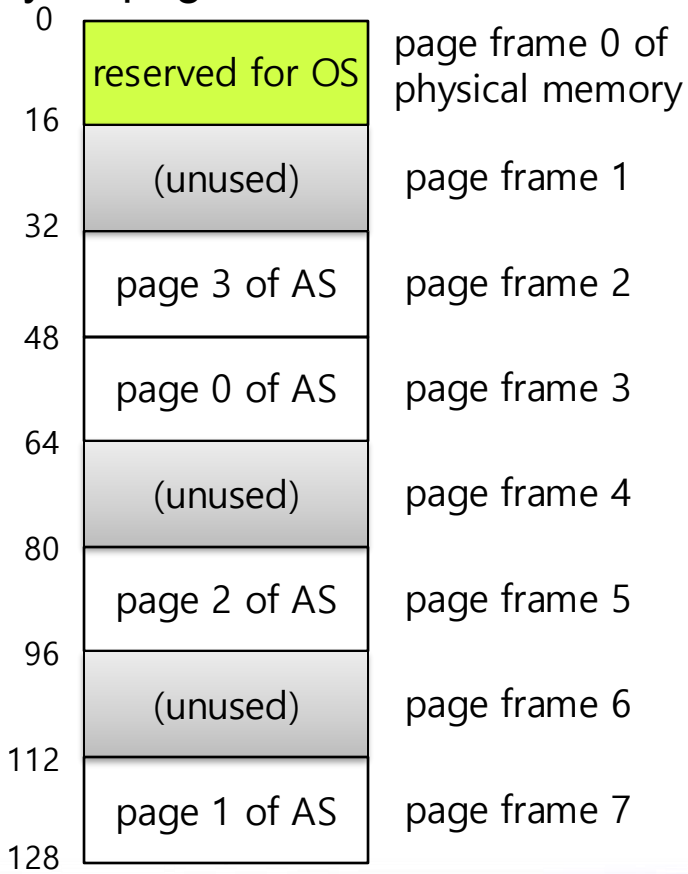
- **Flexibility: 通用性更好** Supporting the abstraction of address space effectively
  - 不需要关心heap and stack地址如何增长，内存如何使用
- **Simplicity**: **空闲空间管理简单** ease of free-space management
  - The page in address space and the page frame are the same size.
  - Easy to allocate and keep a free list

Example: A Simple Paging

- **Physical**: 128-byte physical memory with 16 bytes page frames
- **Virtual**: 64-byte address space with 16 bytes pages



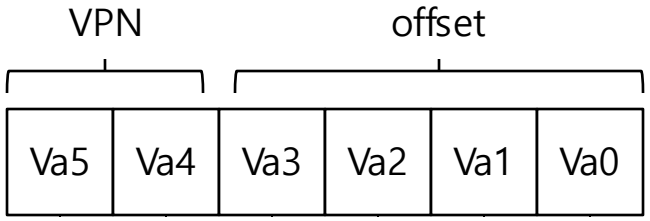
A Simple 64-byte Virtual Address Space



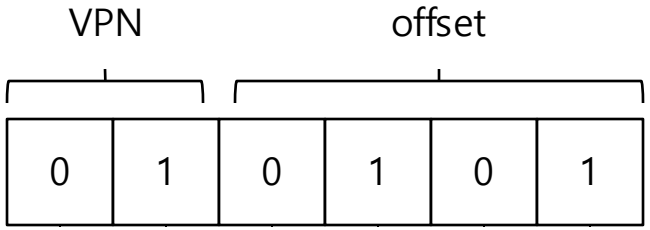
64-Byte Process Address Space Placed In 128-Byte Physical Memory

Address Translation

- Two components in the virtual address
  - **VPN**: Virtual Page Number
  - **Offset**: offset within the page

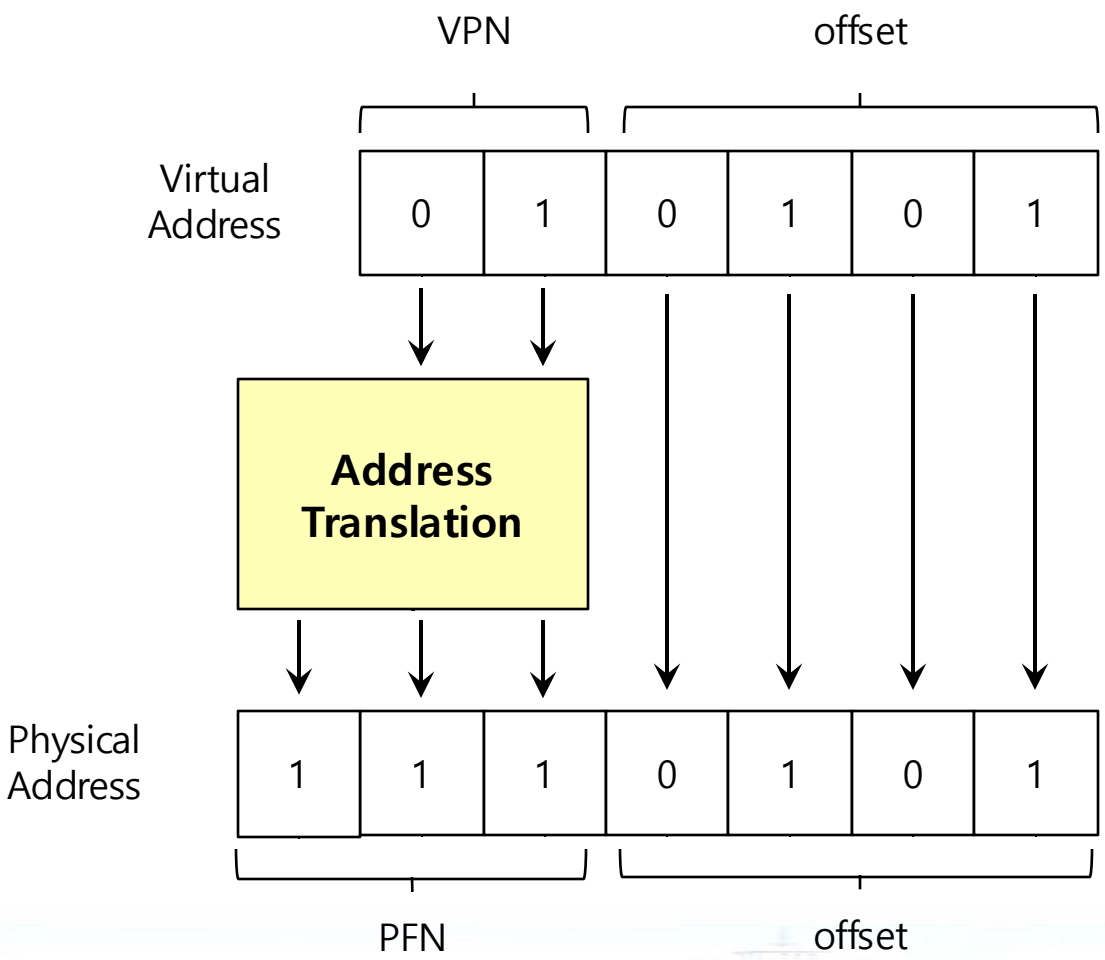


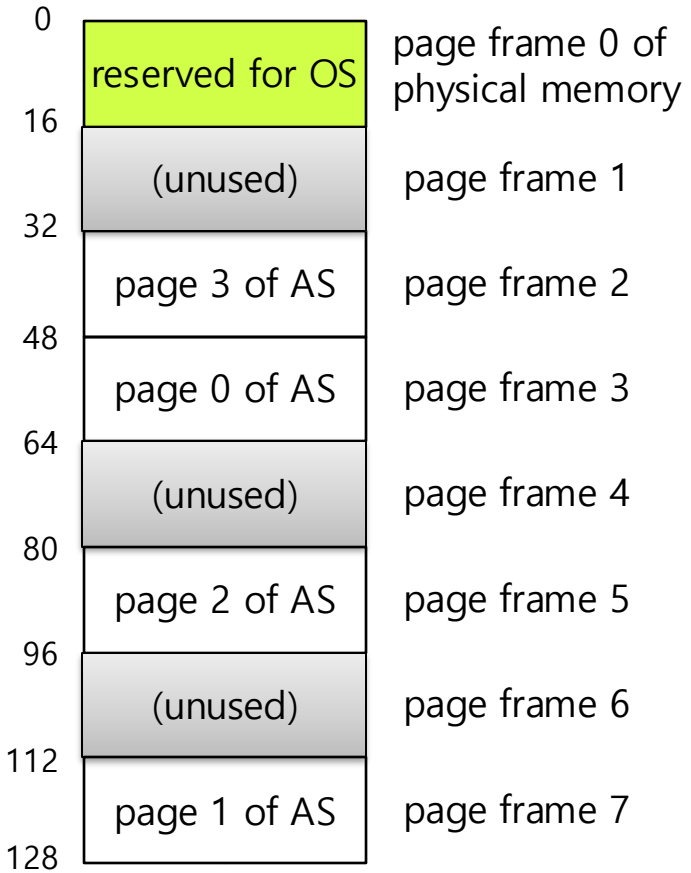
- Example: virtual address 21 in 64-byte address space



Example: Address Translation

- The virtual address 21 in 64-byte address space





64-Byte Address Space Placed In Physical Memory

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
   a) 基础概念（基础）
   b) 页表（基础）
   c) 访问分页内存（基础）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Where Are Page Tables Stored?

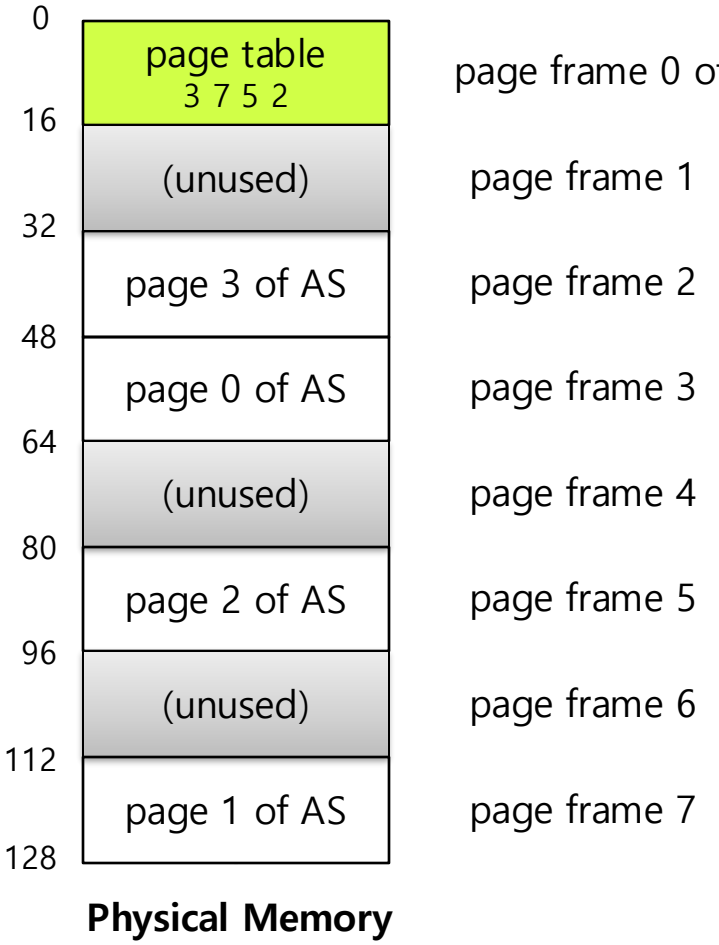
- 每个process都有一个Page Table，which can get awfully large
  - 32-bit address space, with 4KB pages，20bits for VPN
    - 4MB = 2^{20} *entries * 4Bytes per page table entry*
- Page Table存储在内存
  - Because page tables are so big, we don't keep any special on-chip hardware in the MMU to store the page table of the currently-running process.
  - Instead, we store the page table for each process in memory somewhere

What Is In The Page Table?

- The page table is just a **data structure** that is used to map the virtual address to physical address.
  - Simplest form: a linear page table, an array

- The OS **indexes** the array by VPN, and looks up the page-table entry.

Example: Page Table in Kernel Physical Memory

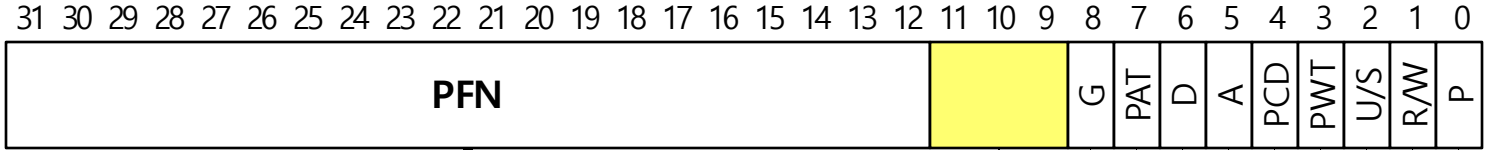


- page frame 0 of physical memory 给出了一个简单的page table示例
- 注意：
  - VPN就不需要存储了
  - Index代表PFN

Common Flags Of Page Table Entry

- **Valid Bit**: Indicating whether the particular translation is valid.
- **Protection Bit**: Indicating whether the page could be read from, written to, or executed from
- **Present Bit**: Indicating whether this page is in physical memory or on disk(swapped out)
- **Dirty Bit**: Indicating whether the page has been modified since it was brought into memory
- **Reference Bit(Accessed Bit):** Indicating that a page has been accessed

Example: x86 Page Table Entry (PTE)



- P: present
- R/W: read/write bit
- U/S: supervisor
- A: accessed bit
- D: dirty bit
- PFN: the page frame number

- Read the Intel Architecture Manuals for more details on x86 paging support.
- Be forewarned, however; reading manuals such as these, while quite informative can be challenging at first. A little **patience**, and a lot of **desire**, is required.

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
   a) 基础概念（基础）
   b) 页表（基础）
   c) 访问分页内存（基础）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 交换（Swapping）

Paging: Too Slow

- To find a location of the desired PTE, the **starting location** of the page table is **needed**.

- For every memory reference, paging requires the OS to perform one **extra memory reference, 准确的说是至少一次**

Accessing Memory With Paging


```
1       // Extract the VPN from the virtual address
2       VPN = (VirtualAddress & VPN_MASK) >> SHIFT
3
4       // Form the address of the page-table entry (PTE)
5       PTEAddr = PTBR + (VPN * sizeof(PTE))
6
7       // Fetch the PTE
8       PTE = AccessMemory(PTEAddr)
9
10      // Check if process can access the page
11      if (PTE.Valid == False)
12              RaiseException(SEGMENTATION_FAULT)
13      else if (CanAccess(PTE.ProtectBits) == False)
14              RaiseException(PROTECTION_FAULT)
15      else
16              // Access is OK: form physical address and fetch it
17              offset = VirtualAddress & OFFSET_MASK
18              PhysAddr = (PTE.PFN << PFN_SHIFT) | offset
19              Register = AccessMemory(PhysAddr)
```

A Memory Trace

- Example: A Simple Memory Access

```
int array[1000];
...
for (i = 0; i < 1000; i++)
        array[i] = 0;
```

- Compile and execute

```
prompt> gcc -o array array.c -Wall -o
prompt>./array
```

- Resulting Assembly code

```
0x1024 movl $0x0,(%edi,%eax,4)
0x1028 incl %eax
0x102c cmpl $0x03e8,%eax
0x1030 jne 0x1024
```

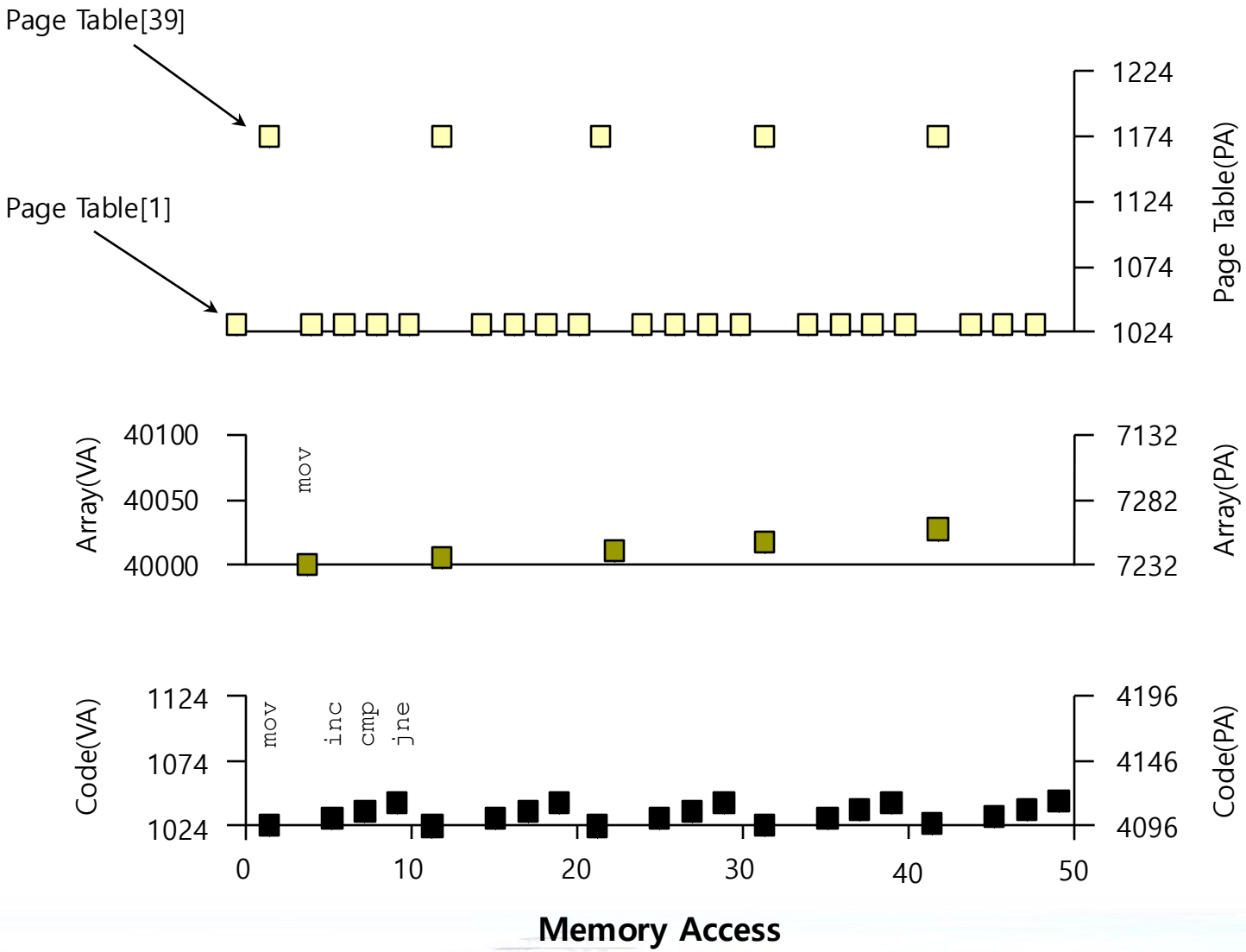
A Virtual(And Physical) Memory Trace

Some assumptions:

- virtual address space size: 64KB
- page size:1KB
- a linear (array-based) page table at physical address 1KB (1024)
- Code: virtual address 1024; VPN=1 → PFN=4
- Array: virtual addresses 40000 through 44000; VPN=39, ... , VPN=42

VPN	PFN
39	7
40	8
41	9
42	10

A Virtual(And Physical) Memory Trace



A Virtual(And Physical) Memory Trace

each instruction fetch will generate two memory references:

- access the page table to find the physical frame that the instruction resides within
- access the instruction itself to fetch it to the CPU for processing

mov instruction will generate one explicit memory reference:

- translate the array virtual address to the correct physical one

each array element will generate one memory references.

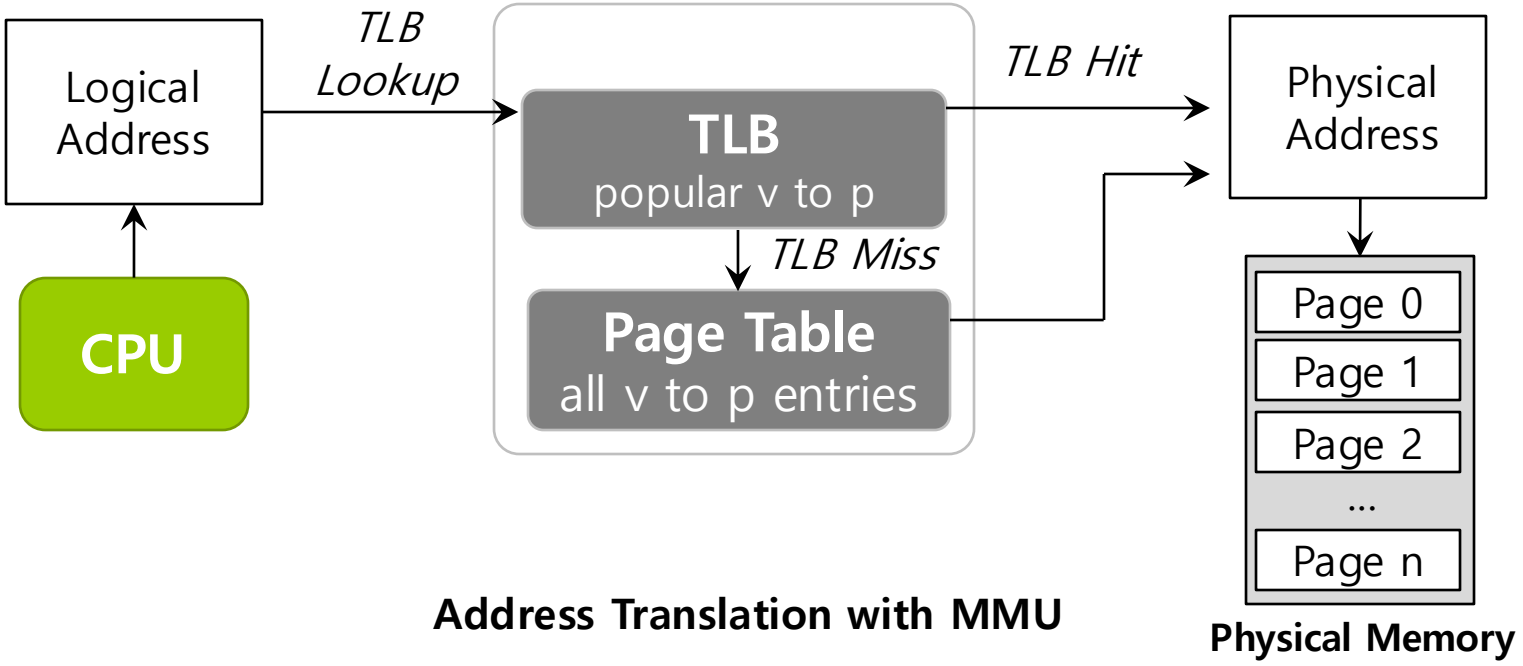
For each element of array:
Mov instruction accesses the page table (topmost graph) and the instruction itself (bottom graph), then accesses array virtual address (topmost graph). After that, the array accesses memory (middle graph). The other three instructions access memory twice for each.

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
   a) TLB基础算法（基础）
   b) TLB控制流算法（基础）
   c) TLB替换策略（基础）
8. 交换（Swapping）

TLB

- Part of the chip's Memory-Management Unit (MMU).
- A hardware cache of **popular** virtual-to-physical address translation.



Address Translation with MMU

TLB Basic Algorithms

```
1: VPN = (VirtualAddress & VPN_MASK ) >> SHIFT
2: (Success , TlbEntry) = TLB_Lookup(VPN)
3:    if(Success == Ture){ // TLB Hit
4:    if(CanAccess(TlbEntry.ProtectBit) == True ){
5:        offset = VirtualAddress & OFFSET_MASK
6:        PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:        AccessMemory( PhysAddr )
8:    }else RaiseException(PROTECTION_ERROR)
```

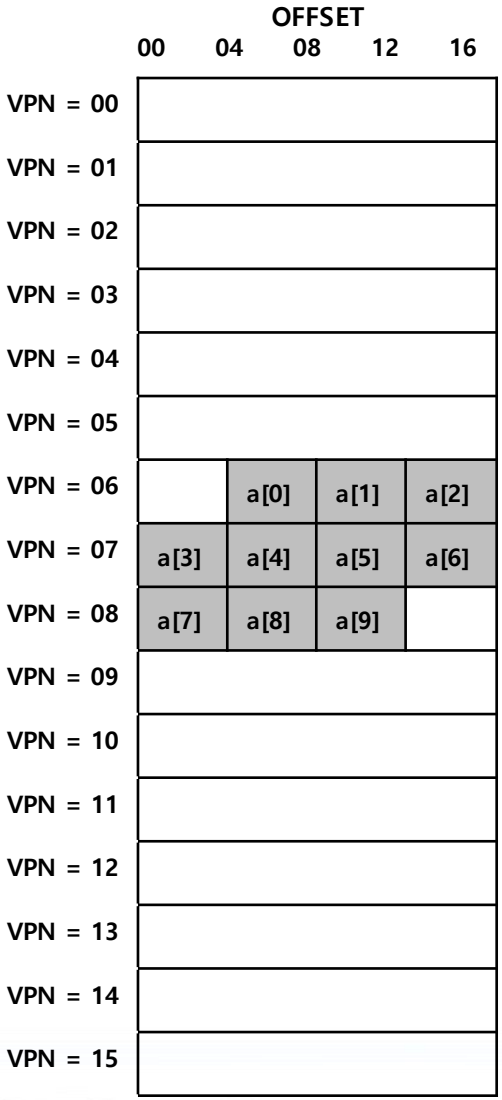
- (1 lines) extract the virtual page number(VPN).
- (2 lines) check if the TLB holds the transalation for this VPN.
- (5-8 lines) extract the page frame number from the relevant TLB entry, and form the desired physical address and access memory.

TLB Basic Algorithms (Cont.)

```
11:     }else{ //TLB Miss
12:         PTEAddr = PTBR + (VPN * sizeof(PTE))
13:         PTE = AccessMemory(PTEAddr)
14:         (…)
15:     }else{
16:         TLB_Insert( VPN , PTE.PFN , PTE.ProtectBits)
17:         RetryInstruction()
18:     }
19:}
```

- (11-12 lines) The hardware accesses the page table to find the translation. Line12，PTBR (Page Table Base Register)由硬件记录页表在内存的起始位置，以及硬件需要清楚页表的结构，即可算出对应页表项的内存地址
- (16 lines) updates the TLB with the translation.

Example: Accessing An Array




```
0:      int sum = 0 ;
1:      for( i=0; i<10; i++){
2:              sum+=a[i];
3:      }
```

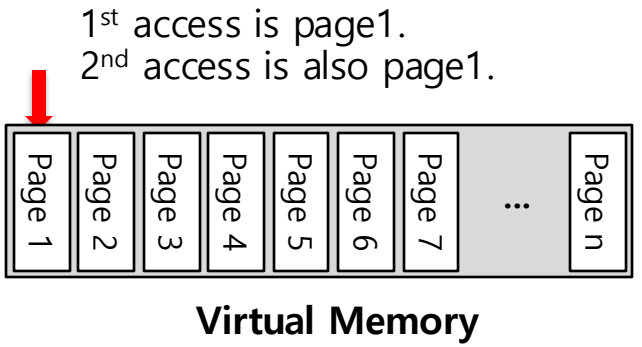
- assume an array of 10 4-byte integers in memory, starting at virtual address 100.
- a small 8-bit virtual address space, with 16-byte pages
- pretend that the only memory accesses the loop generates are to the array
-

The TLB improves performance due to spatial locality

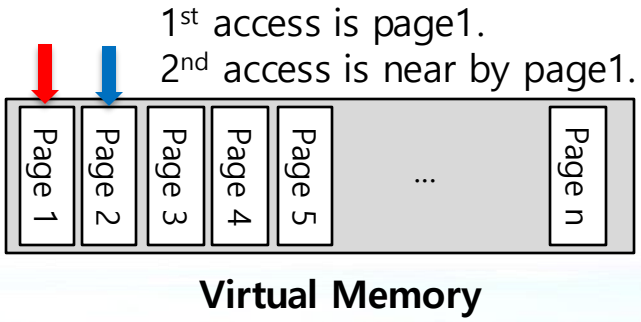
3 misses and 7 hits.
Thus TLB hit rate is 70%.

尽可能利用好TLB缓存!

- Temporal Locality
  - An instruction or data item that has been recently accessed will likely be re-accessed soon in the future.



- Spatial Locality
  - If a program accesses memory at address `x`, it will likely soon access memory near `x`.



Who Handles The TLB Miss?

- 2种方式：硬件或软件
- Hardware handle the TLB miss entirely on CISC（往往由复杂指令集需要提供特定的CPU指令）.
  - The hardware has to know exactly where the page tables are located in memory.
  - The hardware would “walk” the page table, find the correct page-table entry and extract the desired translation, update and retry instruction.
  - **hardware-managed TLB.**
- 更现代体系结构，采用**software-managed TLB，即**：RISC 架构中，若TLB miss，硬件会抛出一个异常（ raises exception ），暂停当前指令流，将特权级提升至内核模式，跳转到trap handler程序进行处理
  - **Trap handler is code** within the OS that is written with the express purpose of handling TLB miss
  - 从trap handler返回的指令不同于系统调用的trap return指令
  - trap handler要避免无限递归

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
   a) TLB基础算法（基础）
   b) TLB控制流算法（基础）
   c) TLB替换策略（基础）
8. 交换（Swapping）

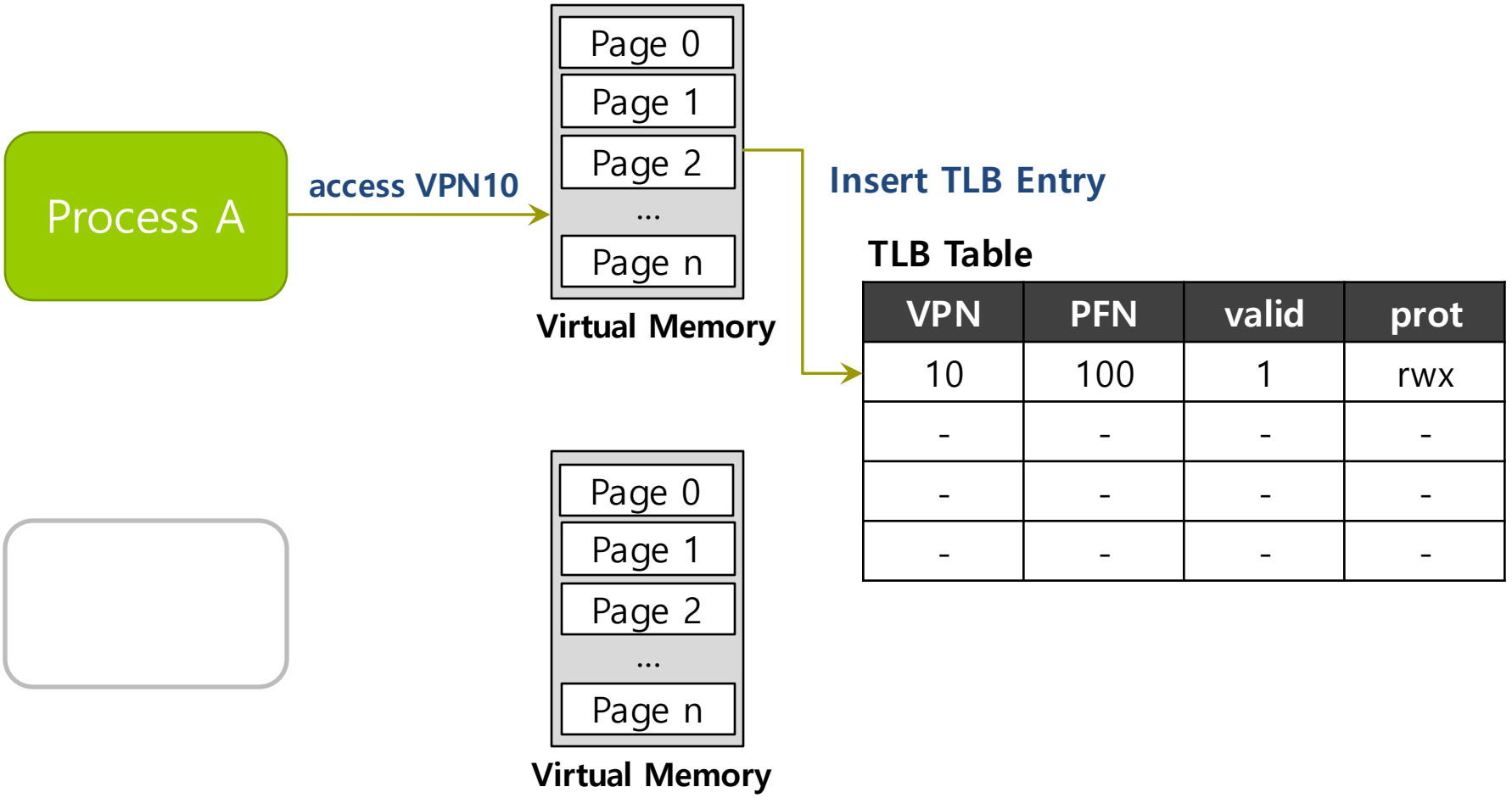
TLB entry

- TLB is managed by **Full Associative** method.
  - A typical TLB might have 32,64, or 128 entries.
  - Hardware search the entire TLB in parallel to find the desired translation.
  - other bits: valid bits , protection bits, address-space identifier, dirty bit

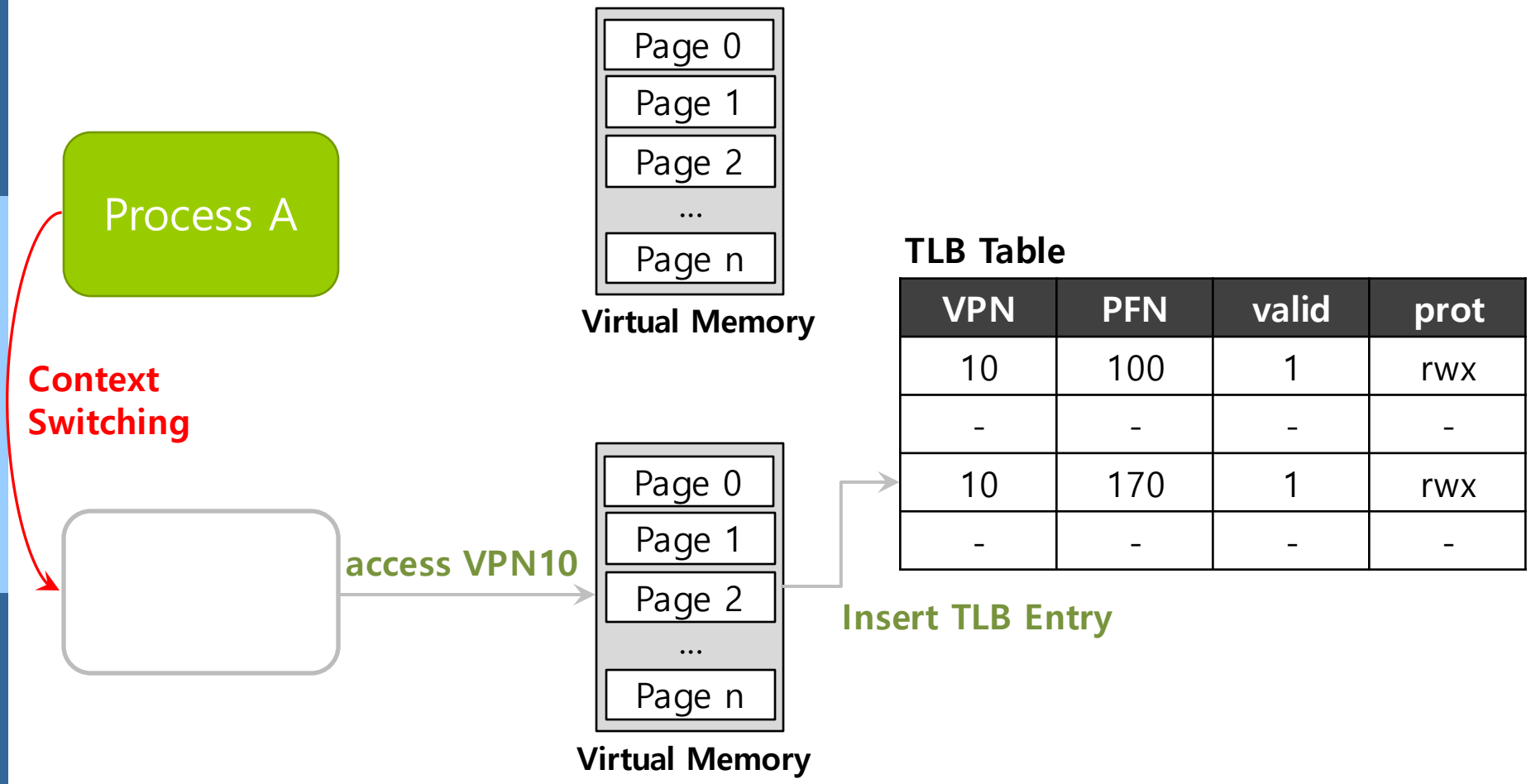


Typical TLB entry look like this

TLB Issue: Context Switching

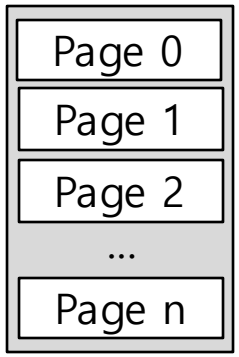


TLB Issue: Context Switching

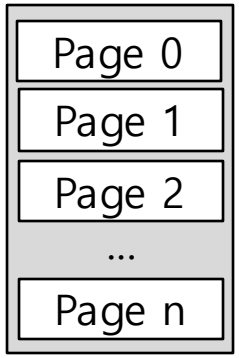


TLB Issue: Context Switching

Process A



Virtual Memory



Virtual Memory

TLB Table

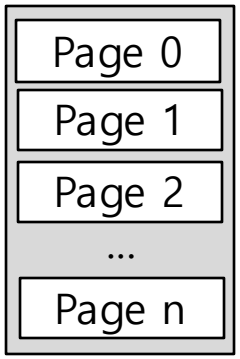
VPN	PFN	valid	prot
10	100	1	rwx
-	-	-	-
10	170	1	rwx
-	-	-	-

Can't Distinguish which entry is meant for which process

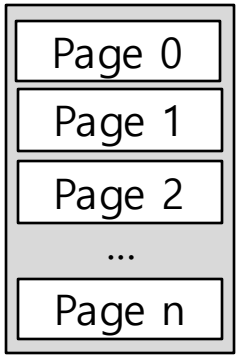
To Solve Problem

- Provide an address space identifier(ASID) field in the TLB.

Process A



Virtual Memory



Virtual Memory

TLB Table

VPN	PFN	valid	prot	ASID
10	100	1	rwx	1
-	-	-	-	-
10	170	1	rwx	2
-	-	-	-	-

例题

关于分段和分页的描述，正确的是（ ）。

A. 分段是信息的逻辑单位，段长由系统决定

B. 引入分段的主要目的是实现离散分配并提高内存利用率

C. 分页是信息的物理单位，页长由用户决定

D. 页面在物理内存中只能从页面大小的整数倍地址开始存放

答案：D

A错误，分段是由用户程序定义的逻辑单位，段长由用户或程序决定。

B错误，分段的目的是提供灵活的内存管理，而不仅仅是提高内存利用率。

C错误，分页的页长是由系统而非用户定义的。

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
   a) TLB基础算法（基础）
   b) TLB控制流算法（基础）
   c) TLB替换策略（基础）
8. 交换（Swapping）

Another Case

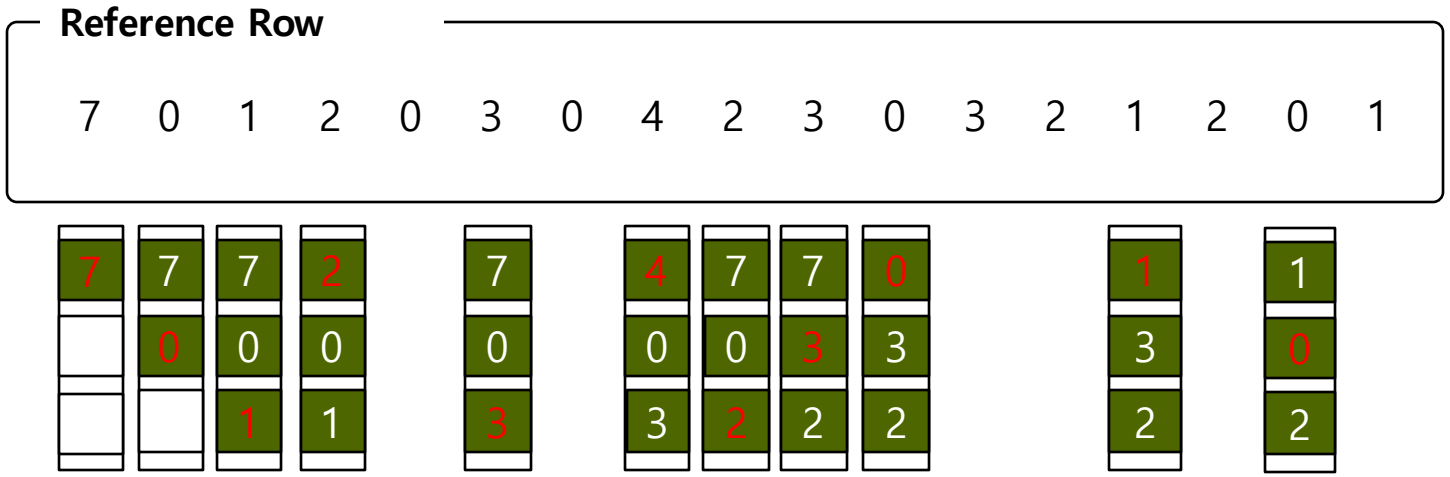
- Two processes share a page.
  - Process 1 is sharing physical page 101 with Process2.
  - P1 maps this page into the 10th page of its address space.
  - P2 maps this page to the 50th page of its address space.

VPN	PFN	valid	prot	ASID
10	101	1	rwx	1
-	-	-	-	-
50	101	1	rwx	2
-	-	-	-	-

Sharing of pages is useful as it reduces the number of physical pages in use.

TLB Replacement Policy

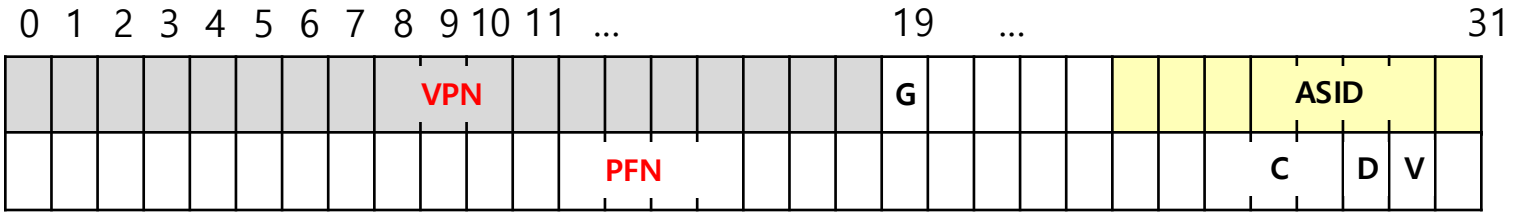
- LRU(Least Recently Used)
  - Evict (驱逐) an entry that has not recently been used.
  - Take advantage of *locality* (局限性) in the memory-reference stream.



Total 11 TLB miss

A Real TLB Entry

All 64 bits of this TLB entry(example of MIPS R4000)



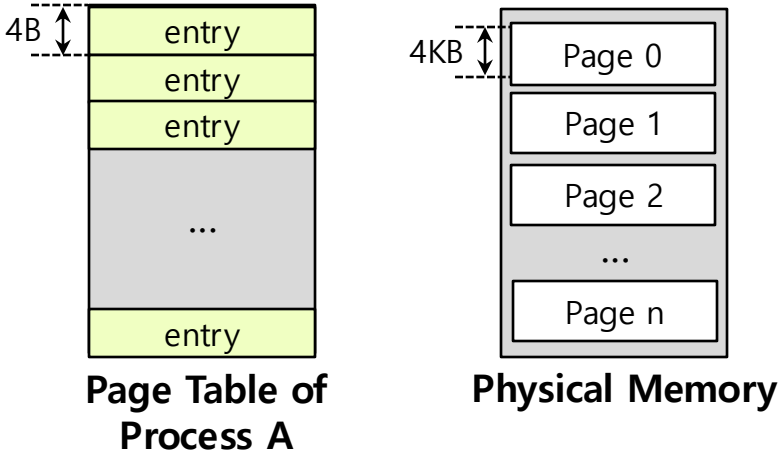
Flag	Content
19-bit VPN	The rest reserved for the kernel.
24-bit PFN	Systems can support with up to 64GB of main memory($2^{24} * 4KB$ pages).
Global bit(G)	Used for pages that are globally-shared among processes.
ASID	OS can use to distinguish between address spaces.
Coherence bit(C)	determine how a page is cached by the hardware.
Dirty bit(D)	marking when the page has been written.
Valid bit(V)	tells the hardware if there is a valid translation present in the entry.

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 多级页表
   a) 线性页表（基础）
   b) 段页式方法（基础）
   c) 多级页表（基础）
9. 交换（Swapping）

Paging: Linear Tables

- 每个进程都有一个page table
  - Assume that 32-bit address space with 4KB pages and 4-byte page-table entry.

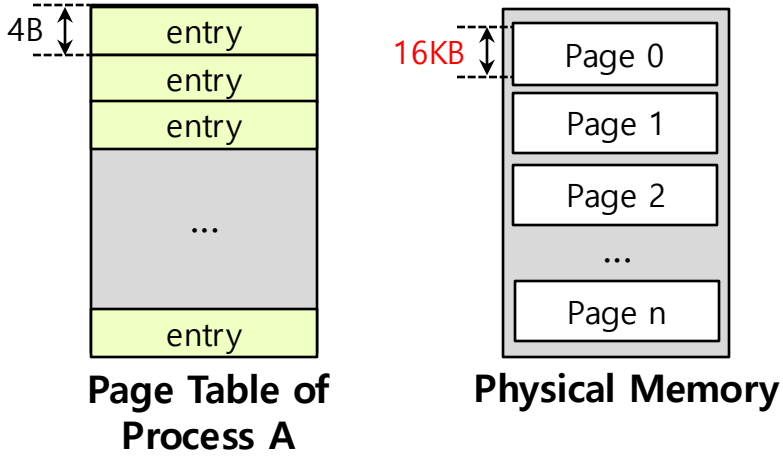


$2^{32} / 2^{12} * 4 = 4MB$ per page table

Page table are too big and thus consume too much memory.

Smaller Tables: Big pages

- Page table are too big and thus consume too much memory.
  - Assume that 32-bit address space with 16KB pages and 4-byte page-table entry.

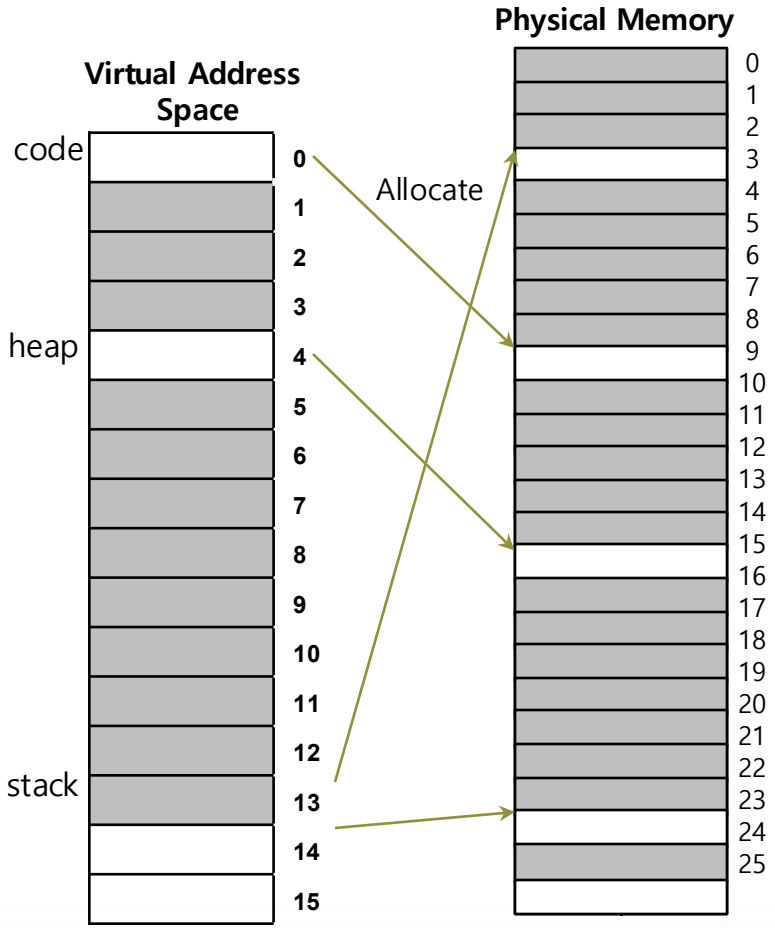


$2^{32} / 2^{16} * 4 = 1MB$ per page table

Big pages lead to internal fragmentation.

Problem

- Single page table for the entries address space of process.



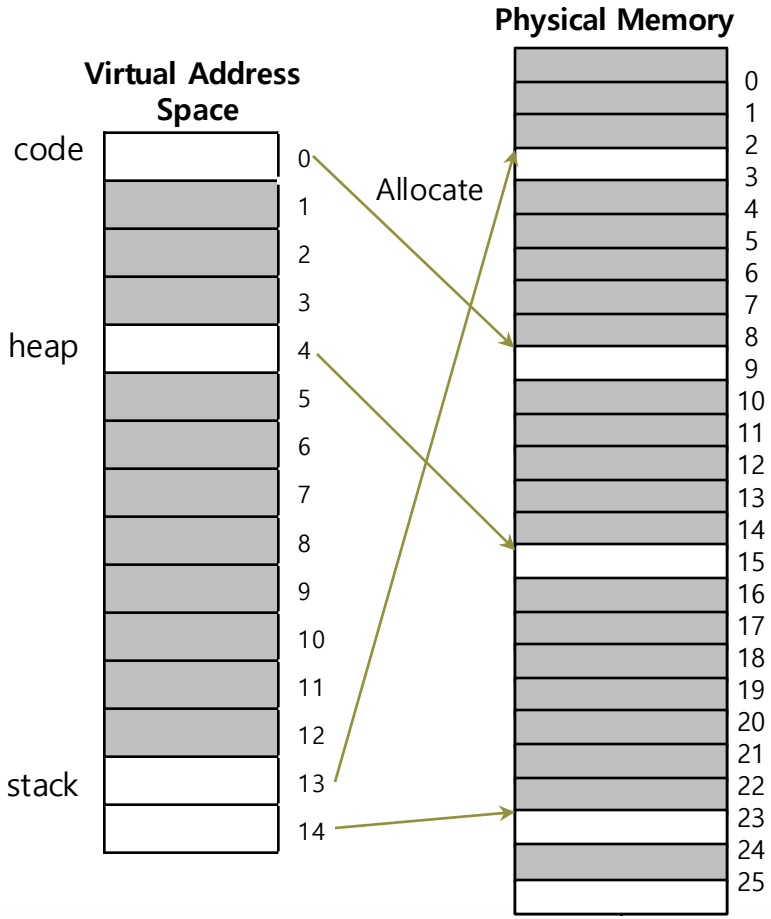
A 16KB Address Space with 1KB Pages

PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
15	1	rw-	1	1
...	...	...	...	...
-	0	-	-	-
3	1	rw-	1	1
23	1	rw-	1	1

A Page Table For 16KB Address Space

Problem

- Most of the page table is **unused**, full of invalid entries.



A 16KB Address Space with 1KB Pages

PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	-	-	-
-	0	-	-	-
-	0	-	-	-
15	1	rw-	1	1
...	...	...	...	...
-	0	-	-	-
3	1	rw-	1	1
23	1	rw-	1	1

A Page Table For 16KB Address Space

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 多级页表
   a) 线性页表（基础）
   b) 段页式方法（基础）
   c) 多级页表（基础）
9. 交换（Swapping）

Hybrid Approach: Paging and Segments

- In order to reduce the memory overhead of page tables.
  - Using base not to point to the segment itself but rather to hold the physical address of the page table of that segment.
  - The bounds register is used to indicate the end of the page table.

TLB miss on Hybrid Approach

- The hardware get to **physical address** from **page table**.
  - The hardware uses the segment bits(SN) to determine which base and bounds pair to use.
  - The hardware then takes the physical address therein and combines it with the VPN as follows to form the address of the page table entry(PTE) .

```
01:      SN = (VirtualAddress & SEG_MASK) >> SN_SHIFT
02:      VPN = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
03:      AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

Problem of Hybrid Approach

- Hybrid Approach is not without problems.
  - If we have a large but sparsely-used （大但很少使用的） heap, we can still end up with a lot of page table waste.
  - Causing external fragmentation （碎片） to arise again.

Big pages 和hybrid都不是好设计！怎么办？改进Page Table的数据结构，引入多级页表（Multi-level Page Tables）

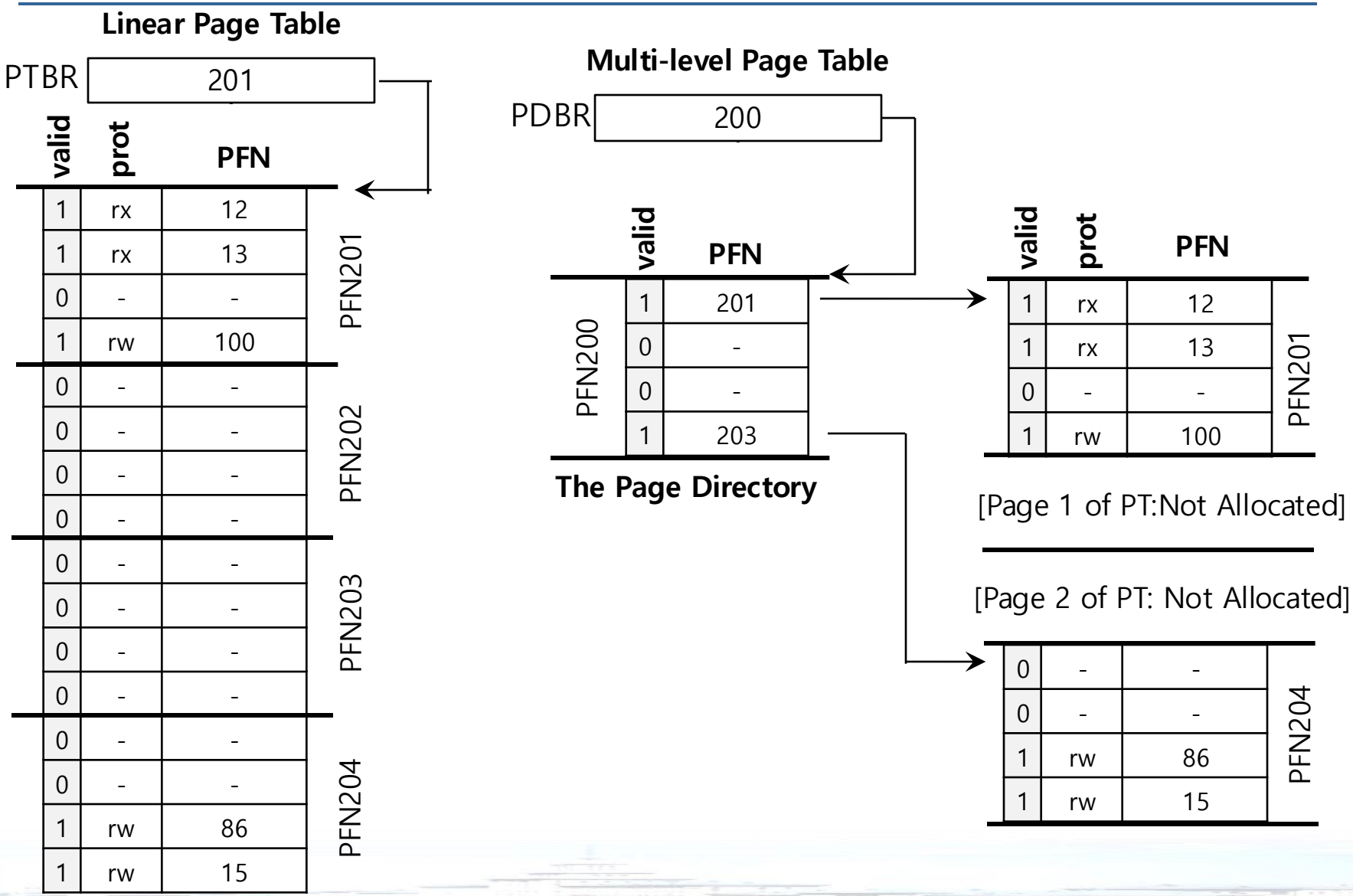
Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 多级页表
   a) 线性页表（基础）
   b) 段页式方法（基础）
   c) 多级页表（基础）
9. 交换（Swapping）

Multi-level Page Tables

- 把linear page table换成另一种数据结构, like a tree.
  - Chop up（分割）the page table into page-sized units.
  - If an entire page of page-table entries is invalid, don't allocate that page of the page table at all.
  - To track whether a page of the page table is valid, use a new structure, called page directory.

Example: Page directory



Linear (Left) And Multi-Level (Right) Page Tables

Example: analysis

- Linear Page Table
  - even though most of the middle regions of the address space are not valid, we still require page-table space allocated for those regions (i.e., the middle two pages of the page table).
- Multi-level Page Table
  - The page directory marks just two pages of the page table as valid (the first and last); thus, just those two pages of the page table reside in memory.
  - one way to visualize what a multi-level table is doing: it just makes parts of the linear page table disappear (freeing those frames for other uses), and **tracks which pages of the page table are allocated with the page directory**.（**它只是使**线性页表的部分消失（释放这些帧**以供其他用途**），**并跟踪**页表的哪些页分配有页目录）

Multi-level Page Tables: Page directory entries

- The page directory contains one entry per page of the page table.
  - It consists of a number of page directory entries(PDE).
- PDE has a valid bit and page frame number(PFN).

Multi-level Page Tables: Pros. & Cons.

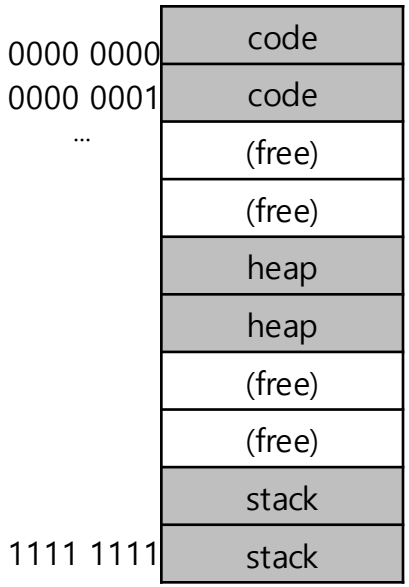
- Advantage
  - Only allocates page-table space in proportion to the amount of address space you are using.（仅根据正在使用的地址空间量按比例分配页表空间）
  - The OS can grab the next free page when it needs to allocate or grow a page table.（操作系统可以在需要分配或增长页表时抓取下一个空闲页）
- Disadvantage
  - Multi-level table is a small example of a time-space trade-off.
  - Complexity.

Multi-level Page Table: Level of indirection

- A multi-level structure can adjust level of indirection through use of the page directory.
  - Indirection place page-table pages wherever we would like in physical memory.

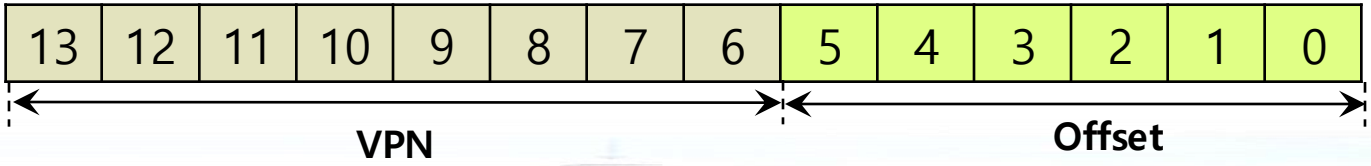
A Detailed Multi-Level Example

- To understand the idea behind multi-level page tables better, let's do an example.



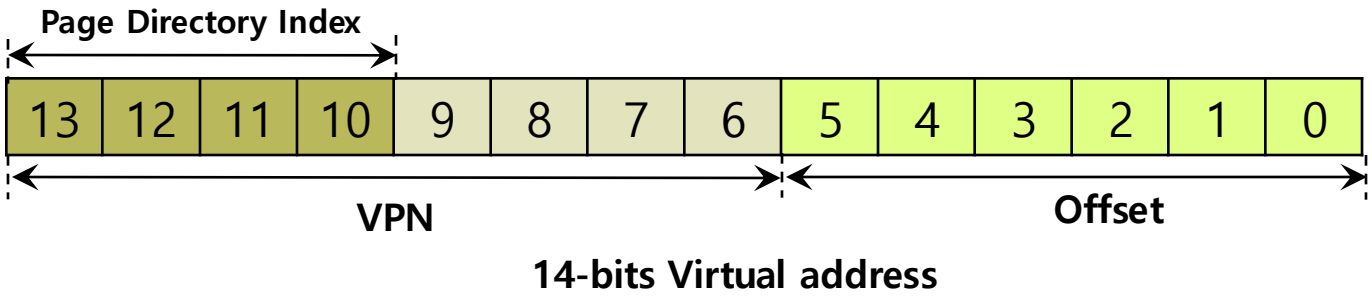
Flag	Detail
Address space	16 KB
Page size	64 byte
Virtual address	14 bit
VPN	8 bit
Offset	6 bit
Page table entry	2^8(256)

A 16-KB Address Space With 64-byte Pages



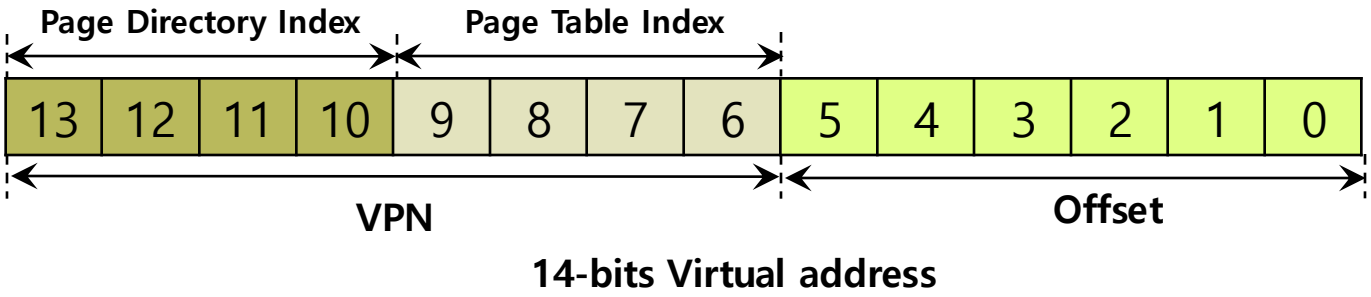
Page Directory Idx

- The page directory needs one entry per page of the page table
  - it has 16 entries.
- The page-directory entry is invalid → Raise an exception (The access is invalid)



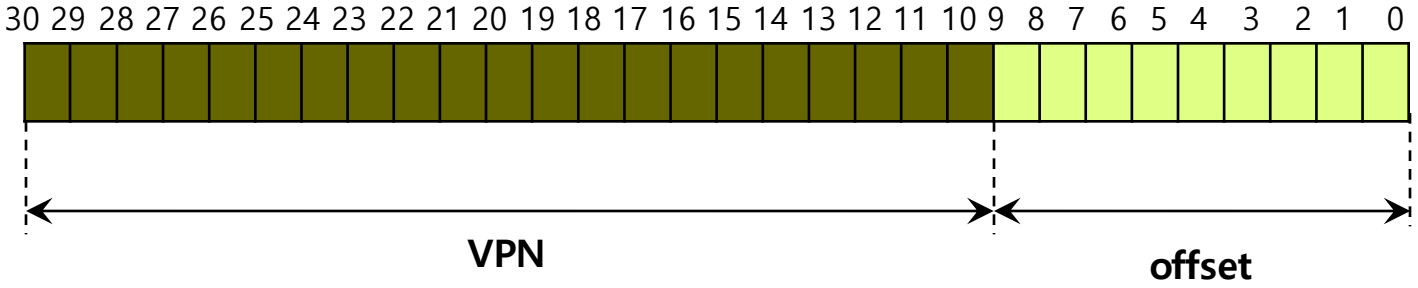
Page Table Idx (cont.)

- The PDE is valid, we have more work to do.
  - To fetch the page table entry(PTE) from the page of the page table pointed to by this page-directory entry.
- This page-table index can then be used to index into the page table itself.



More than Two Level

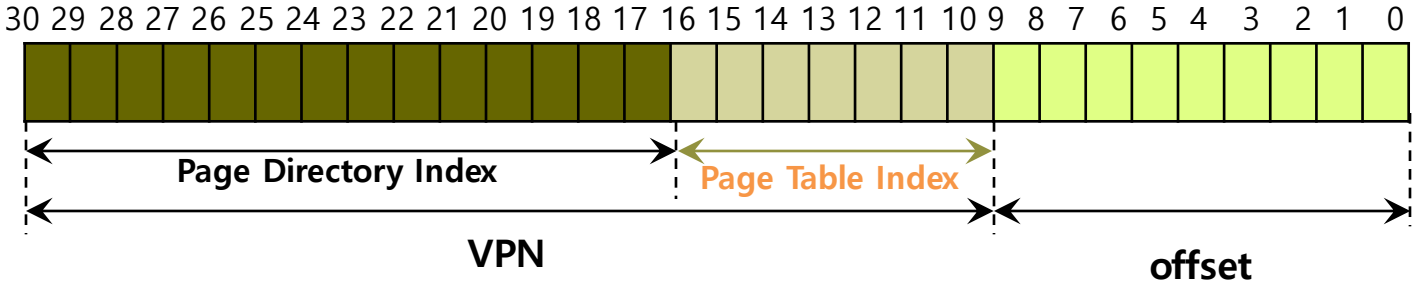
- In some cases, a deeper tree is possible.



Flag	Detail
Virtual address	30 bit
Page size	512 byte
VPN	21 bit
Offset	9 bit

More than Two Level : Page Table Index

- In some cases, a deeper tree is possible.

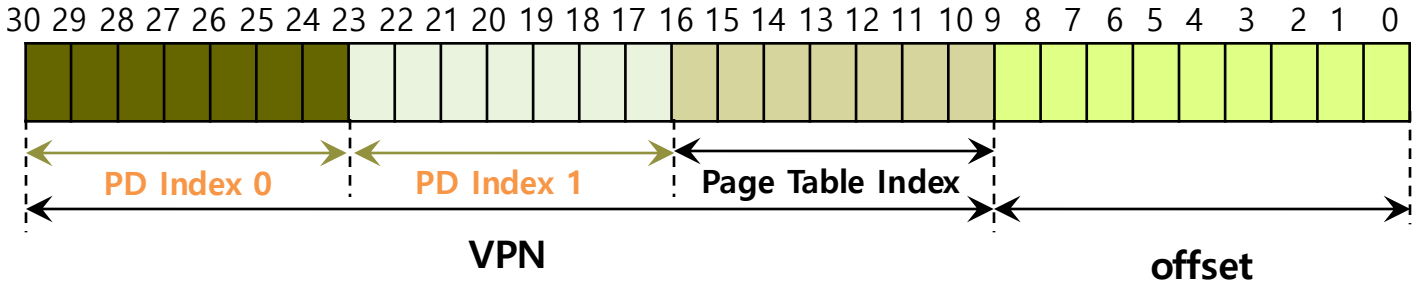


Flag	Detail
Virtual address	30 bit
Page size	512 byte
VPN	21 bit
Offset	9 bit
Page entry per page	128 PTEs

$\text{Log}_2\ 128 = 7$

More than Two Level : Page Directory

- If our page directory has 2^{14} entries, it spans not one page but 128.
- To remedy this problem, we build a further level of the tree, by splitting the page directory itself into multiple pages of the page directory.



Multi-level Page Table Control Flow

```
01:      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
02:      (Success,TlbEntry) = TLB_Lookup(VPN)
03:      if(Success == True)        //TLB Hit
04:        if(CanAccess(TlbEntry.ProtectBits) == True)
05:              Offset = VirtualAddress & OFFSET_MASK
06:              PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
07:              Register = AccessMemory(PhysAddr)
08:        else RaiseException(PROTECTION_FAULT);
09:      else // perform the full multi-level lookup
```

- (1 lines) extract the virtual page number(VPN)
- (2 lines) check if the TLB holds the transalation for this VPN
- (5-8 lines) extract the page frame number from the relevant TLB entry, and form the desired physical address and access memory

Multi-level Page Table Control Flow

```
11:       else
12:                 PDIndex = (VPN & PD_MASK) >> PD_SHIFT
13:                 PDEAddr = PDBR + (PDIndex * sizeof(PDE))
14:                 PDE = AccessMemory(PDEAddr)
15:                 if(PDE.Valid == False)
16:                           RaiseException(SEGMENTATION_FAULT)
17:                 else // PDE is Valid: now fetch PTE from PT
```

- (11 lines) extract the Page Directory Index(PDIndex)
- (13 lines) get Page Directory Entry(PDE)
- (15-17 lines) Check PDE valid flag. If valid flag is true, fetch Page Table entry from Page Table

The Translation Process: Remember the TLB

```
18:     PTIndex = (VPN & PT_MASK) >> PT_SHIFT
19:     PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
20:     PTE = AccessMemory(PTEAddr)
21:     if(PTE.Valid == False)
22:             RaiseException(SEGMENTATION_FAULT)
23:     else if(CanAccess(PTE.ProtectBits) == False)
24:             RaiseException(PROTECTION_FAULT);
25:     else
26:             TLB_Insert(VPN, PTE.PFN , PTE.ProtectBits)
27:             RetryInstruction()
```

Inverted Page Tables

- A single page table that has an entry for each physical page of the system.
  以物理页号作为索引，形成一张倒排表，作为页表
- The entry tells:
  - which process is using this page
  - Which virtual page of that process maps to this physical page
- Finding the correct entry is now a matter of searching through this data structure
  - hash table is often built over the base structure to speed up lookups
  - PowerPC 是典型例子

例题

某计算机系统按字节编址，采用二级页表的分页存储管理方式，虚拟地址格式如下所示：

10位	10位	12位
页目录号	页表索引	页内偏移量

请回答下列问题。

(1)页和页框的大小各为多少字节?进程的虚拟地址空间大小为多少页?

(2)假定页目录项和页表项均占4个字节，则进程的页目录和页表共占多少页?

(3)若某指令周期内访问的虚拟地址为0100 0000H和0111 2048H，则进行地址转换时共访问多少个二级页表?要求说明理由。

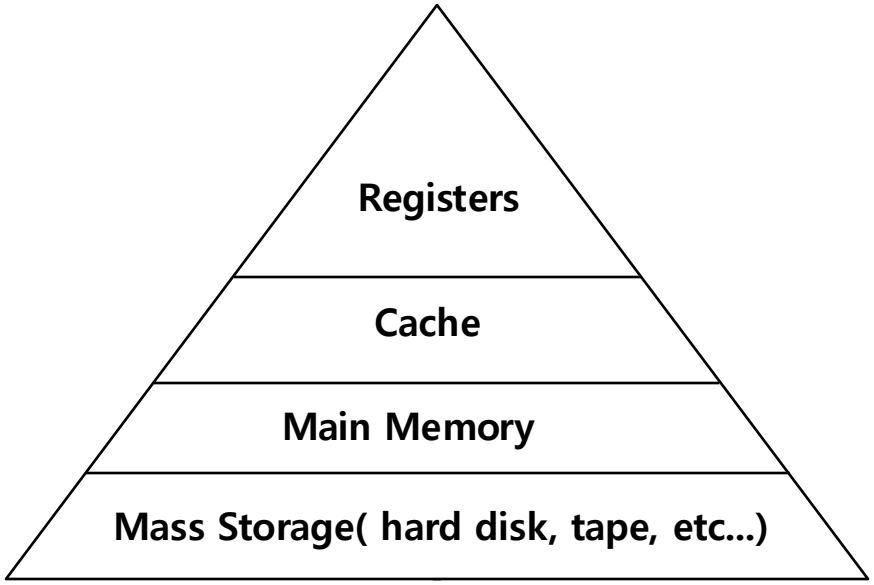
答案要点:(1)页和页框大小均为4 KB。进程的虚拟地址空间大小为$2^{32}/2^{12}=2^{20}$页。
(2)$(2^{10}\times4)/2^{12}$(页目录所占页数)+ $(2^{10}\times4)/2^{12}$ (页表所占页数)$\times 2^{10}=1025$页。
(3)需要访问一个二级页表。因为虚拟地址0100 0000H和0111 2048H的最高10位的值都是4，访问的是同一个二级页表。

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 多级页表
9. 交换（Swapping）
   a) 基础概念（基础）
   b) 页错误（基础）
   c) 交换策略： OPT、FIFO、Random、LRU、Clock （基础）

Beyond Physical Memory: Mechanisms

- Require an additional level in the memory hierarchy（内存层次结构）.
  - 实际中，OS需要支持同时运行的巨大地址空间，因此OS需要把当前地址空间没有使用的部分找个地方贮存起来
  - In modern systems, this role is usually served by a hard disk drive



Memory Hierarchy in modern system

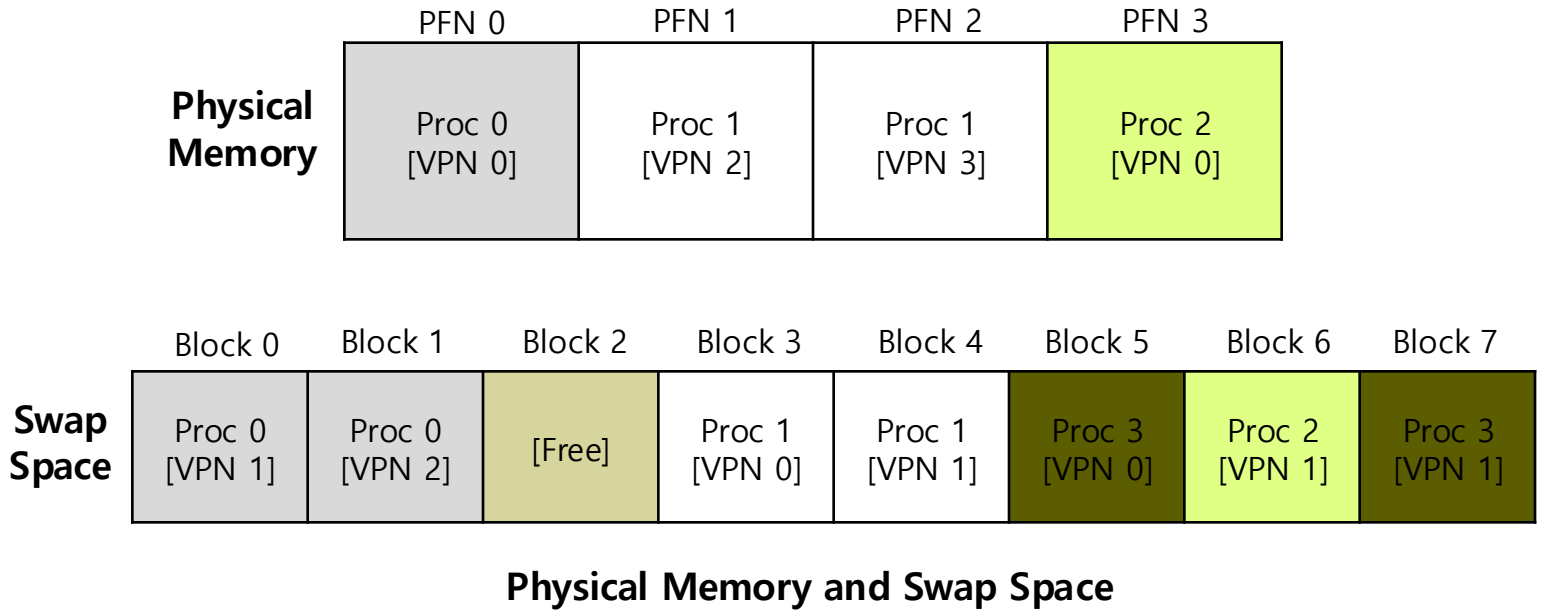
Requirement: Single large address for a process

- 到目前为止，假设虚拟地址空间足够小，能挣个放入物理内存。但事实上，我们需要支持许多同时运行的巨大地址空间
- Always need to first arrange for the code or data to be in memory when before calling a function or accessing data.

- To Beyond just a single process.
  - The addition of swap space allows the OS to support the illusion of a large virtual memory for multiple concurrently-running（并发运行） process.

Swap Space

- Reserve some space on the disk for moving pages back and forth.
- OS need to remember to the swap space, in page-sized unit



Physical Memory and Swap Space

Present Bit

- Add some machinery higher up in the system in order to support swapping pages to and from the disk.
  - When the hardware looks in the PTE, it may find that the page is not present in physical memory.

Value	Meaning
1	page is present in physical memory
0	The page is not in memory but rather on disk.

What If Memory Is Full ?

- OS like to page out pages to make room for the new pages the OS is about to bring in.
  - The process of picking a page to kick out, or replace is known as page-replacement policy

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 多级页表
9. 交换（Swapping）
   a) 基础概念（基础）
   b) 页错误（基础）
   c) 交换策略： OPT、FIFO、Random、LRU、Clock （基础）

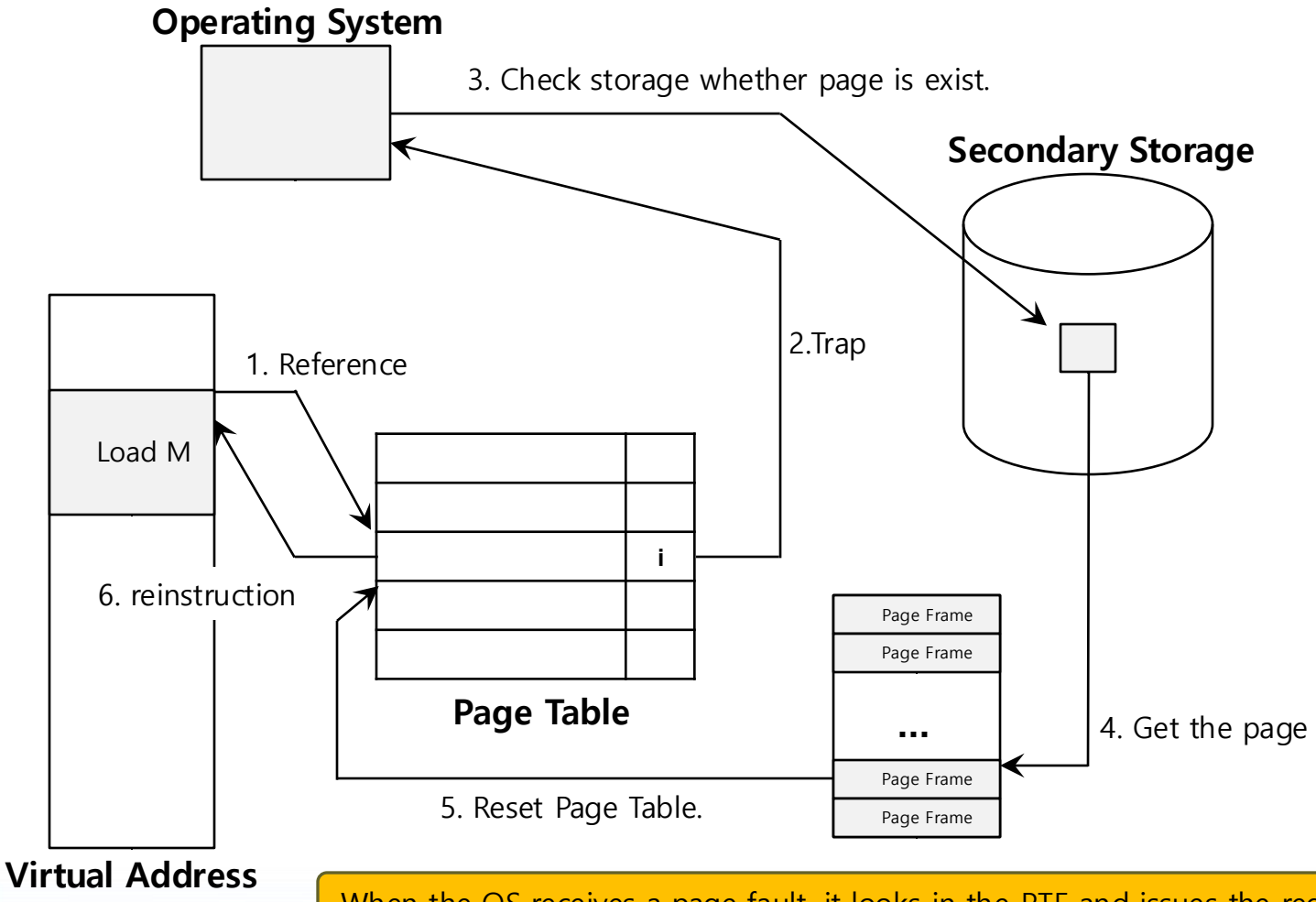
The Page Fault

- Accessing page that is not in physical memory.
  - If a page is not present and has been swapped out to disk, the OS need to swap the page into memory in order to service the page fault.

(如果页面不存在并且已被换出到磁盘，则操作系统需要将该页面交换到内存中以处理页面错误)

Page Fault Control Flow

- PTE used for data such as the PFN of the page for a disk address.



When the OS receives a page fault, it looks in the PTE and issues the request to disk.

Page Fault Control Flow – Hardware

```
1:      VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2:      (Success, TlbEntry) = TLB_Lookup(VPN)
3:      if (Success == True) // TLB Hit
4:              if (CanAccess(TlbEntry.ProtectBits) == True)
5:                      Offset = VirtualAddress & OFFSET_MASK
6:                      PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7:                      Register = AccessMemory(PhysAddr)
8:              else RaiseException(PROTECTION_FAULT)
```

Page Fault Control Flow – Hardware

```
9:      else // TLB Miss
10:             PTEAddr = PTBR + (VPN * sizeof(PTE))
11:             PTE = AccessMemory(PTEAddr)
12:             if (PTE.Valid == False)
13:                     RaiseException(SEGMENTATION_FAULT)
14:             else
15:     if (CanAccess(PTE.ProtectBits) == False)
16:             RaiseException(PROTECTION_FAULT)
17:     else if (PTE.Present == True)
18:     // assuming hardware-managed TLB
19:             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
20:             RetryInstruction()
21:     else if (PTE.Present == False)
22:             RaiseException(PAGE_FAULT)
```

Page Fault Control Flow – Software

```
1:        PFN = FindFreePhysicalPage()
2:        if (PFN == -1) // no free page found
3:                PFN = EvictPage() // run replacement algorithm
4:                DiskRead(PTE.DiskAddr, pfn) // sleep (waiting for I/O)
5:                PTE.present = True // update page table with present
6:                PTE.PFN = PFN // bit and translation (PFN)
7:                RetryInstruction() // retry instruction
```

- The OS must find a physical frame for the soon-be-faulted-in page to reside within.
- If there is no such page, waiting for the replacement algorithm to run and kick some pages out of memory.

Module 4: Virtualizing Memory 虚拟内存

1. 虚拟地址空间
2. 内存管理API（Memory API ）
3. Mechanism: 地址翻译（Address Translation）
4. 分段（ Segmentation ）
5. 空闲空间管理（ Free-Space Management ）
6. 分页（Paging）
7. 地址翻译缓存 TLB（Translation Lookaside Buffers）
8. 多级页表
9. 交换（Swapping）
   a) 基础概念（基础）
   b) 页错误 Page Fault（基础）
   c) 交换策略：OPT、FIFO、Random、LRU、Clock （基础）

When Replacements Really Occur

- OS waits until memory is entirely full, and only then replaces a page to make room for some other page 理想情况下，只当内存完全使用，此时若发生超额内存请求，才需要进行内存替换
  - This is a little bit unrealistic, and there are many reason for the OS to keep a small portion of memory free more proactively (主动). 但实际上，OS始终需要保留一部分空闲内存，以便掌握管理的主动权

- 后台任务： Swap Daemon, Page Daemon
  - There are fewer than LW pages available, a background thread that is responsible for freeing memory runs.
  - The thread evicts pages until there are HW pages available.

Beyond Physical Memory: Policies

- Memory pressure forces the OS to start paging out pages to make room for actively-used pages.
- Deciding which page to evict is encapsulated within the replacement policy of the OS. （决定驱逐哪个页面被封装在操作系统的替换策略中）

Cache Management

- Goal in picking a replacement policy for this cache is to minimize the number of cache misses. （为该缓存选择替换策略的目标是最小化缓存未命中的数量）
- The number of cache hits and misses let us calculate the *average memory access time(AMAT)* （平均内存访问时间）.

$$AMAT = (P_{hit} * T_M) + (P_{miss} * T_D)$$

Arguement	Meaning
T_M	The cost of accessing memory
T_D	The cost of accessing disk
P_{Hit}	The probability of finding the data item in the cache(a hit)
P_{Miss}	The probability of not finding the data in the cache(a miss)

假设内存访问的成本约100ns，磁盘访问的成本约10ms
若P_{hit} = 90%，则AMAT是？ 若P_{hit} = 99.9%呢？ 若P_{hit} 接近 100%呢？

The Optimal Replacement Policy

- Leads to the fewest number of misses overall
  - 选择要替换的页： the page that will be accessed furthest in the future
  - Resulting in the fewest-possible cache misses
- Serve only as a comparison point （比较点）, to know how close we are to perfect

Tracing the Optimal Policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	2	0,1,3
0	Hit		0,1,3
3	Hit		0,1,3
1	Hit		0,1,3
2	Miss	3	0,1,2
1	Hit		0,1,2

Future is not known.

A Simple Policy: FIFO

- Pages were placed in a queue when they enter the system.
- When a replacement occurs, the page on the tail of the queue(the "**First-in**" pages) is evicted.
  - It is simple to implement, but can't determine the importance of blocks.

Tracing the FIFIO Policy

Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss		3,0,1
2	Miss	3	0,1,2
1	Hit		0,1,2

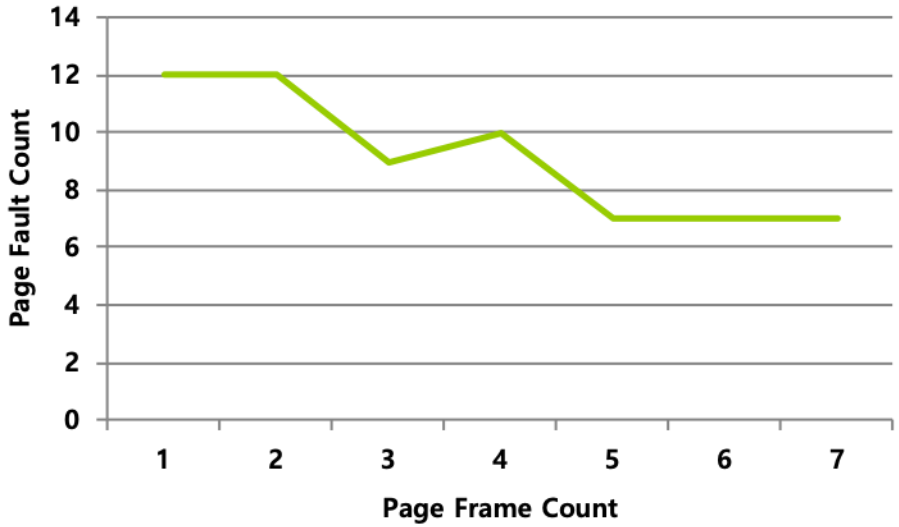
Even though page 0 had been accessed a number of times, FIFO still kicks it out.

BELADY'S ANOMALY

- We would expect the cache hit rate to increase when the cache gets larger. But in this case, with FIFO, it gets worse.

Reference Row

1 2 3 4 1 2 5 1 2 3 4 5



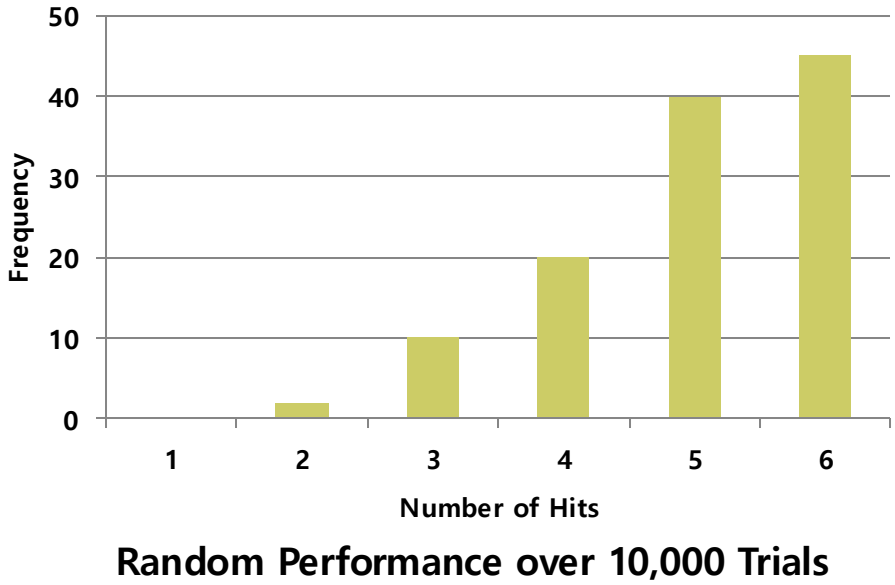
Another Simple Policy: Random

- Picks a random page to replace under memory pressure.
  - It doesn't really try to be too intelligent in picking which blocks to evict.
  - Random does depends entirely upon how lucky Random gets in its choice.

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		0,1,2
1	Hit		0,1,2
3	Miss	0	1,2,3
0	Miss	1	2,3,0
3	Hit		2,3,0
1	Miss	3	2,0,1
2	Hit		2,0,1
1	Hit		2,0,1

Random Performance

- Sometimes, Random is as good as optimal（随机与最优一样好）, achieving 6 hits on the example trace.



Random Performance over 10,000 Trials

Using History

- Lean on the past and use **history**.
  - Two type of historical information.

Historical Information	Meaning	Algorithms
recency	The more recently a page has been accessed, the more likely it will be accessed again	LRU
frequency	If a page has been accessed many times, It should not be replcaed as it clearly has some value	LFU

Using History : LRU

- Replaces the least-recently-used page.

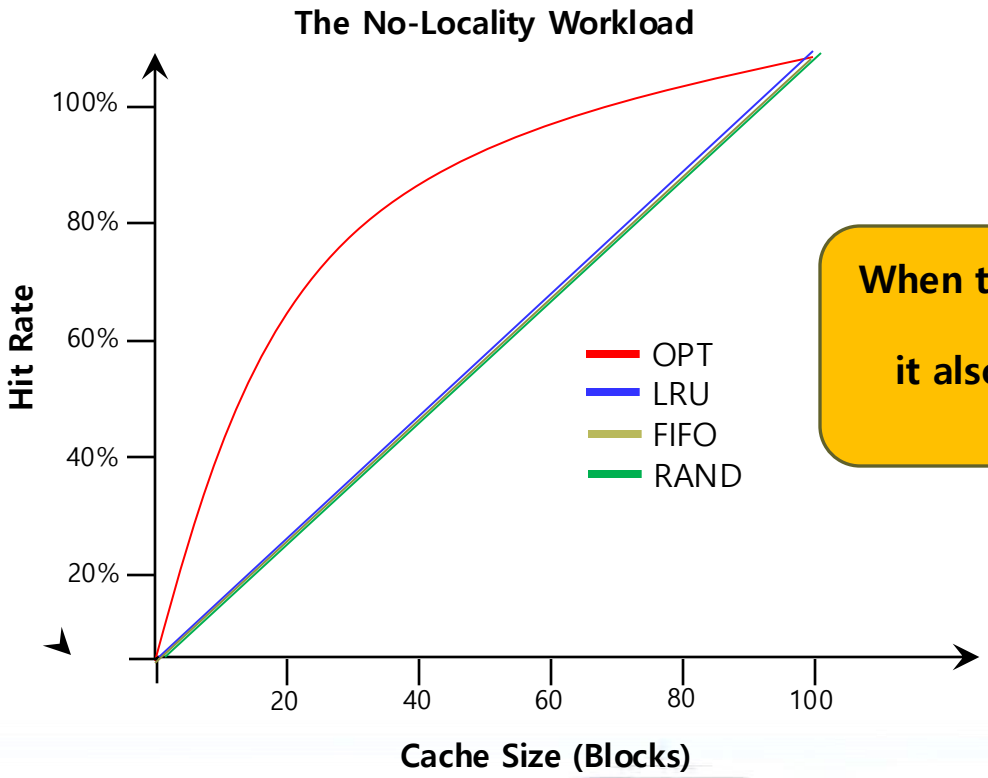
Reference Row

0 1 2 0 1 3 0 3 1 2 1

Access	Hit/Miss?	Evict	Resulting Cache State
0	Miss		0
1	Miss		0,1
2	Miss		0,1,2
0	Hit		1,2,0
1	Hit		2,0,1
3	Miss	2	0,1,3
0	Hit		1,3,0
3	Hit		1,0,3
1	Hit		0,3,1
2	Miss	0	3,1,2
1	Hit		3,2,1

Workload Example : The No-Locality Workload

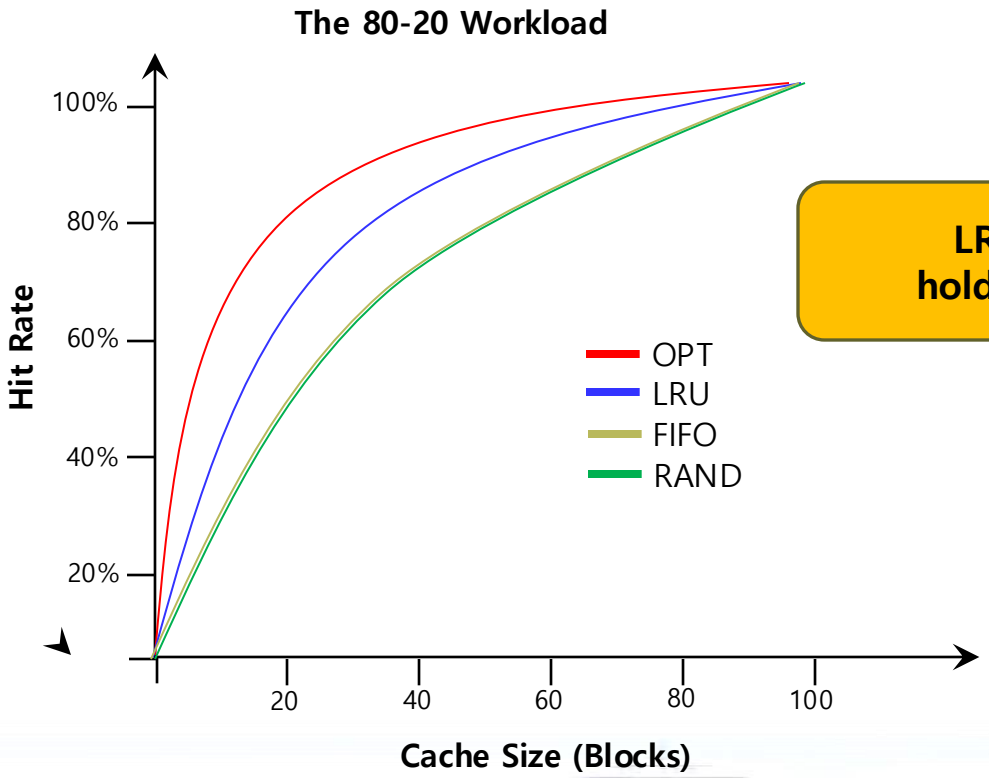
- Each reference is to a random page within the set of accessed pages.
  - Workload accesses 100 unique pages over time.
  - Choosing the next page to refer to at random



When the cache is large enough to fit the entire workload, it also doesn't matter which policy you use.

Workload Example : The 80-20 Workload

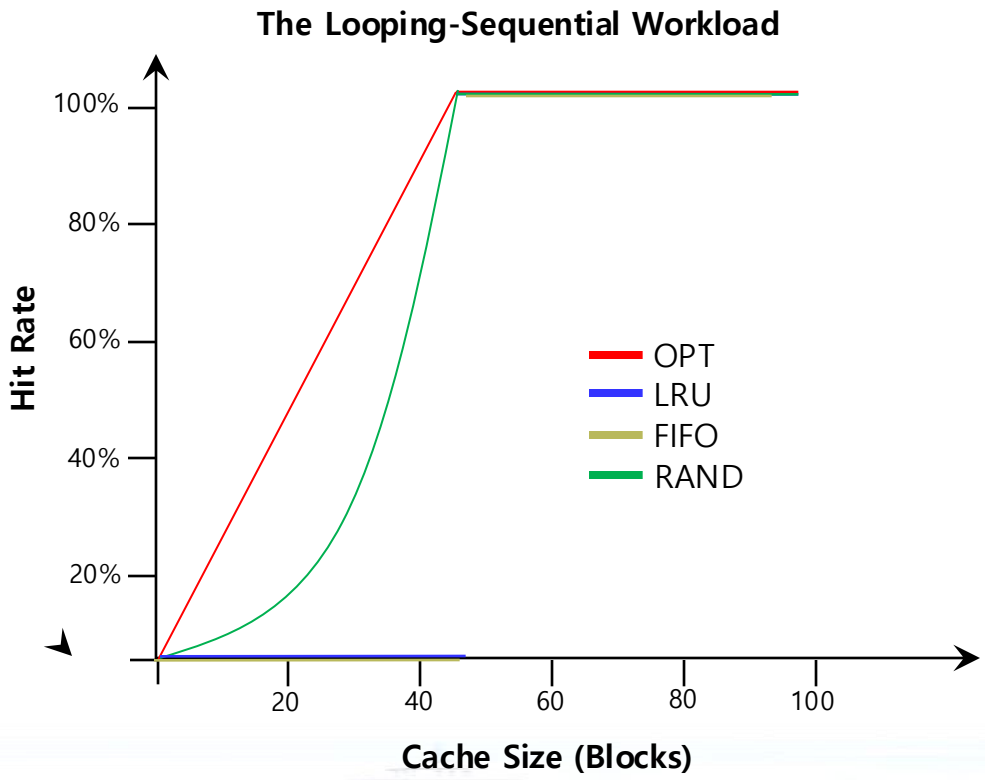
- Exhibits locality: 80% of the reference are made to 20% of the page
- The remaining 20% of the reference are made to the remaining 80% of the pages.



LRU is more likely to hold onto the hot pages.

Workload Example : The Looping Sequential

- Refer to 50 pages in sequence.
  - Starting at 0, then 1, … up to page 49, and then we Loop, repeating those accesses, for total of 10,000 accesses to 50 unique pages.



Implementing Historical Algorithms

- To keep track of which pages have been least-and-recently used, the system has to do some accounting work on **every memory reference (每个内存引用)**.
  - Add a little bit of hardware support.

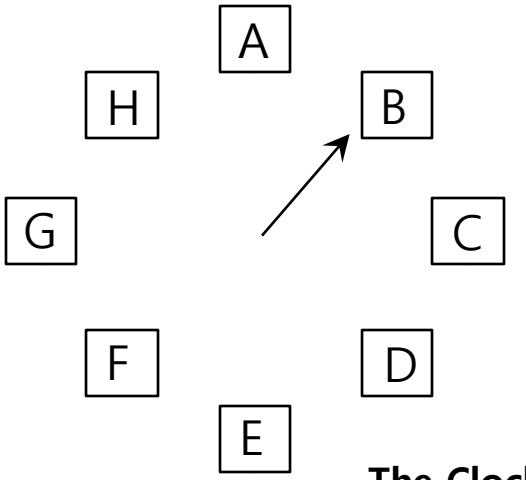
Approximating LRU

- Require some hardware support, in the form of a **use bit**
  - Whenever a page is referenced, the use bit is set by hardware to 1.
  - Hardware never clears the bit, though; that is the responsibility of the OS

- Clock Algorithm
  - All pages of the system arranges in a circular list（循环列表）.
  - A clock hand points to some particular page to begin with.

Clock Algorithm

- The algorithm continues until it finds a use bit that is set to 0.



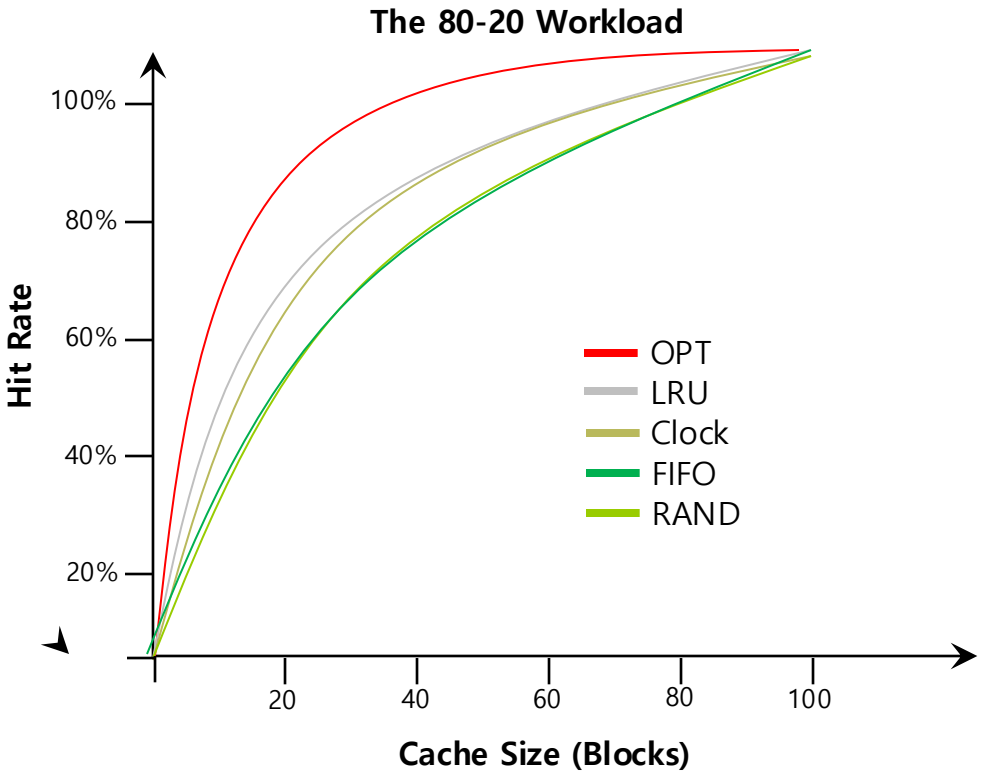
Use bit	Meaning
0	Evict the page
1	Clear **Use bit** and advance hand

The Clock page replacement algorithm

When a page fault occurs, the page the hand is pointing to is inspected.
The action taken depends on the Use bit

Workload with Clock Algorithm

- Clock algorithm doesn't do as well as perfect LRU, it does better then approach that don't consider history at all.



Considering Dirty Pages

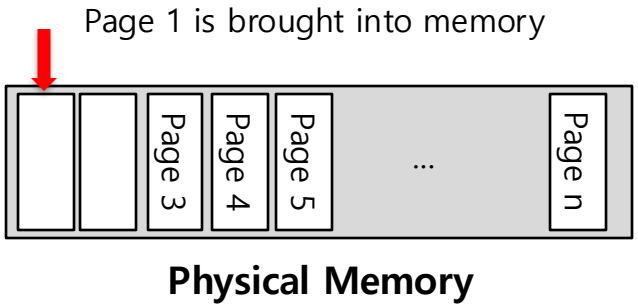
- The hardware include a **modified bit** (a.k.a **dirty bit**)
  - Page has been **modified** and is thus **dirty**, it must be written back to disk to evict it. 在踢出脏页的时候，需要首先把该页写到硬盘，代价很大，故VMM倾向于evict干净页
  - Page has not been modified, the eviction is free.

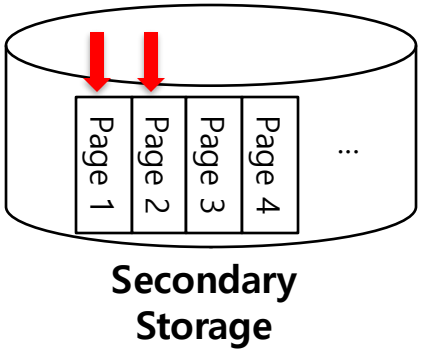
Page Selection Policy

- The OS has to decide when to bring a page into memory.
- Presents the OS with some different options.

Prefetching

- The OS guess that a page is about to be used, and thus bring it in ahead of time.

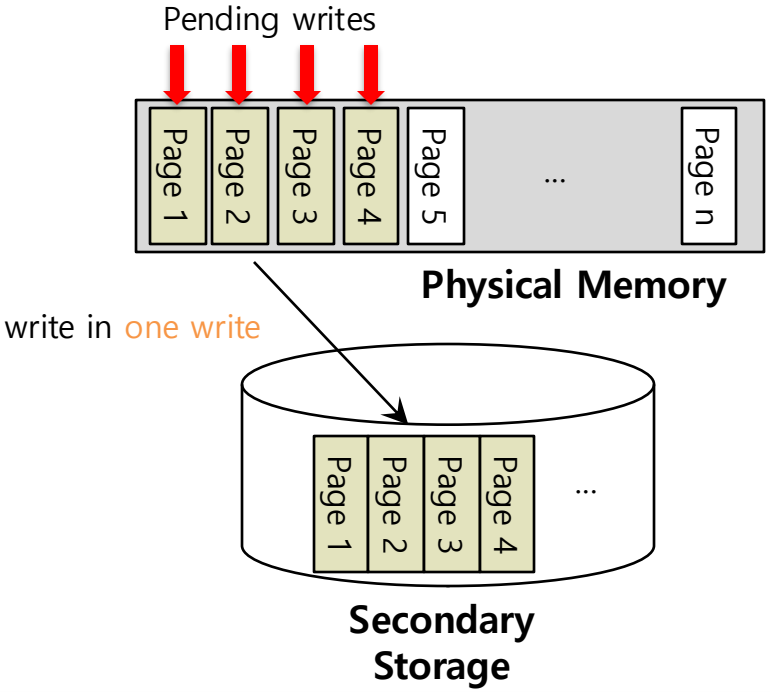




Page 2 likely soon be accessed and
thus should be brought into memory too

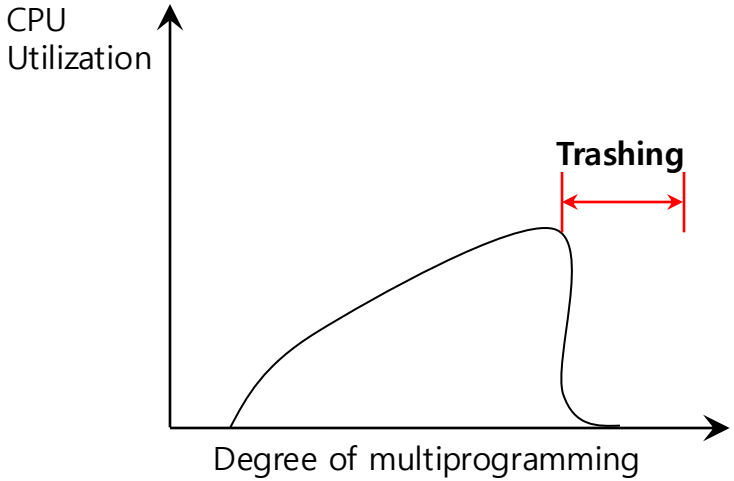
Clustering, Grouping

- Collect a number of pending writes （挂起的写入） together in memory and write them to disk in one write.
  - Perform a **single large write** more efficiently than **many small ones**.



Thrashing

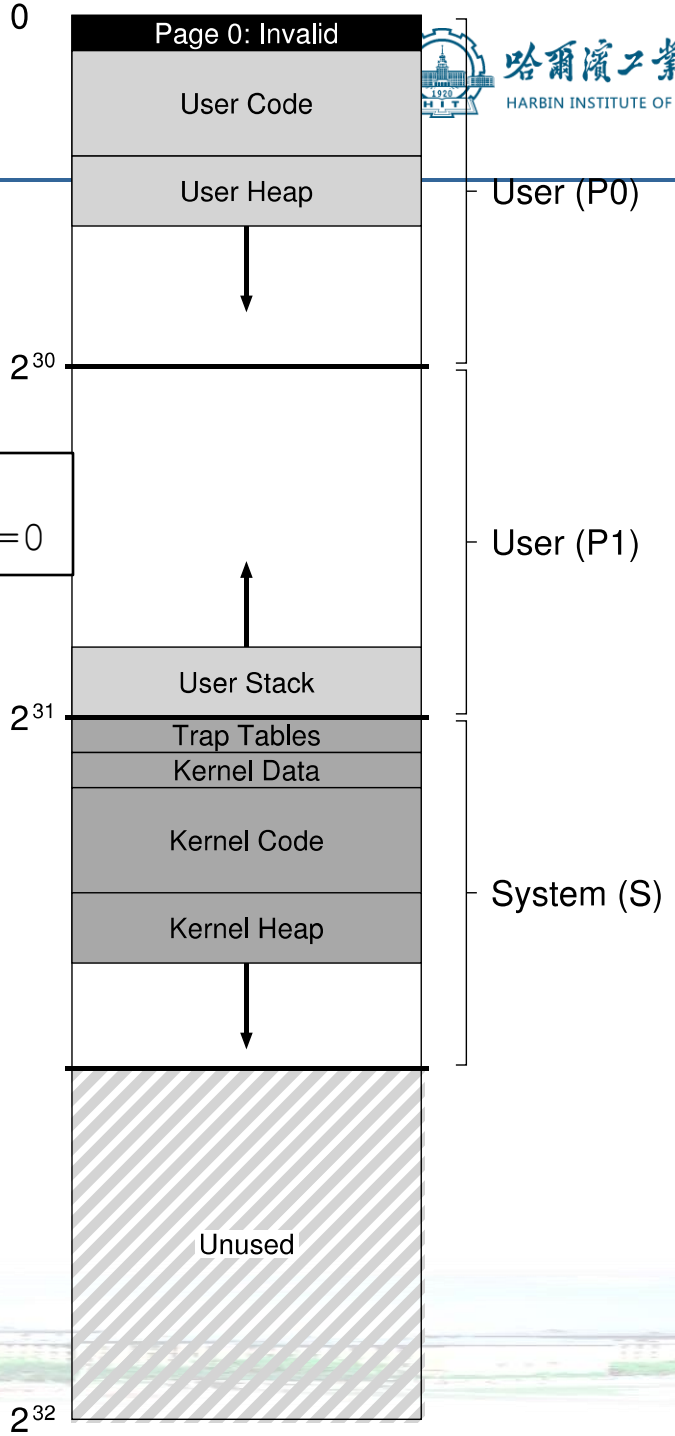
- Memory is oversubscribed （超额使用） and the memory demands of the set of running processes exceeds the available physical memory.
  - Decide not to run a subset of processes.
  - Reduced set of processes working sets fit in memory.



A real world 地址空间

```
int *p = NULL; //set p = 0
*p = 10;//try to store value 10 to VA=0
```

- 内核是每个进程地址空间的一部分
  - 硬件保护.
- 页替换
  - 分段的FIFO
  - 页聚集



例题

在缺页处理过程中，操作系统执行的操作可能是( )。

a. 修改页表　　b. 磁盘I/O　　c. 分配页框

A. 仅a, b　　B. 仅b　　C. 仅c　　D. a, b, c

答案：D

解析：

修改页表：将缺页的虚拟页与物理页框进行映射。

磁盘I/O：从磁盘读取缺失的页到内存。

分配页框：为缺页分配一个空闲的物理页框。

例题

有一个虚拟存储系统，若进程在内存中占3页（开始时内存为空），若采用先进先出（FIFO）页面淘汰算法，当执行如下访问页号序列后1，2，3，4，5, 1，2，5，1，2，3，4，5，会发生多少缺页?

A. 7

B. 8

C. 9

D. 10

答案：D

解析：

访问页号	1	2	3	4	5	1	2	5	1	2	3	4	5
物理块1	1	1	1	4	4	4	2	2	2	2	2	2	5
物理块2		2	2	2	5	5	5	5	5	5	3	3	3
物理块3			3	3	3	1	1	1	1	1	1	4	4
是否缺页	是	是	是	是	是	是	是	否	否	否	是	是	是

例题

某计算机按字节编址，其动态分区内存管理采用最佳适应算法，每次分配和回收内存后都对空闲分区链重新排序（按分区大小由小到大进行排序）。当前空闲分区信息如下表所示。回收起始地址为60K、大小为140KB的分区后，系统中空闲分区的数量、空闲分区链第一个分区的起始地址和大小分别是（ ）。

分区起始地址	20 K	500 K	1000 K	200 K
分区大小	40 KB	80 KB	100 KB	200 KB

A．3、20K、380KB　　B．3、500K、80KB

C．4、20K、180KB　　D．4、500K、80KB

答案：B

解析：回收起始地址为60K、大小为140KB的分区后，该分区会与表中第一个分区和第四个分区合并成一个分区，其起始地址为20K、大小为380KB，此时只有三个空闲分区，其他两个空闲分区起始地址为500K，大小为80KB和起始地址为1000K，大小为100KB。在回收内存后，算法会对空闲分区链按分区大小由小到大进行排序，表中第二个分区排第一，即空闲分区链第一个分区起始地址为500K，大小为80KB。

例题

某系统采用改进型CLOCK置换算法，页表项中字段A为访问位，M为修改位。A=0表示页最近没有被访问，A=1表示页最近被访问过。M=0表示页没有被修改过，M=1表示页被修改过。按(A, M)所有可能的取值，将页分为四类：(0, 0)、(1, 0)、(0, 1)和(1, 1)，则该算法淘汰页的次序为（ ）。

A．(0, 0)，(0, 1)，(1, 0)，(1, 1)

B．(0, 0)，(1, 0)，(0, 1)，(1, 1)

C．(0, 0)，(0, 1)，(1, 1)，(1, 0)

D．(0, 0)，(1, 1)，(0, 1)，(1, 0)

答案：A

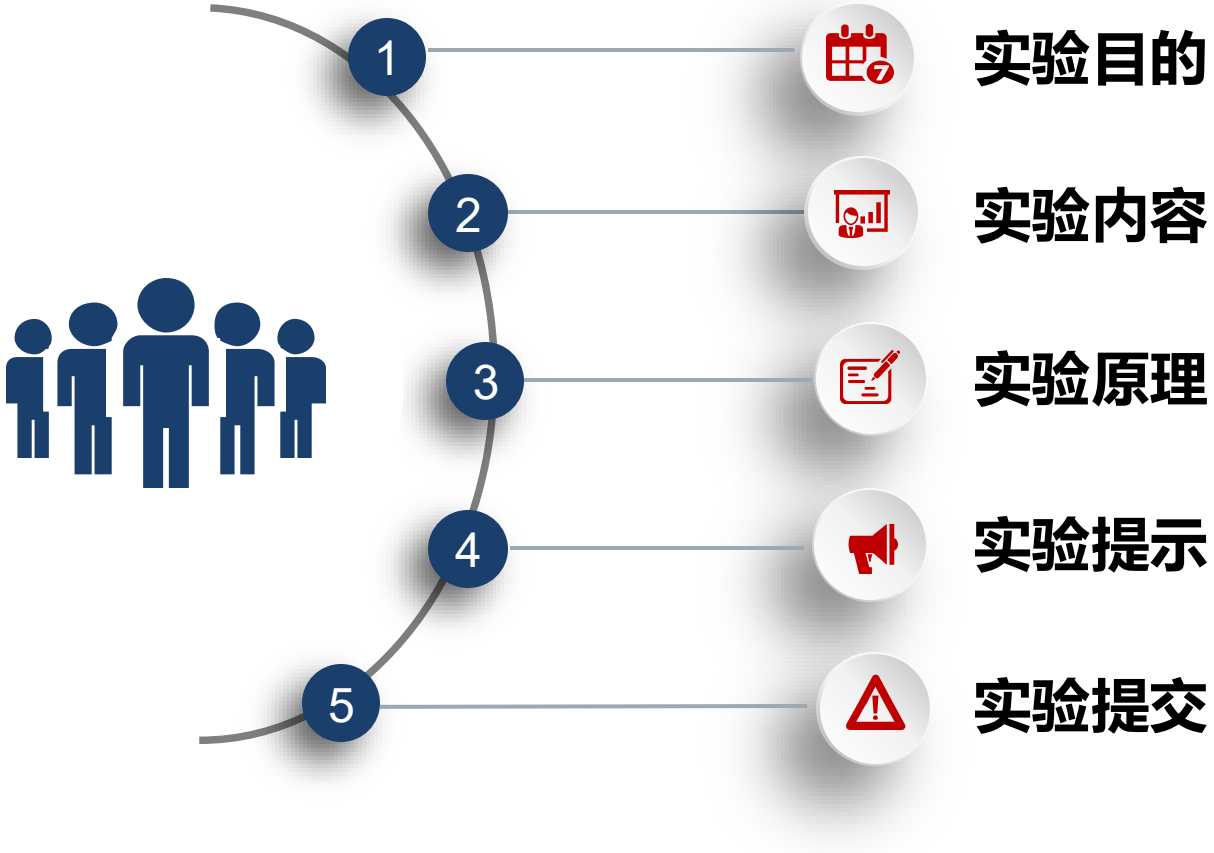
解析：改进型的CLOCK置换算法执行的步骤如下：

① 从指针的当前位置开始，扫描帧缓冲区。在这次扫描过程中，对使用位不做任何修改。选择遇到的第一个(0, 0)的帧用于替换。

② 若①失败则重新扫描，选择遇到的第一个(0, 1)的帧用于替换。在这个扫描过程中，对每个跳过的帧，将其访问位A设置为0。

③ 若②失败，则指针重新回到它最初的位置，此时集合中所有帧的访问位均为0。重复①，并在有必要时重复②，这样将可以找到供替换的帧。即该算法淘汰页的次序为：(0, 0)，(0, 1)，(1, 0)，(1, 1)。

实验三：锁机制的应用



实验目的

- 了解锁的原理，掌握锁竞争对程序并行性的影响；
- 理解xv6的内存分配器memory allocator以及磁盘缓存buffer cache的工作原理；
- 掌握Lock在xv6中内存分配器/磁盘缓存中的作用；
- 学习利用减少锁竞争优化xv6系统。

实验内容 | 2个实验任务

- 多核计算机上并行性差的一个常见症状是高锁争用。为了减少锁争用、提高并行性，通常需要同时改变数据结构和锁的策略，本实验要求重新设计xv6以下两个模块：

①重新设计内存分配器 Memory Allocator

②重新设计磁盘缓存 Buffer Cache

切换到 lock 分支，在该分支的代码上进行实验

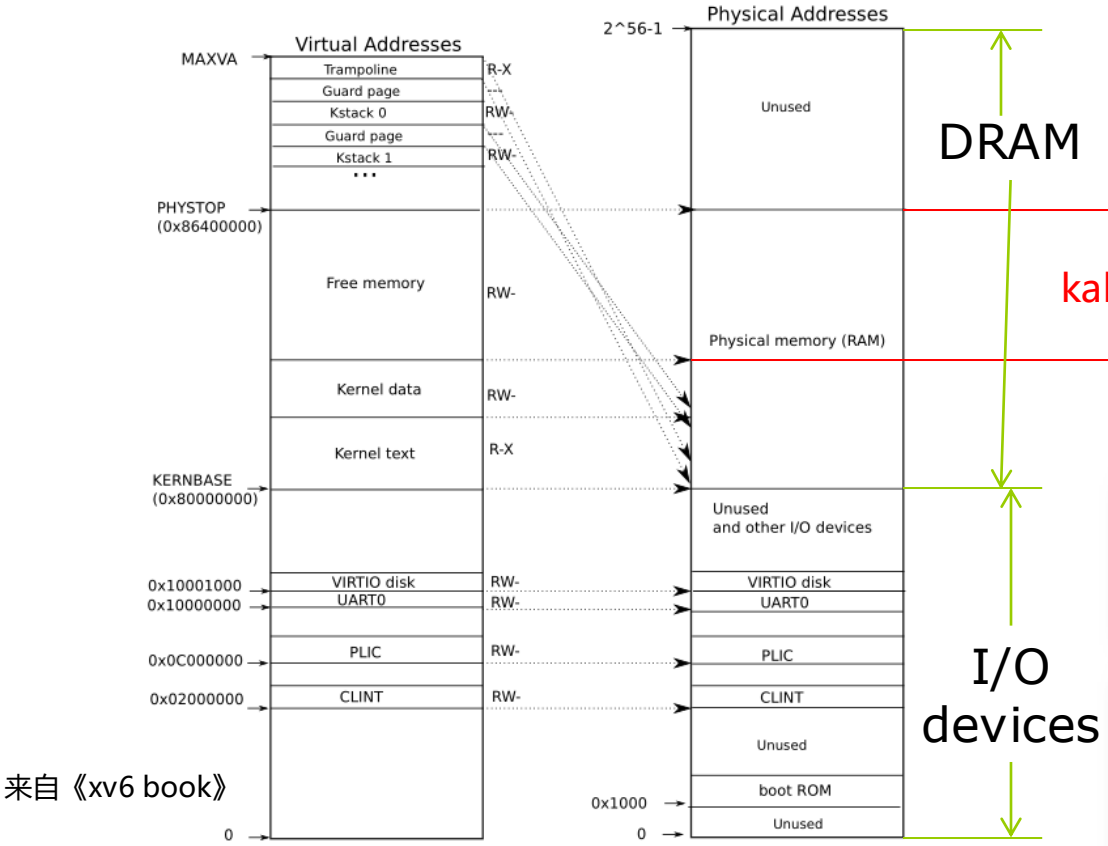
```
$ git checkout lock
$ git fetch
```

实验任务一

重新设计内存分配器（Memory Allocator）

- 修改空闲内存链表的管理机制，使每个CPU使用独立的链表；
- 支持在当前CPU的空闲链表（freelist）为空，其他链表不为空的情况下，可以从其他CPU的空闲链表中窃取（steal）空闲内存；
- 实验前后运行测评程序kalloctest、usertests，查看是否减少锁争用，并检查它是否仍然可用于分配所有内存。

实验原理 | Xv6内核地址空间



来自《xv6 book》

Figure 3.3: On the left, xv6's kernel address space. RWX refer to PTE read, write, and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.

PHYSTOP

kalloc可分配的物理内存

end

Kernel/memlayout.h

```
15  // the kernel uses physical memory thus:
16  // 80000000 -- entry.S, then kernel text and data
17  // end -- start of kernel page allocation area
18  // PHYSTOP -- end RAM used by the kernel
```

```
44  // the kernel expects there to be RAM
45  // for use by the kernel and user pages
46  // from physical address 0x80000000 to PHYSTOP.
47  #define KERNBASE 0x80000000L
48  #define PHYSTOP (KERNBASE + 128*1024*1024)
```

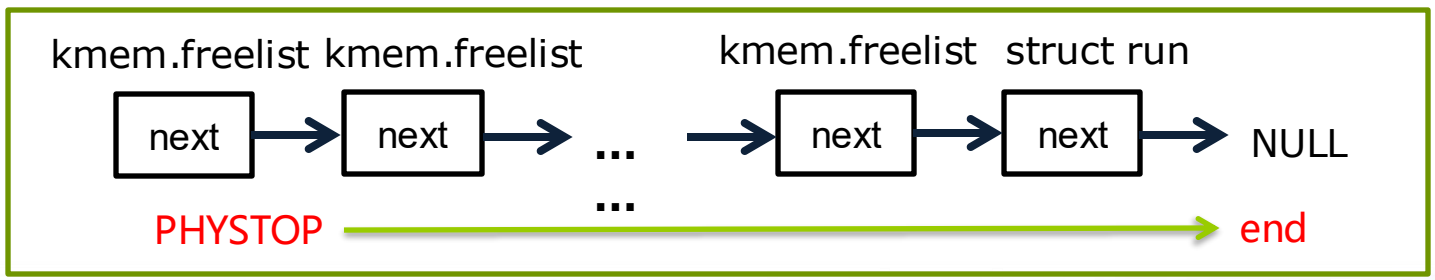
实验原理 | 内存分配器——初始化kinit()

- 通过保存所有空闲页（1页是4KB）来初始化链表。

kernel/kalloc.c

```
25
26 void
27 kinit()
28 {
29   initlock(&kmem.lock, "kmem");
30   freerange(end, (void*)PHYSTOP);
31 }
32
```

```
32
33 void
34 freerange(void *pa_start, void *pa_end)
35 {
36   char *p;
37   p = (char*)PGROUNDUP((uint64)pa_start);
38   for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE)
39     kfree(p);
40 }
41
```



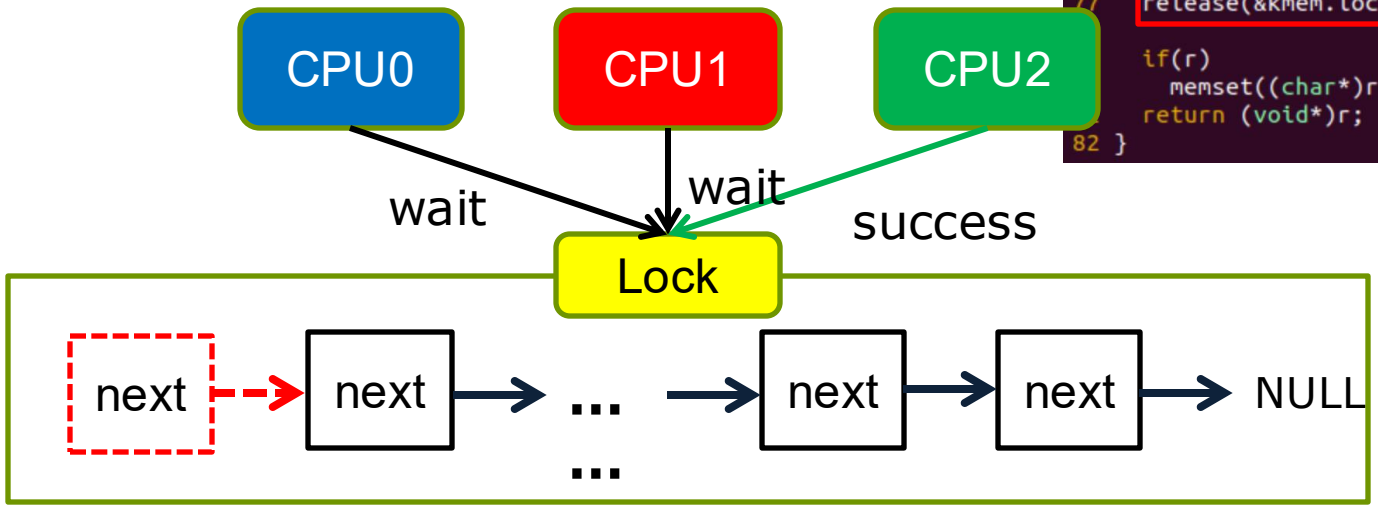
空闲链表

实验原理 | 内存分配器—申请内存 kalloc()

- Xv6支持运行在多核的系统上
- 例如，当CPU2调用kalloc时，获取到kmem.lock，并可以得到1页物理内存

```
68 void *
69 kalloc(void)
70 {
71   struct run *r;
72
73   acquire(&kmem.lock);
74   r = kmem.freelist;
75   if(r)
76     kmem.freelist = r->next;
77   release(&kmem.lock);

   if(r)
     memset((char*)r, 5, PGSIZE); // fill with junk
   return (void*)r;
82 }
```

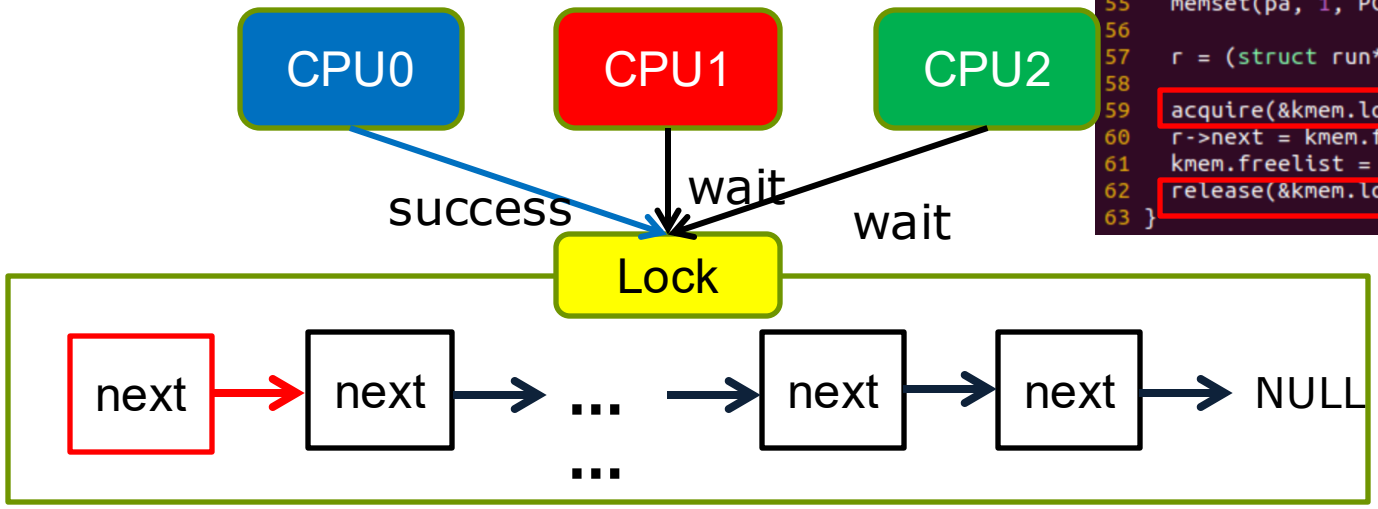


空闲链表

实验原理 | 内存分配器—释放内存 kfree()

- 例如，当CPU0调用kfree时，获取到kmem.lock，并将1页物理内存加到空闲链表头

```
46 void
47 kfree(void *pa)
48 {
49   struct run *r;
50
51   if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end
52     panic("kfree");
53
54   // Fill with junk to catch dangling refs.
55   memset(pa, 1, PGSIZE);
56
57   r = (struct run*)pa;
58
59   acquire(&kmem.lock);
60   r->next = kmem.freelist;
61   kmem.freelist = r;
62   release(&kmem.lock);
63 }
```

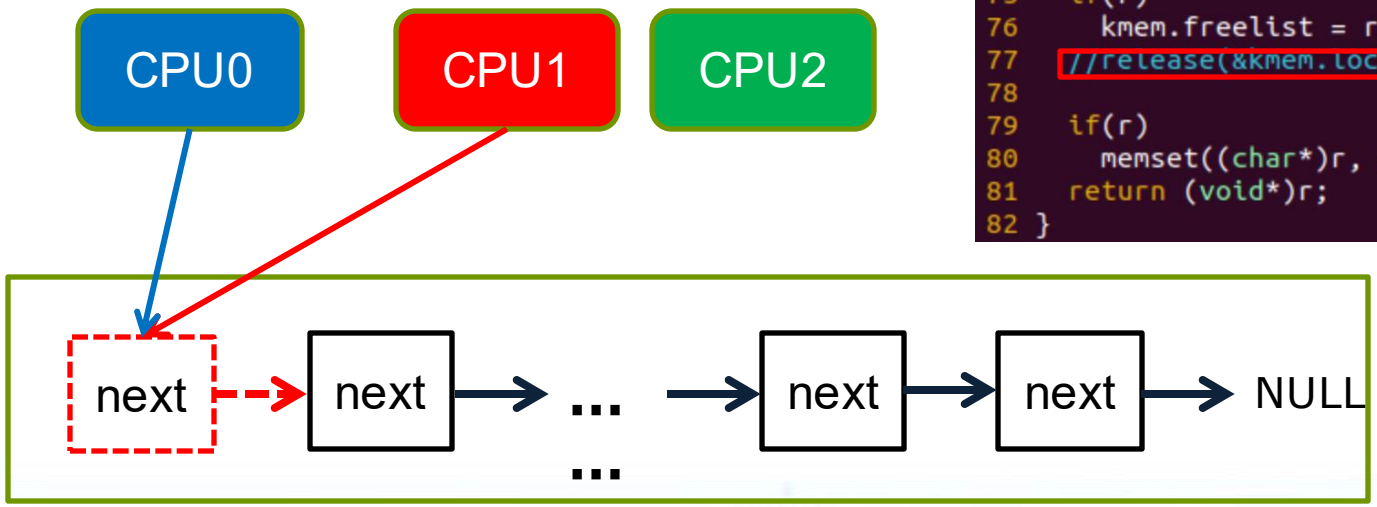


空闲链表

实验原理 | 为什么要上锁?

- 假设第73和77行的锁操作被注释掉了，此刻有CPU0和CPU1同时执行到第74行，两个CPU就会从freelist中拿出同一个内存块， 这就会导致两个进程共用一块内存

```
68 void *
69 kalloc(void)
70 {
71   struct run *r;
72
73   //acquire(&kmem.lock);
74   r = kmem.freelist;
75   if(r)
76     kmem.freelist = r->next;
77   //release(&kmem.lock);
78
79   if(r)
80     memset((char*)r, 5, PGSIZE); // fill with junk
81   return (void*)r;
82 }
```



空闲链表

实验提示

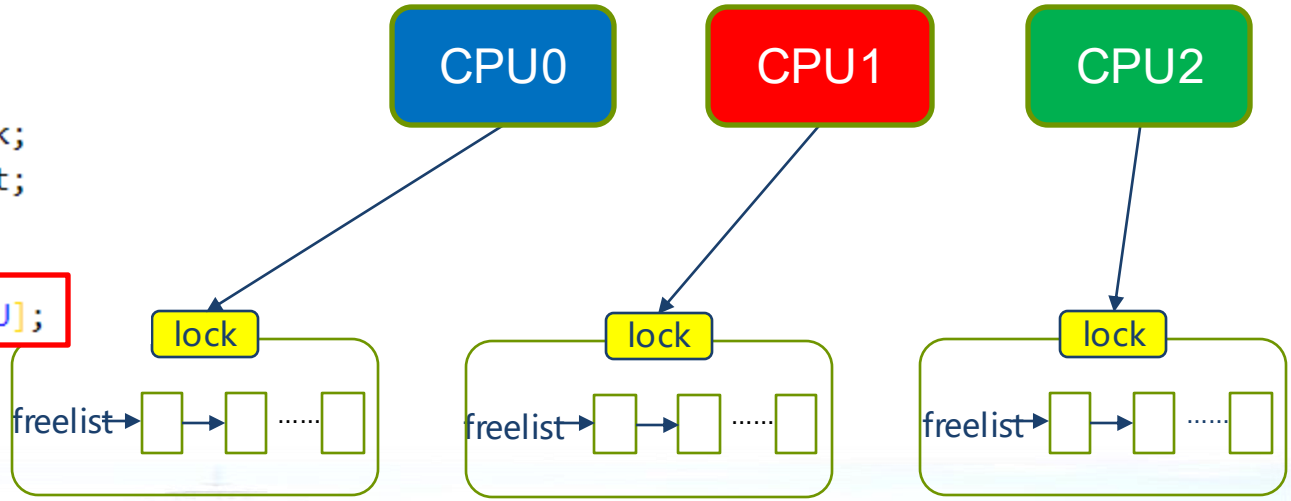
- 可以使用常量NCPU（kernel/param.h）

给每个CPU构建一个空闲页面链表（**默认只有3个CPU** （Makefile））

```
264  ifndef CPUS
265  CPUS := 3
266  endif
```

```
2   #define NCPU          8  // maximum number of CPUs
```

```
102 struct run {
103   struct run *next;
104 };
105
106 struct kmem{
107   struct spinlock lock;
108   struct run *freelist;
109 };
110
111 struct kmem kmems[NCPU];
```



实验提示

- 获取当前CPU编号（ cpuid()）

  使用cpuid函数应关闭中断。

  使用push_off()关闭中断，使用pop_off()打开中断。

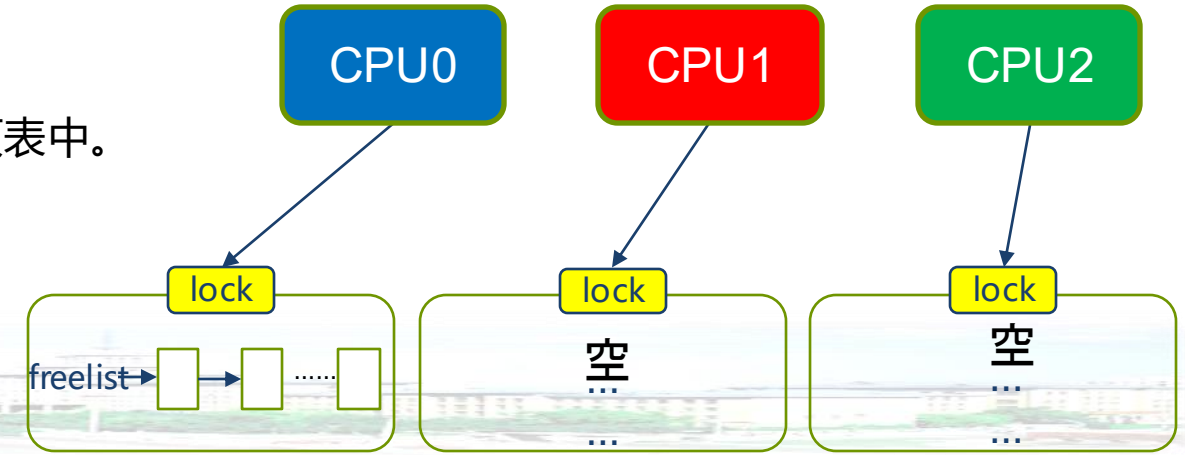
- kalloc分配内存时，假设CPU1和CPU2没有空闲页表怎么办?

  从其他CPU的空闲链表中窃取（steal）空闲内存

  窃取成功后，返回1页物理内存给调用者

- kfree释放内存时

  将该页放入当前CPU的空闲页表中。



实验提示

- 请使用initlock()初始化锁，并**要求锁名字以kmem开头**；
- 运行kalloctest查看实现的代码是否减少了锁争用（tot 没有获取到此锁总的次数小于10则为通过），并确认它可以分配所有的内存；
- 实现的代码不得影响xv6的基本功能，所以请确保usertests能够全部通过；
- 图中具体的数据会有所差别。

获取锁的次数 (lk->n)

```
$ kalloctest
start test0
test0 results:
=== lock kmem/bcache stats
lock: kmem: #test-and-set 0 #acquire() 33167
lock: kmem: #test-and-set 0 #acquire() 200114
lock: kmem: #test-and-set 0 #acquire() 199752
lock: bcache: #test-and-set 0 #acquire() 28
=== top 5 contended locks:
lock: proc: #test-and-set 22303 #acquire() 180082
lock: proc: #test-and-set 4152 #acquire() 180130
lock: proc: #test-and-set 4115 #acquire() 180129
lock: proc: #test-and-set 342 #acquire() 180070
lock: proc: #test-and-set 39 #acquire() 180070
test0 OK
start test1
total allocated                                  768)
test1 OK
$
$ usertests
...
ALL TESTS PASSED
$
```

没有获取到锁的次数（lk->nts）

usertests应显示PASSED

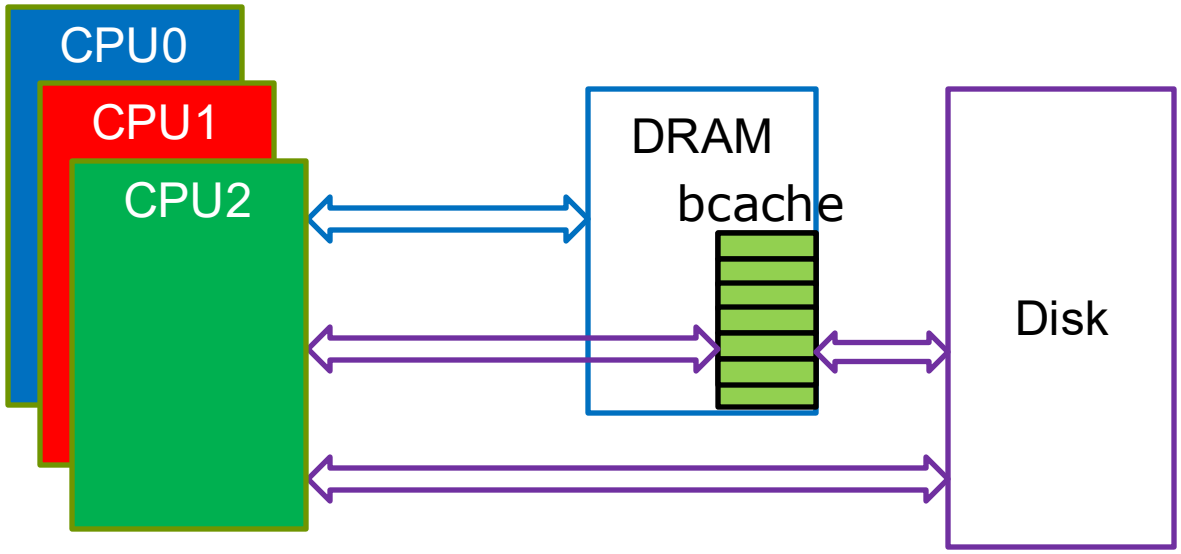
实验任务二

重新设计磁盘缓存（Buffer cache）

- 修改磁盘缓存块列表的管理机制;
- 理想情况下，块缓存中涉及的所有锁的测试与设置（test-and-set）总数应为零，但如果总数小于 500 也可以。
- 修改bget 和brelse，以便 bcache 中不同块的并发查找和释放不太可能在锁上发生冲突;
- 实验前后运行测评程序bcachetest，查看是否减少锁争用;
- 运行测评程序usertests，用例全部通过。

实验原理 | Buffer cache简介

- 在xv6文件系统中，Buffer Cache（也称为bcache）为一个磁盘与内存文件系统交互的中间层，在kernel/bio.c中实现；

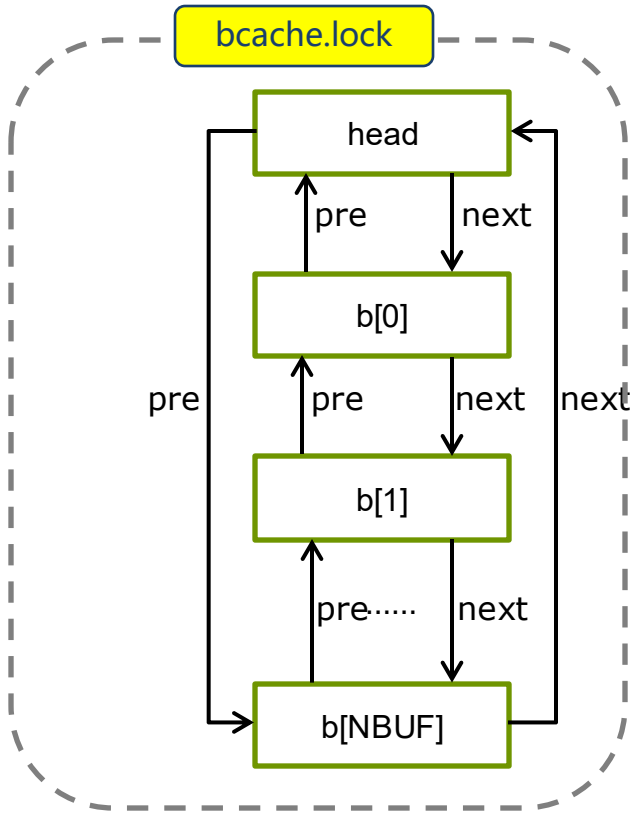


实验原理 | Buffer cache简介

- binit()构建了循环双向链表
- 数据结构bcache维护了一个由静态数组struct buf buf[NBUF]组成的双向链表，它以块为单位，每次读入或写出一个磁盘块，放到一个内存缓存块中（bcache.buf），同时自旋锁bcache.lock用于用户互斥访问。

bio.c

```
26  struct {
27    struct spinlock lock;
28    struct buf buf[NBUF];
29
30    // Linked list of all buffers, through prev/next.
31    // Sorted by how recently the buffer was used.
32    // head.next is most recent, head.prev is least.
33    struct buf head;
34  } bcache;
35
36  void
37  binit(void)
38  {
39    struct buf *b;
40
41    initlock(&bcache.lock, "bcache");
42
43    // Create linked list of buffers
44    bcache.head.prev = &bcache.head;
45    bcache.head.next = &bcache.head;
46    for(b = bcache.buf; b < bcache.buf+NBUF; b++){
47      b->next = bcache.head.next;
48      b->prev = &bcache.head;
49      initsleeplock(&b->lock, "buffer");
50      bcache.head.next->prev = b;
51      bcache.head.next = b;
52    }
53  }
```

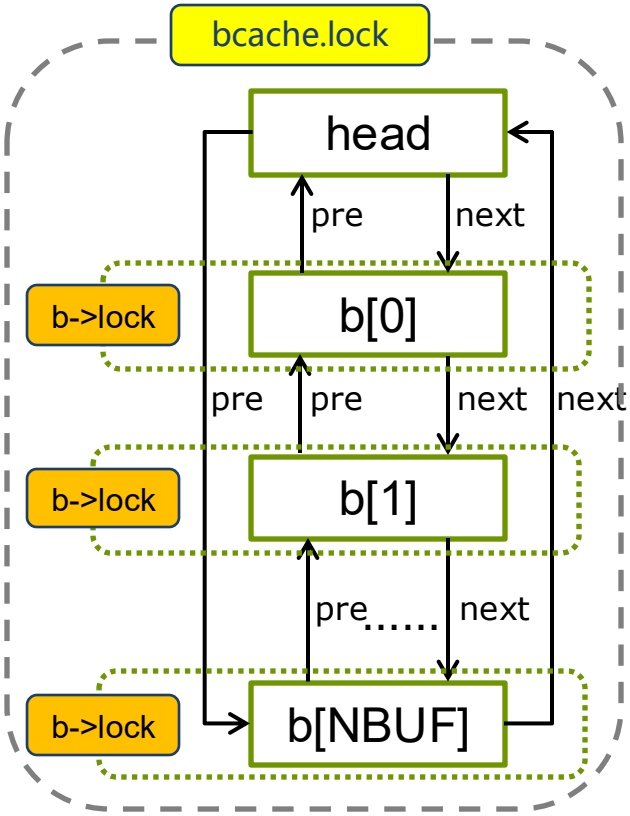


实验原理 | Buffer cache简介

- kernel/buf.h
- 静态数组struct buf buf[NBUF]定义

```
1  struct buf {
2    int valid;   // has data been read from disk?
3    int disk;    // does disk "own" buf?
4    uint dev;  //设备号
5    uint blockno; //块号
6    struct sleeplock lock; //睡眠锁
7    uint refcnt;  //引用计数
8    struct buf *prev; // LRU cache list
9    struct buf *next;
10   uchar data[BSIZE]; //数据
11 };
12
```

//前后指针



实验原理 | Buffer cache简介

- bcache.lock **自旋锁**

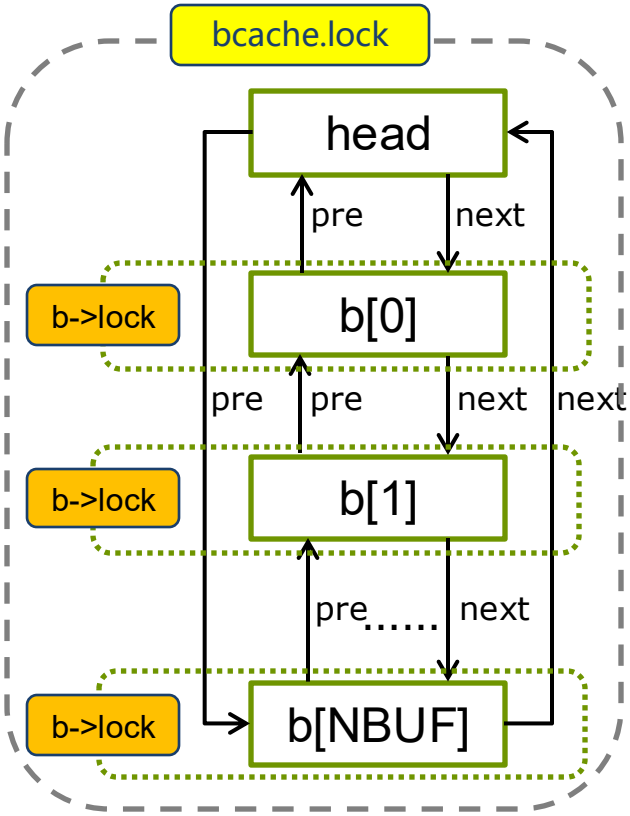
  (用于表示bcache链表是否被锁住)

  - bget()查找缓存块
  - brelse()将缓存块挂载置head.prev
  - bpin()引用缓存块
  - bunpin()去除引用缓存块

- b->lock **睡眠锁**

  (用于表示缓存数据块buf是否被锁住)

  - bget()获取缓存块
  - bwrite()将缓存块写入磁盘
  - brelse()释放缓存块



实验原理 | Buffer cache简介

bio.c:bget函数

```
58 static struct buf*
59 bget(uint dev, uint blockno)
60 {
61   struct buf *b;
62
63   acquire(&bcache.lock);
64
65   // Is the block already cached?
66   for(b = bcache.head.next; b != &bcache.head; b = b->next){
67     if(b->dev == dev && b->blockno == blockno){
68       b->refcnt++;
69       release(&bcache.lock);
70       acquiresleep(&b->lock);
71       return b;
72     }
73   }
74
75   // Not cached.
76   // Recycle the least recently used (LRU) unused buffer.
77   for(b = bcache.head.prev; b != &bcache.head; b = b->prev){
78     if(b->refcnt == 0) {
79       b->dev = dev;
80       b->blockno = blockno;
81       b->valid = 0;
82       b->refcnt = 1;
83       release(&bcache.lock);
84       acquiresleep(&b->lock);
85       return b;
86     }
87   }
88   panic("bget: no buffers");
89 }
```

bcache.lock锁

数据块有缓存

bcache.lock保护区域：缓存块缓冲区列表、每个块缓冲区中的引用计数（b->refcnt）以及缓存块的标识（b->dev和b->blockno）

数据块未缓存，重新循环查找未使用的buf（采用LRU）

实验原理 | Buffer cache简介

bio.c:brelse函数

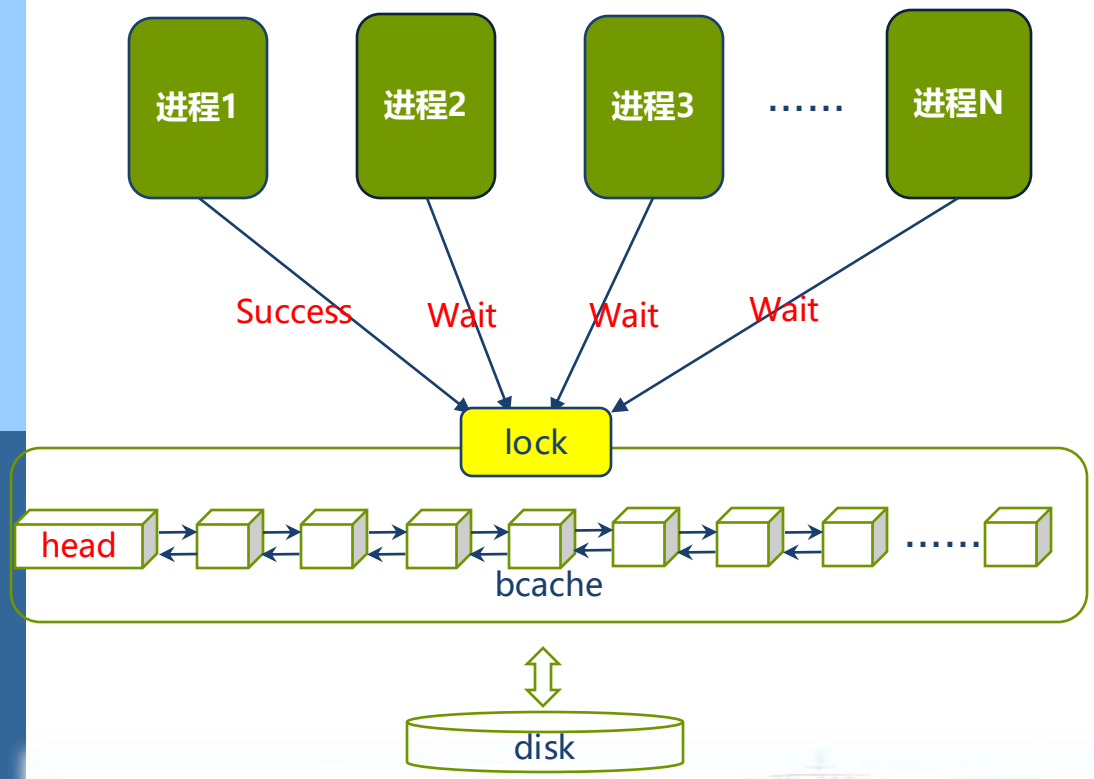
```
116 void
117 brelse(struct buf *b)
118 {
119   if(!holdingsleep(&b->lock))
120     panic("brelse");
121
122   releasesleep(&b->lock);
123
124   acquire(&bcache.lock);
125   b->refcnt--;
126   if (b->refcnt == 0) {
127     // no one is waiting for it.
128     b->next->prev = b->prev;
129     b->prev->next = b->next;
130     b->next = bcache.head.next;
131     b->prev = &bcache.head;
132     bcache.head.next->prev = b;
133     bcache.head.next = b;
134   }
135
136   release(&bcache.lock);
137 }
```

bcache.lock锁

bcache.lock保护区域
把buf移动到链表的前端

实验原理 | bcache存在问题

如果多个进程密集使用文件系统，它们可能会争用bcache.lock




```
$ bcachetest
start test0
test0 results:
--- lock kmem/bcache stats
lock: kmem: #test-and-set 0 #acquire() 33035
lock: bcache: #test-and-set 16142 #acquire() 65978
--- top 5 contended locks:
lock: virtio_disk: #test-and-set 162870 #acquire() 1188
lock: proc: #test-and-set 51936 #acquire() 73732
lock: bcache: #test-and-set 16142 #acquire() 65978
lock: uart: #test-and-set 7505 #acquire() 117
lock: proc: #test-and-set 6937 #acquire() 73420
tot= 16142
test0: FAIL
start test1
test1 OK
```

fetch-and-add：
没有成功获取到锁的次数

实验要求 | 构建哈希表

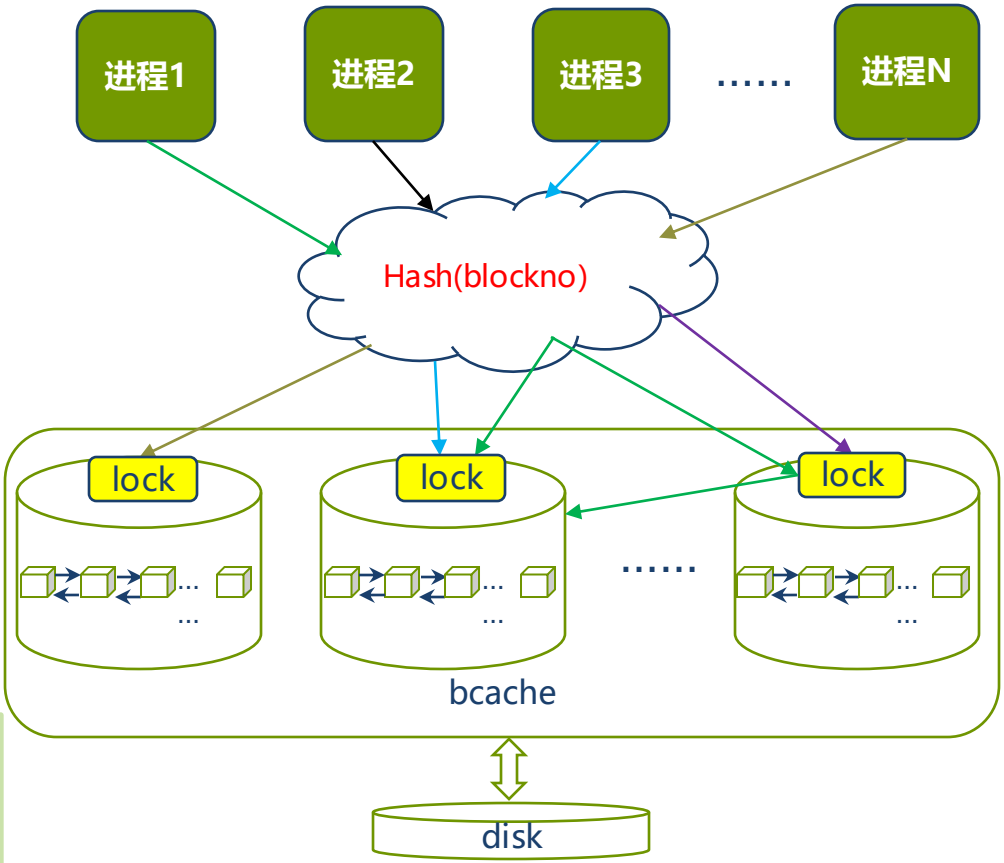
建议使用哈希表来查找块号blockno

- 以blockno为key构建哈希表和哈希桶
- 每个哈希桶（bucket）有一个锁

```
#define NBUCKETS 13

struct {
  struct spinlock lock[NBUCKETS];
  struct buf buf[NBUF];

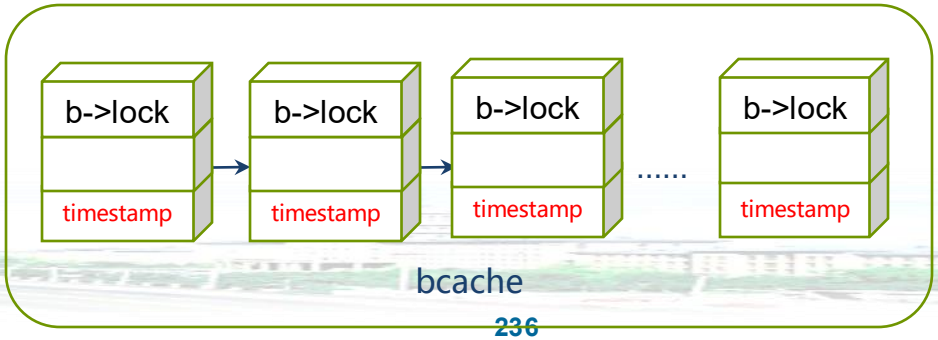
  // Linked list of all buffers, through prev/next.
  // head.next is most recently used.
  //struct buf head;
  struct buf hashbucket[NBUCKETS]; //每个哈希队列一个linked list及一个lock
} bcache;
```



当bget()查找数据块未命中时，
bget可从其他哈希桶选择一个未被使用的缓存块，
移入到当前的哈希桶链表中使用

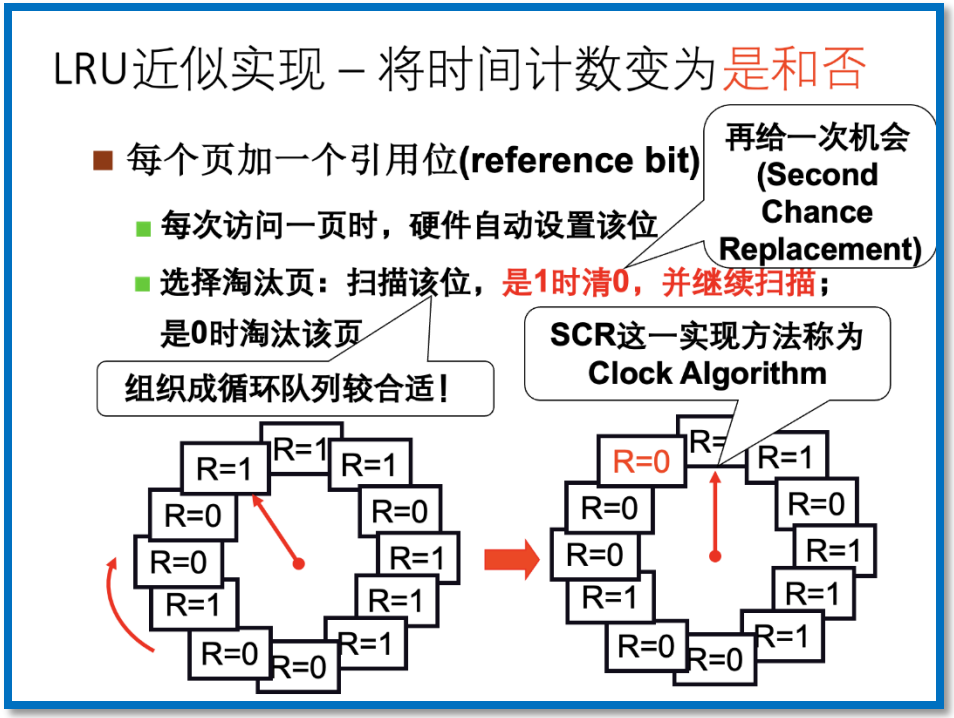
实验提示

- 可以使用固定数量的 bucket，并且不动态调整哈希表的大小。使用素数的 bucket（例如 13）可以降低哈希冲突的可能性。
- 删除所有缓冲区的列表（bcache.head等），改为使用缓冲区上次使用的时间（即使用 kernel/trap.c 中的ticks ）对缓冲区进行时间戳标记。此项改动可使brelse不再需要锁上bcache lock。
  - brelse不需要将缓存块添加到头部，只需改写时间戳即可。
- bget可通过时间戳得知最后一次被访问时间最早的空闲缓存块。
  - 循环遍历每个buf查看timestamp。



实验原理 | 优化策略（三）CLOCK算法

- 将数据结构改为使用数组构成的循环队列，每个 buf 拥有自己的锁。每个 buf 维护一个引用位。
- bget() 查找时，不需要获取对应的锁，查找到之后，锁上对应的锁，然后重新检查这个 buf 是否是想要的，检查完后进行对应的操作。
- 当 bget() 没有找到对应的缓存 buf 时，根据 CLOCK 算法维护引用位并分配一个 buf。



实验要求

- 请将本实验涉及的锁**以“bcache”开头来命名**，可以调用initlock()来初始化bcache锁；
- 理想状态下，bcachetest中数据块缓存相关的所有锁fetch-and-add的总和应该为0，但本实验中tot总和不超过500即可；
- 每个数据块最多只能维护一份缓存；
- 同样要求usertests中的用例全部通过；
- 图中具体的数据会有所差别。

```
$ bcachetest
start test0
test0 results:
=== lock kmem/bcache stats
lock: kmem: #test-and-set 0 #acquire() 32968
lock: kmem: #test-and-set 0 #acquire() 53
lock: kmem: #test-and-set 0 #acquire() 53
lock: bcache: #test-and-set 0 #acquire() 598
lock: bcache.bucket: #test-and-set 0 #acquire() 4139
lock: bcache.bucket: #test-and-set 0 #acquire() 4131
lock: bcache.bucket: #test-and-set 0 #acquire() 2360
lock: bcache.bucket: #test-and-set 0 #acquire() 4307
lock: bcache.bucket: #test-and-set 0 #acquire() 2419
lock: bcache.bucket: #test-and-set 0 #acquire() 4420
lock: bcache.bucket: #test-and-set 0 #acquire() 4934
lock: bcache.bucket: #test-and-set 18 #acquire() 8692
lock: bcache.bucket: #test-and-set 0 #acquire() 6457
lock: bcache.bucket: #test-and-set 0 #acquire() 6197
lock: bcache.bucket: #test-and-set 0 #acquire() 6191
lock: bcache.bucket: #test-and-set 0 #acquire() 6210
lock: bcache.bucket: #test-and-set 0 #acquire() 6198
=== top 5 contended locks:
lock: proc: #test-and-set 1113301 #acquire() 68753
lock: proc: #test-and-set 845107 #acquire() 68685
lock: proc: #test-and-set 822143 #acquire() 68685
lock: proc: #test-and-set 808826 #acquire() 68685
lock: proc: #test-and-set 664514 #acquire() 68727
test0: OK
start test1
test1 OK
$ usertests
  ...
ALL TESTS PASSED
$
```

实验提示

- 出现死锁的常见场景
  - 以哈希桶的优化为例，同学们一般做法都会为每个哈希桶设置一个锁。但是在需要进行替换/分配的时候，往届同学们的做法通常是内核持有一个桶的锁，然后同时抢另一个桶的锁，此时出现了同时持有**两个自旋锁**。
  - 有的同学为了解决死锁问题，设计了一种方法：先释放原有桶的锁，然后再获取另一个桶的锁。这种做法是不安全的，请大家思考为什么不安全?

1.首先获取请求对应桶的锁，查找不到以后进入替换/分配流程，释放锁。

2.然后获取全局分配大锁，获取原有桶的锁，获取新桶的锁，检查是否已完成替换，如未完成则进行替换，最后按序释放所有锁。

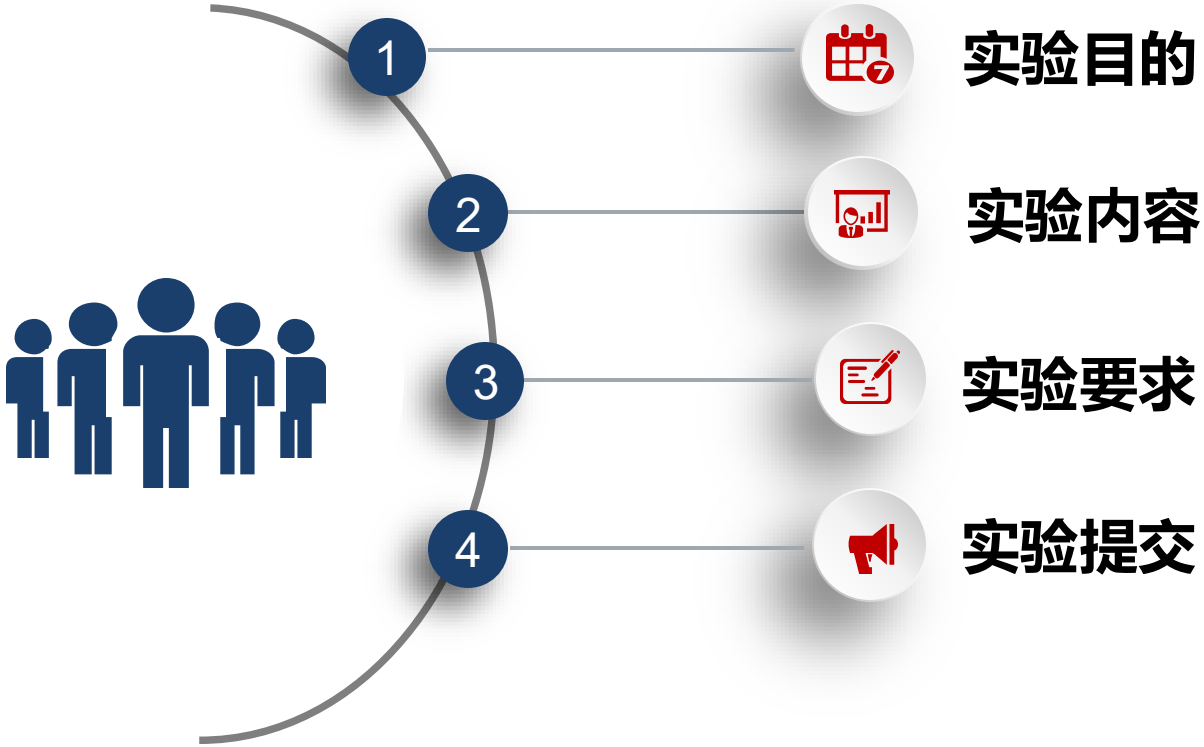
实验提交

- 请务必提前自测**grade-lab-lock**
- 实验三提交截止时间：两个星期内
- 实验提交内容（将以下内容打包提交）
  - 实验报告
  - **commit.patch文件**（使用make diff命令导出，无需提交整个实验）

下一次实验课现场验收

- 请同学们在下一次实验课前，准备好前三次实验（util、syscall、lock）的代码，助教会一对一问实验内容及代码相关的问题，请大家认真对待。

实验四：页表



实验目的

- 了解页表的实现原理。
- 修改页表，使内核更方便地进行用户虚拟地址翻译。

实验内容 | 3个实验任务

- 确保实验三已经提交
- 实验四有以下任务：

①打印页表

②独立内核页表

②简化软件模拟地址翻译

切换到 pgtbl 分支，在该分支的代码上进行实验

```
$ git checkout pgtbl
$ git fetch
```

实验原理 | 页表结构

页表是什么?

- 页表就是存放虚实地址映射的表格，里面装满了虚实地址的映射。
- xv6采用的页表标准为SV39标准，也就是虚拟地址最多为39位。

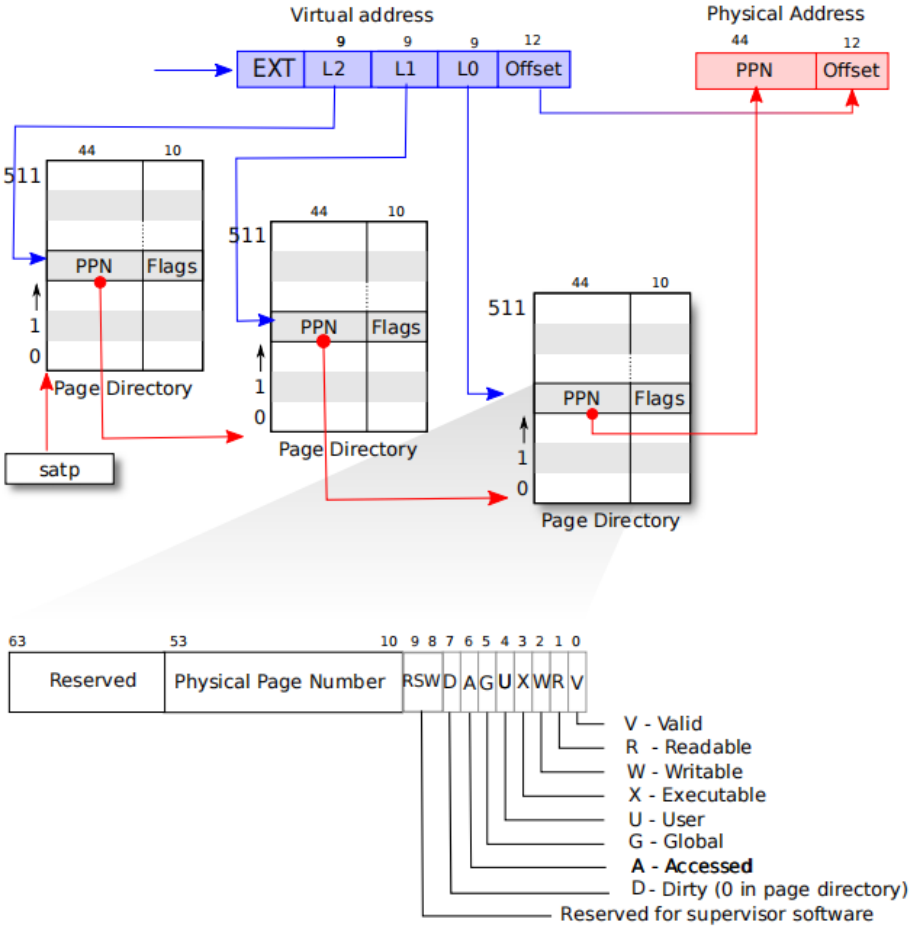


Figure 3.2: RISC-V page table hardware.

实验原理 | 页表结构

- L2：根页表的索引（index），可以得到次页表的基地址。
- L1：次页表的索引，可以得到叶子页表的基地址。
- L0：叶子页表的索引，可以得到64位的页表项。
- Offset：虚实地址的Offset完全相同，低12位刚好覆盖1个page中的每个字节。

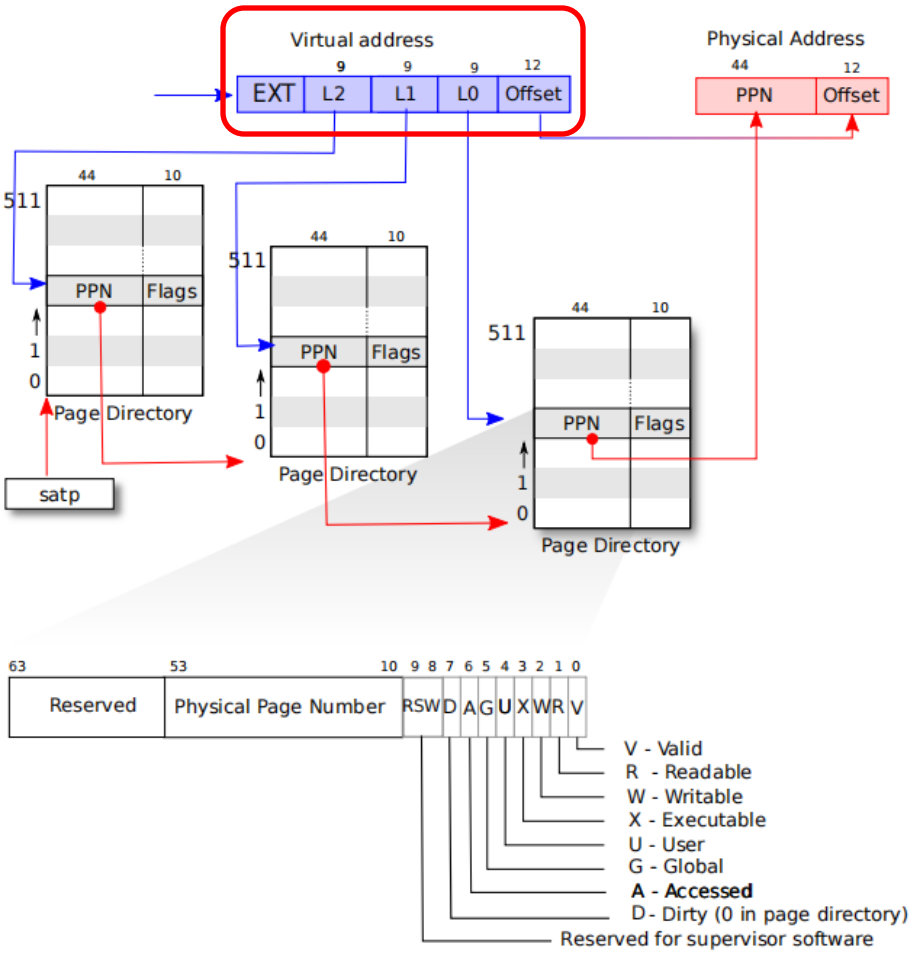


Figure 3.2: RISC-V page table hardware.

实验原理 | 页表结构

- SATP：页表首地址
- PTE：页表项
  - PPN: 物理页帧号
  - Flags标志位
    - Valid: 有效位
    - Readable/ Writable/ Executable：可读/可写/可执行
    - User：该页表项指向的物理页能在用户态访问

根页表是个4K的页，包含了512个PTE，每个PTE都对应了第二级的1个页，在第二级的每个页都包含了第三级的512个PTE。

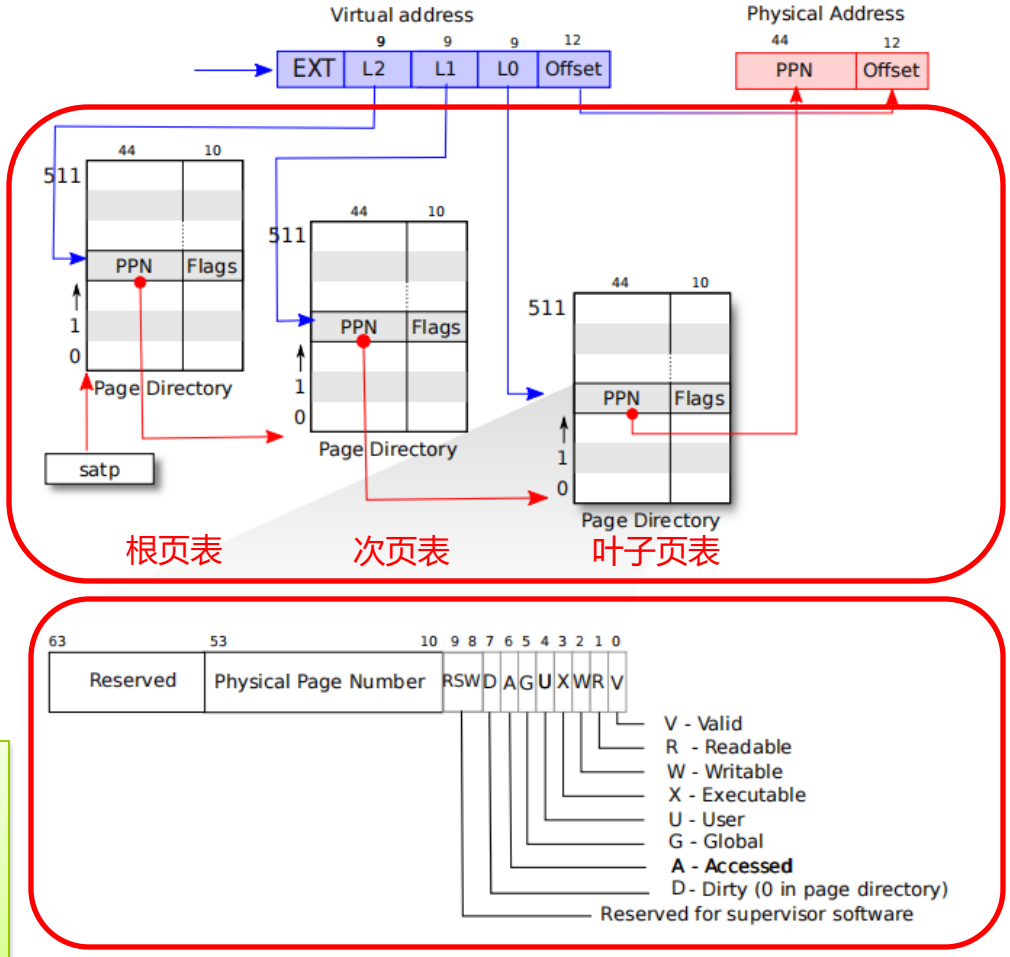


Figure 3.2: RISC-V page table hardware.

实验原理 | Xv6用户虚拟地址空间

- 范围：从0到MAXVA（256GB）

仅包含自身的代码和数据的虚实地址映射，内核代码和数据不包含在内。

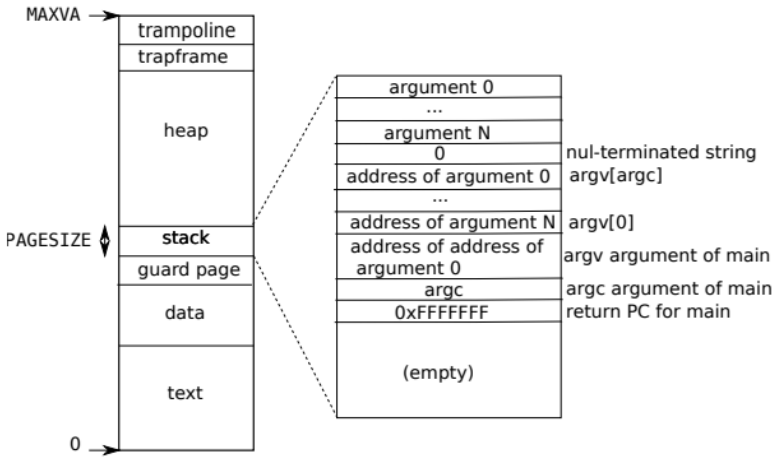


Figure 3.4: A process's user address space, with its initial stack.

- 其代码实现在kernel/exec.c。
- exec使用proc_pagetable分配了TRAMPOLINE和TRAPFRAME的页表映射，然后用uvmalloc来为每个ELF段分配内存及页表映射，并用loadseg把每个ELF段载入内存。

- **trampoline**：用户态-内核态跳板
- **trapframe**：用来存放每个进程的用户寄存器的内存空间
- **heap**：堆
- **stack**：用户栈
- **guard page**：守护页
- **data**：用户程序的数据段
- **text**：用户程序的代码段

实验原理 | Xv6内核虚拟地址空间

- 范围：从0到MAXVA（256GB）

仅包含内核的代码和数据的虚实地址映射，用户程序的代码和数据不包含在内。

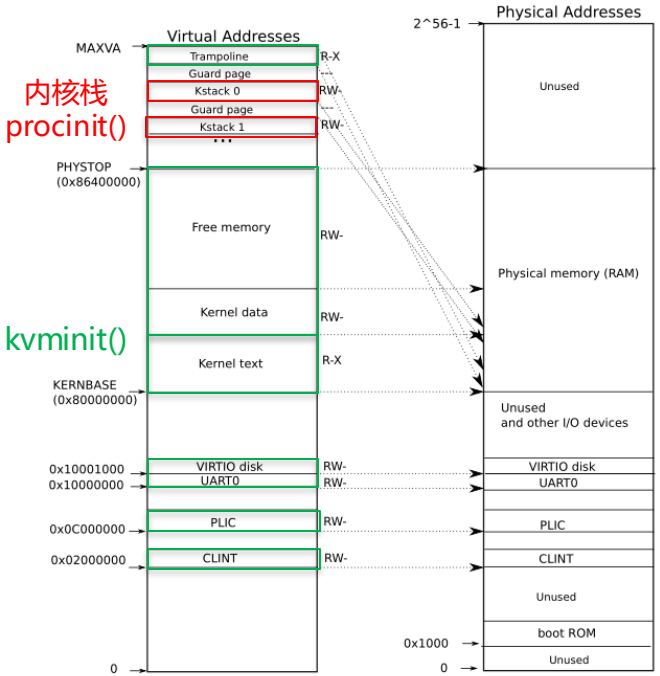


Figure 3.3: On the left, xv6's kernel address space. RWX refer to PTE read, write, and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.

- 其代码实现在kernel/vm.c的kvminit()。
- 在kvminit()函数内，除了内核栈外，完成了UART0、VIRTIO0、CLINT、PLIC、kernel text、kernel data和TRAMPOLINE的虚拟地址和物理地址的映射。

```
21  void
22  kvminit()
23  {
24    kernel_pagetable = (pagetable_t) kalloc();
25    memset(kernel_pagetable, 0, PGSIZE);
26
27    // uart registers
28    kvmmap(UART0, UART0, PGSIZE, PTE_R | PTE_W);
29
30    // virtio mmio disk interface
31    kvmmap(VIRTIO0, VIRTIO0, PGSIZE, PTE_R | PTE_W);
32
33    // CLINT
34    kvmmap(CLINT, CLINT, 0x10000, PTE_R | PTE_W);
35
36    // PLIC
37    kvmmap(PLIC, PLIC, 0x400000, PTE_R | PTE_W);
38
39    // map kernel text executable and read-only.
40    kvmmap(KERNBASE, KERNBASE, (uint64)etext-KERNBASE, PTE_R | PTE_X);
41
42    // map kernel data and the physical RAM we'll make use of.
43    kvmmap((uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R | PTE_W);
44
45    // map the trampoline for trap entry/exit to
46    // the highest virtual address in the kernel.
47    kvmmap(TRAMPOLINE, (uint64)trampoline, PGSIZE, PTE_R | PTE_X);
48  }
```

实验任务一：打印页表

void vmprint(pagetable_t pgtbl)

- 定义一个名为vmprint() 的函数。它应该接受一个pagetable_t参数，并以右图描述的格式打印该页表。
- 在 exec.c 中的return argc之前插入if(p->pid==1) vmprint(p->pagetable)，以打印第一个进程的页表。
- 通过make grade的pte 打印输出测试。

```
page table 0x0000000087f25000
||idx: 0: pa: 0x0000000087f21000, flags: ----
|| ||idx: 0: pa: 0x0000000087f20000, flags: ----
|| || ||idx: 0: va: 0x0000000000000000 -> pa: 0x0000000087f22000, flags: rwxu
|| || ||idx: 1: va: 0x0000000000001000 -> pa: 0x0000000087f1f000, flags: rwx-  Guard page
|| || ||idx: 2: va: 0x0000000000002000 -> pa: 0x0000000087f1e000, flags: rwxu
||idx: 255: pa: 0x0000000087f24000, flags: ----
|| ||idx: 511: pa: 0x0000000087f23000, flags: ----
|| || ||idx: 510: va: 0x0000003fffffe000 -> pa: 0x0000000087f76000, flags: rw--
|| || ||idx: 511: va: 0x0000003ffffff000-> pa: 0x0000000080007000, flags: r-x-
```

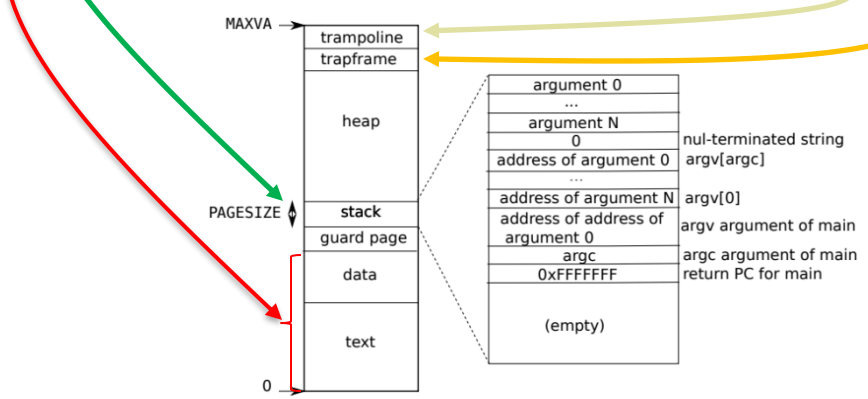


Figure 3.4: A process's user address space, with its initial stack.

如果是非叶节点，则为：*idx: [索引编号]: pa: [物理地址], flags: [四个权限位(r/w/x/u)]*
如果是叶子节点，则为：*idx: [索引编号]: va: [虚拟地址] -> pa: [物理地址], flags: [四个权限位(r/w/x/u)]*

实验提示

- 可以将vmprint()放在kernel/vm.c中。
- 使用文件 kernel/riscv.h 末尾的宏。
- 可以参考freewalk函数。
- 在 kernel/defs.h 中定义vmprint的原型，以便可以从 exec.c 调用它。
- 在 printf 调用中使用%p来打印出完整的 64 位十六进制 PTE 和地址。

kernel/vm.c的freewalk函数

```
228 // Recursively free page-table pages.
229 // All leaf mappings must already have been removed.
230 void freewalk(pagetable_t pagetable) {
231   // there are 2^9 = 512 PTEs in a page table.
232   for (int i = 0; i < 512; i++) {  //遍历页表的PTE
233     pte_t pte = pagetable[i];
234     if ((pte & PTE_V) && (pte & (PTE_R | PTE_W | PTE_X)) == 0) {
235       // this PTE points to a lower-level page table.
236       uint64 child = PTE2PA(pte);
237       freewalk((pagetable_t)child);    //递归调用
238       pagetable[i] = 0;
239     } else if (pte & PTE_V) {
240       panic("freewalk: leaf");
241     }
242   }
243   kfree((void *)pagetable);
244 }
```

实验任务二

独立内核页表

- **将共享内核页表改成独立内核页表**，使得每个进程拥有自己独立的内核页表。
- 虚实地址相同的映射应该要保留，**先不需要**加上用户页表的内容，在**任务三**中再加上用户页表的内容。
- 首先在xv6上运行测试程序kvmtest，然后再运行usertests，确保所有测试通过。

```
$ kvmtest
kvmtest: start
kvmtest: OK
$
```

实验实现 | 任务2：独立内核页表

- 将共享内核页表改成独立内核页表，使得每个进程拥有自己独立的内核页表
- ***参考步骤:***
  - Step1：修改kernel/proc.h中的 struct proc， 增加两个新成员：
    内核独立页表k_pagetable; 内核栈的物理地址kstack_pa;，
  - Step2：仿照kvminit函数写一个创建内核页表的函数

注：实现的时候不要映射CLINT，否则会发生地址重合问题。

为什么要保留初始内核页表?

保留原有的kvminit()以及kernel/vm.c中kernel_pagetable，因为有些时候CPU可能并未执行用户进程。

实验实现 | 任务2：独立内核页表

- Step3：修改procinit函数。procinit()是在系统引导时，用于给进程分配内核栈的物理页并在页表建立映射。

参考优化方法:

把procinit()中内核栈的物理地址pa拷贝到PCB新增的成员**kstack_pa**中，同时还需要保留内核栈在全局页表kernel_pagetable的映射，然后在allocproc()中再把它映射到进程的内核页表里。

```
24  // initialize the proc table at boot time.
25  void
26  procinit(void)
27  {
28    struct proc *p;
29
30    initlock(&pid_lock, "nextpid");
31    for(p = proc; p < &proc[NPROC]; p++) {
32        initlock(&p->lock, "proc");
33
34        // Allocate a page for the process's kernel stack.
35        // Map it high in memory, followed by an invalid
36        // guard page.
37        char *pa = kalloc();
38        if(pa == 0)
39          panic("kalloc");
40        uint64 va = KSTACK((int) (p - proc));
41        kvmmap(va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
42        p->kstack = va;
43    }
44    kvminithart();
45  }
```

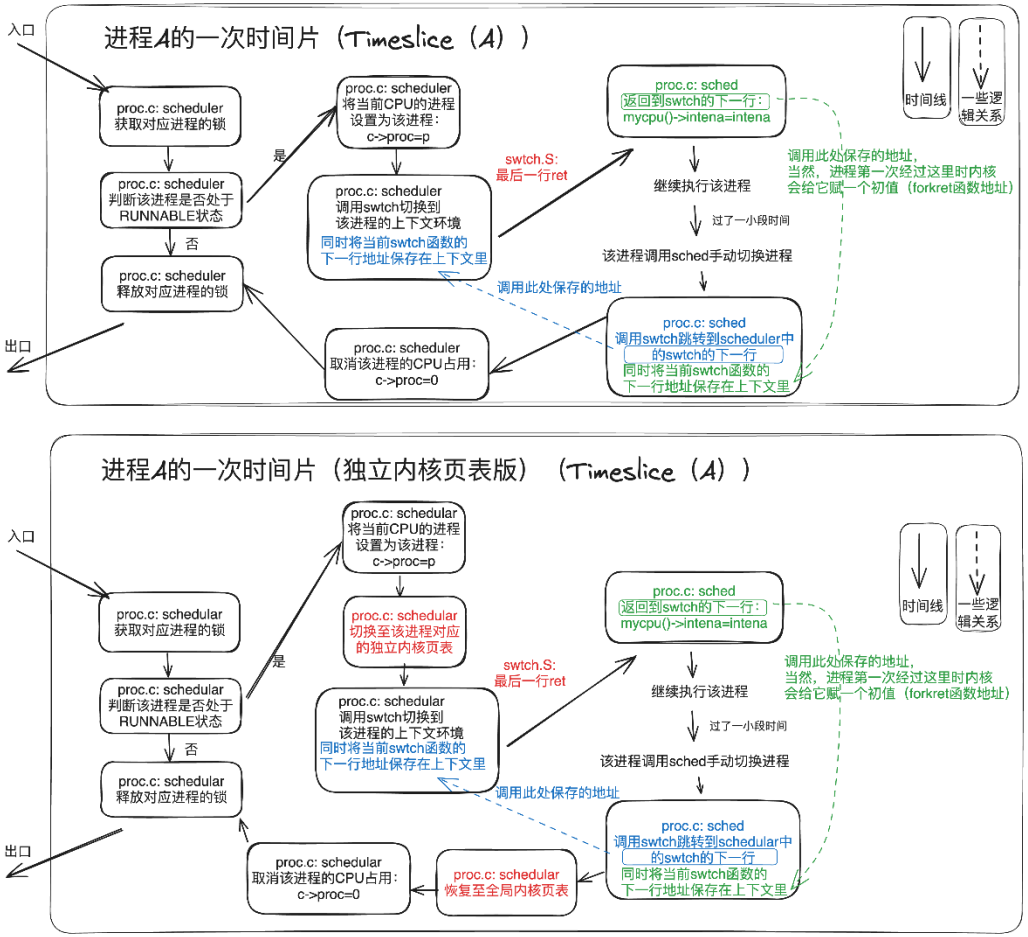
实验实现 | 任务2：独立内核页表

- Step4：修改allocproc函数。allocproc()会在系统启动时被第一个进程和fork调用。
- ***allocproc函数功能***：在进程表中查找空闲PCB，如果找到，初始化在内核中运行所需的状态，并保持p->lock返回。如果没有空闲PCB，或者内存分配失败，则返回0。
- 在allocproc创建内核页表以及映射内核栈
  - 调用Step2创建的内核页表映射函数
  - 调用kvmmap映射PCB新增的成员kstack_pa

```
88  // Look in the process table for an UNUSED proc.
89  // If found, initialize state required to run in the kernel,
90  // and return with p->lock held.
91  // If there are no free procs, or a memory allocation fails, return 0.
92  static struct proc*
93  allocproc(void)
94  {
95    struct proc *p;
96
97    for(p = proc; p < &proc[NPROC]; p++) {
98      acquire(&p->lock);
99      if(p->state == UNUSED) {
100       goto found;
101     } else {
102       release(&p->lock);
103     }
104   }
105   return 0;
106
107 found:
108   p->pid = allocpid();
109
110   // Allocate a trapframe page.
111   if((p->trapframe = (struct trapframe *)kalloc()) == 0){
112     release(&p->lock);
113     return 0;
114   }
115
116   // An empty user page table.
117   p->pagetable = proc_pagetable(p);
118   if(p->pagetable == 0){
119     freeproc(p);
120     release(&p->lock);
121     return 0;
122   }
```

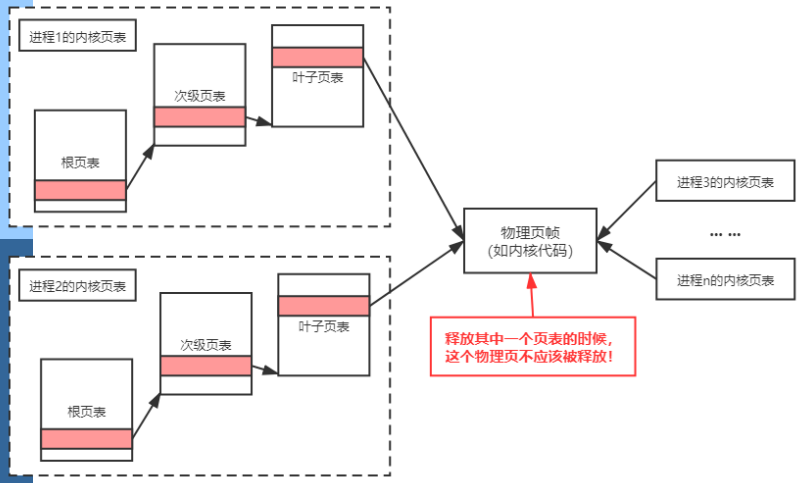
实验实现 | 任务2：独立内核页表

- Step5：修改调度器中的**scheduler()函数**，使得切换进程的时候切换内核页表。
- ***参考方法***：在进程切换的同时也要切换页表将其放入寄存器satp中（一定要借鉴kvminithart()的页表载入方式）



实验实现 | 任务2：独立内核页表

- Step6：修改freeproc()函数，释放对应的内核页表。
- ***参考方法***：在各个内核页表的叶子页表中，页表项指向了共享的物理页。你需要找到 **释放页表但不释放叶子页表指向的物理页帧** 的方法。你可参考kernel/vm.c中的freewalk，其用于释放整个页表，但要求叶子页表的页项已经被清空。




```
136  static void
137  freeproc(struct proc *p)
138  {
139    if(p->trapframe)
140      kfree((void*)p->trapframe);
141    p->trapframe = 0;
142    if(p->pagetable)
143      proc_freepagetable(p->pagetable, p->sz);
144    p->pagetable = 0;
145    p->sz = 0;
146    p->pid = 0;
147    p->parent = 0;
148    p->name[0] = 0;
149    p->chan = 0;
150    p->killed = 0;
151    p->xstate = 0;
152    p->state = UNUSED;
153  }
```

```
251  void
252  freewalk(pagetable_t pagetable)
253  {
254    // there are 2^9 = 512 PTEs in a page table.
255    for(int i = 0; i < 512; i++){
256      pte_t pte = pagetable[i];
257      if((pte & PTE_V) && (pte & (PTE_R|PTE_W|PTE_X)) == 0){
258        // this PTE points to a lower-level page table.
259        uint64 child = PTE2PA(pte);
260        freewalk((pagetable_t)child);
261        pagetable[i] = 0;
262      } else if(pte & PTE_V){
263        panic("freewalk: leaf");
264      }
265    }
266    kfree((void*)pagetable);
267  }
```

实验任务三

简化软件模拟地址翻译

- 在独立内核页表**加上用户地址空间**的映射，同时将函数 copyin()/copyinstr()替换成copyin_new()/copyinstr_new()，即将软件模拟地址翻译改成直接访问。
- 首先在xv6上运行测试程序stats stats，然后再运行usertests，确保所有测试通过。

```
$ stats stats
copyin: 62
copyinstr: 12
$
```

实验任务三

什么情况下需要软件模拟地址翻译?

- 内核有自己的page table，用户进程也有自己的page table。当执行系统调用read/write等函数，陷入内核态时，内核需要将用户态传入的读/写buf指针翻译成一个内核可以读写的物理地址，即**软件模拟地址翻译**。

```
// Return the address of the PTE in page table pagetable
// that corresponds to virtual address va.  If alloc!=0,
// create any required page-table pages.
//
// The risc-v Sv39 scheme has three levels of page-table
// pages. A page-table page contains 512 64-bit PTEs.
// A 64-bit virtual address is split into five fields:
//   39..63 -- must be zero.
//   30..38 -- 9 bits of level-2 index.
//   21..29 -- 9 bits of level-1 index.
//   12..20 -- 9 bits of level-0 index.
//    0..11 -- 12 bits of byte offset within the page.
pte_t *walk(pagetable_t pagetable, uint64 va, int alloc) {
  if (va >= MAXVA) panic("walk");

  for (int level = 2; level > 0; level--) {
    pte_t *pte = &pagetable[PX(level, va)];
    if (*pte & PTE_V) {
      pagetable = (pagetable_t)PTE2PA(*pte);
    } else {
      if (!alloc || (pagetable = (pde_t *)kalloc()) == 0) return 0;
      memset(pagetable, 0, PGSIZE);
      *pte = PA2PTE(pagetable) | PTE_V;
    }
  }
  return &pagetable[PX(0, va)];
}
```

实验原理 | 用户页表和内核页表合并

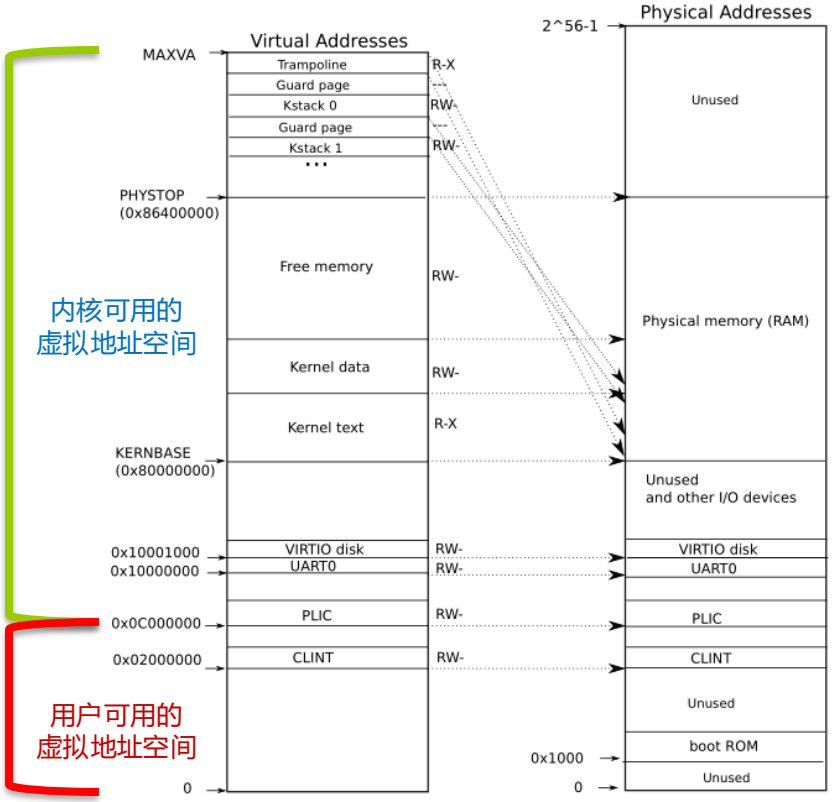


Figure 3.3: On the left, xv6's kernel address space. RWX refer to PTE read, write, and execute permissions. On the right, the RISC-V physical address space that xv6 expects to see.

来自《xv6 book》

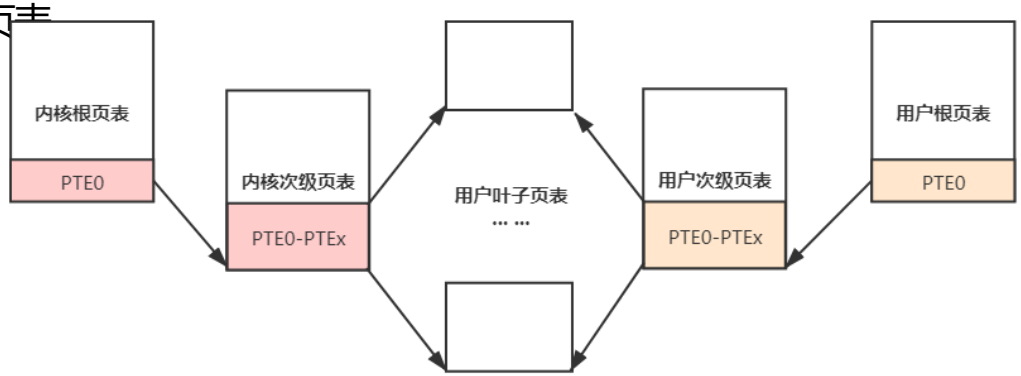
前提：地址不重合

- xv6只有在内核初始化时用到CLINT，因此为用户进程生成内核页表时，可以不必映射这段地址。
- 用户页表是从虚拟地址0开始，用多少建多少，但最高不能超过内核的起始地址（即PLIC地址）。
- **内核的可用虚拟地址空间：**
  0xC000000~MAXVA
- **用户的可用虚拟地址空间：**
  0~0xC000000

实验实现 | 任务3：简化软件模拟地址翻译

- 在独立内核页表加上用户页表的映射，同时替换 copyin()/copyinstr()为 copyin_new()/copyinstr_new()
- ***参考步骤:***
  - Step1：写一个sync_pagetable函数把进程的用户页表映射到内核页表中，同时在defs.h中声明。
  - 推荐一种较为***优雅的实现方法***：内核页表直接**共享**用户页表的叶子页表，即内核页表中次页表的部分目录**直接指向**用户页表的叶子页表

提示：前面已经提到用户地址空间的范围为0x0-0xC000000，试计算，多少个次级页表项就能涵盖整个用户地址空间?



实验实现 | 任务3：简化软件模拟地址翻译

- Step2：用函数 copyin_new() （在 kernel/vmcopyin.c中定义）代替 copyin()（在 kernel/vm.c中定义）。确保程序能运行之后再用 copyinstr_new() 以代替 copyinstr()。
- Step3：注意独立内核页表的用户页表的映射的标志位的选择。

  （标志位User一旦被设置，内核就不能访问该虚拟地址了）

  - ***推荐方案：*** 在调用copyin_new()/ copyinstr_new()

    之前修改sstatus寄存器的SUM位：*w_sstatus(r_sstatus() | SSTATUS_SUM);*
    之后去掉sstatus寄存器的SUM位：*w_sstatus(r_sstatus() & ~SSTATUS_SUM);*

```
356 int
357 copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
358 {
359   uint64 n, va0, pa0;
360
361   while(len > 0){
362     va0 = PGROUNDDOWN(srcva);
363     pa0 = walkaddr(pagetable, va0);
364     if(pa0 == 0)
365       return -1;
366     n = PGSIZE - (srcva - va0);
367     if(n > len)
368       n = len;
369     memmove(dst, (void *)(pa0 + (srcva - va0)), n);
370
371     len -= n;
372     dst += n;
373     srcva = va0 + PGSIZE;
374   }
375   return 0;
376 }
```

```
29  int
30  copyin_new(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
31  {
32    struct proc *p = myproc();
33
34    if (srcva >= p->sz || srcva+len >= p->sz || srcva+len < srcva)
35      return -1;
36    memmove((void *) dst, (void *)srcva, len);
37    stats.ncopyin++;   // XXX lock
38    return 0;
39  }
```

实验实现 | 任务3：简化软件模拟地址翻译

- Step4：在xv6中，涉及到进程页表改变的只有三个地方：**fork(), exec(), sbrk()**，要将改变后的进程页表同步到内核页表中。
- Step5：注意：第一个用户进程也需要将用户页表映射到内核页表中（kernel/proc.c: **userinit()**）
- Step6：通过测评测程序stats 和 usertests 的测试

```
$ stats stats
copyin: 62
copyinstr: 12
$
```

实验任务三

- 当完成上述三个实验后，在命令行输入make grade进行测试

```
== Test pte printout ==
$ make qemu-gdb
pte printout: OK (5.5s)
== Test count copyin ==
$ make qemu-gdb
count copyin: OK (1.1s)
== Test kernel pagetable test ==
$ make qemu-gdb
kernel pagetable test: OK (1.0s)
== Test usertests ==
$ make qemu-gdb
(178.8s)
== Test   usertests: copyin ==
  usertests: copyin: OK
== Test   usertests: copyinstr1 ==
  usertests: copyinstr1: OK
== Test   usertests: copyinstr2 ==
  usertests: copyinstr2: OK
== Test   usertests: copyinstr3 ==
  usertests: copyinstr3: OK
== Test   usertests: sbrkmuch ==
  usertests: sbrkmuch: OK
== Test   usertests: all tests ==
  usertests: all tests: OK
Score: 100/100
```

实验要点 | 注意事项

- 好好利用 vmprint() 来帮助debug。
- 如果内核缺失了地址映射造成了页缺失（page fault），通常会打印个sepc=0x00000000XXXXXXXX，这代表的是出错时pc的值，你可以查kernel/kernel.asm看看对应地址的代码的含义。**可参考常见问题来找bug**
- 请认真阅读实验指导书 http://hitsz-cslab.gitee.io/os-labs/lab4/part1/

实验要点 | 作业提交

- 请务必提前自测grade-lab-pgtbl;
- 实验四提交截止时间：两个星期内，请参考实验二的提交方式;
  - 实验报告
  - **commit.patch文件**
- **注意查看作业提交截止时间！！！**

谢 谢!